Fundamentos de Programação para a Web



Cláudio Luís Vieira Oliveira
claudio.oliveira@cps.sp.gov.br

Índice

HTML....4
- Cabeçalhos....6
- Parágrafos....7
- Fontes....8
- Imagens....9
- Links....10
- Listas....13
- Tabelas....15
- Exercícios....16

Cascading Style Sheets....17
- CSS Inline....17
- CSS Interno....18
- CSS Externo....19
- Seletores e classes....21
- Layout: Dimensão e posicionamento....22
- Exercícios....25

Linguagens de programação para a Web....27

JavaScript....28
- Primeiros scripts....28
- Caixas de mensagem e diálogo....30
- Formulários: Introdução....32
- Eventos....32
- Containers....33
- Variáveis em JavaScript....34
- Operadores....37
- Depuração de Scripts....39
- Formulários: Caixas de texto e botões....42
- Estruturas de seleção e repetição....43
- Calculadora básica....47
- Vetores....51
- Exibição de imagens....54
- Interação com CSS....56
- Exercícios....57
- Formulários: Botões de rádio....58
- Formulários: Caixas de seleção....60

JavaScript e CSS externos....62
Exercícios....65
Interação entre páginas....67
Formulários: Validação....69
Exercícios....76
Requisições....78
Primitivas de Desenho: Objetos básicos....80
Primitivas de Desenho: Imagens e animações....85
Controle do mouse....88
JavaScript Object Notation (JSON)....98
Manipulação em JavaScript....98
Exercícios....103
jQuery....104
Olá jQuery....104
Efeitos....105
AJAX (Asynchronous Javascript and XML)....107
Application Programming Interface (API)....109
Exercícios....114
Bootstrap....115
Olá Bootstrap....115
Design responsivo....117
Formulários com Bootstrap....119
Exercícios....123

HTML

A linguagem HTML é a base de criação de qualquer página para a Web, porém trata-se de uma linguagem que é composta por elementos estáticos e está fortemente focada na composição e formatação de documentos. HTML é a sigla para *HyperText Markup Language* ou Linguagem de Formatação de Hipertexto, trata-se de uma linguagem de marcação utilizada para criar páginas na Internet. De modo geral, são documentos de texto escritos em códigos que podem ser interpretados pelos navegadores que, por sua vez, exibem as páginas devidamente formatadas. Todo documento HTML apresenta os comandos delimitados pelos sinais de maior e menor, sendo esses elementos denominados *tags* ou etiquetas, e que correspondem aos comandos de formatação da linguagem. A maioria das etiquetas tem sua correspondente de fechamento que é identificado pelo uso do caracter / (barra), por exemplo:



```
<h1>Olá</h1>
```

Uma *tag* também pode ser formada por um conjunto de atributos e valores. Os atributos modificam os resultados padrões dos comandos e os valores caracterizam essa mudança, por exemplo, temos o atributo color que possibilita mudar a cor da letra, conforme ilustra o seguinte trecho de código-fonte.



```
<font color='blue'>Olá</font>
```

De maneira geral a HTML é uma linguagem muito simples e acessível para a produção e compartilhamento de documentos.


Atenção!

Recomendamos o uso do **Visual Studio Code** e do **XAMPP** para o desenvolvimento dos exemplos contidos nesta obra. O Visual Studio code pode ser gratuitamente obtido em https://code.visualstudio.com, enquanto que o XAMPP está disponível, também de forma gratuita, em https://www.apachefriends.org/pt_br/index.html.

O exemplo a seguir mostra uma página contendo apenas elementos da HTML também conhecidos como *tags* ou etiquetas.



```
<!DOCTYPE html>
<html>
  <head>
    <title>Primeira página</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Olá pessoal</h1>
  </body>
</html>
```

ola.html

Note que as etiquetas devem ser usadas de modo hierárquico, isto é, a etiqueta html foi a primeira a ser aberta então, deverá ser a última a ser fechada. A etiqueta head, por sua vez, deve estar dentro da etiqueta html e deverá ser fechada antes da etiqueta html e assim sucessivamente.

Ao executarmos essa página o navegador Internet é ativado e ele solicita ao servidor web o envio dela. O resultado deste processo deve ser similar ao mostrado pela Figura 1 a seguir:



Figura 1: Exemplo de página em HTML

Como já mencionado anteriormente a base para a construção de aplicações para a web é a linguagem HTML, porém basicamente realiza a formatação de documentos não apresentando conceitos das linguagens de programação como variáveis e estruturas de controle, entre outros. Desta forma, para se criar aplicações mais complexas e dinâmicas, torna-se necessário o uso de outra linguagem de programação que irá complementar o HTML.

A seguir iremos detalhar a utilização das etiquetas mais comumente utilizadas em páginas HTML. Porém, muitas outras estão disponíveis e podem ser facilmente encontradas na Internet através de sites de busca.



Dica!

Uma ótima referência é a **Mozilla Developer Network**, acesse https://developer.mozilla.org/pt-BR/docs/Web/HTML

Cabeçalhos

As etiquetas de cabeçalho são usadas para destacar determinados títulos e subtítulos. São identificados pelas etiquetas <h1> até <h6>.

```
<html>
  <head>
    <title>Cabeçalhos</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Cabeçalho 1</h1>
    <h2>Cabeçalho 2</h2>
    <h3>Cabeçalho 3</h3>
    <h4>Cabeçalho 4</h4>
    <h5>Cabeçalho 5</h5>
    <h6>Cabeçalho 6</h6>
    Texto com a formatação padrão (sem destaque).
  </body>
</html>
```

cabecalho.html

Na Figura 2 temos a aparência da página após ser carregada pelo navegador.

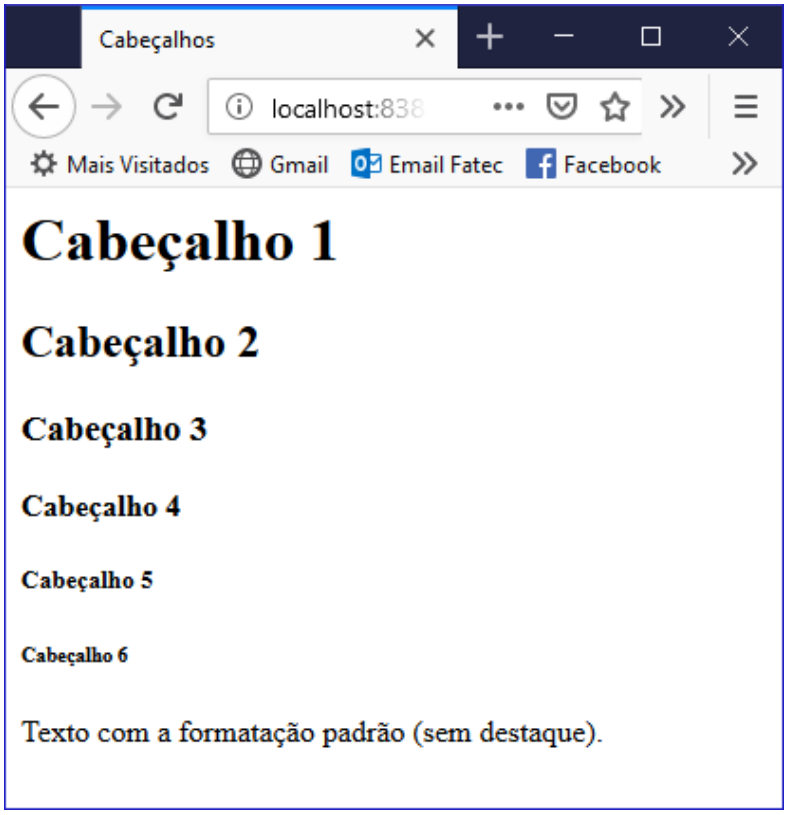


Figura 2: Cabeçalhos

Parágrafos

A etiqueta de parágrafo permite agrupar e realizar o alinhamento do texto.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Parágrafos</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Parágrafos</h1>
    <p>
      A ligeira raposa marrom salta sobre o cão preguiçoso. A ligeira raposa marrom salta sobre o cão preguiçoso. A ligeira raposa marrom salta sobre o cão preguiçoso.
    </p>
    <p align="right">
      A ligeira raposa marrom salta sobre o cão preguiçoso. A ligeira raposa marrom salta sobre o cão preguiçoso. A ligeira raposa marrom salta sobre o cão preguiçoso.
    </p>
    <p align="center">
      A ligeira raposa marrom salta sobre o cão preguiçoso. A ligeira raposa marrom salta sobre o cão preguiçoso. A ligeira raposa marrom salta sobre o cão preguiçoso.
    </p>
    <p align="justify">
      A ligeira raposa marrom salta sobre o cão preguiçoso. A ligeira raposa marrom salta sobre o cão preguiçoso. A ligeira raposa marrom salta sobre o cão preguiçoso.
    </p>
  </body>
</html>
```

paragrafo.html

Na Figura 3 mostramos os parágrafos alinhados à esquerda (padrão), direita, centro e justificado.

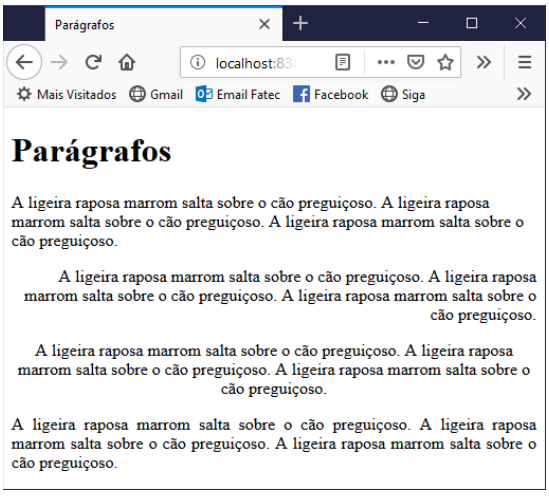


Figura 3: Parágrafos

Fontes

É possível mudar a fonte de letra, tamanho e cor de um texto através da utilização da etiqueta font. Também existem as etiquetas para itálico (i), negrito (b ou strong) e sublinhado (u).

```
<!DOCTYPE html>
<html>
  <head>
    <title>Fontes</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Fontes</h1>
    HTML possui um conjunto de etiquetas (tags) que possibilitam a
formatação do texto de uma página. As etiquetas mais usadas para
realizar a formação são:<br />
    <b>b - Negrito</b><br />
    <strong>strong - Reforçado</strong><br />
    <i>i - Itálico</i><br />
    <u>u - Sublinhado</u><br /><br />
    A etiqueta font permite a alteração do tipo da fonte de letra,
tamanho e cor do texto, por exemplo:<br />
    <font size="+2" color="red">
    Texto com tamanho ampliado e cor vermelha</font>
    <br /><br />
    <font size="-1" color="#FF8000">
    Texto com tamanho reduzido e cor laranja</font>
  </body>
</html>
```

fonte.html

A Figura 4 mostra as diferentes opções disponíveis para formatação dos textos de uma página HTML.



Figura 4: Fontes

Pratique!

Exibir a palavra BRASIL usando o tag H1 sendo que as letras B – A – I devem estar na cor verde e as letras R – S – L na cor amarela.

Imagens

A etiqueta img permite através dos atributos inserir (src), dimensionar (width e height) e alinhar (align) imagens dentro de uma página web.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Imagens</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Imagens</h1>
    A etiqueta img permite exibir imagens em uma página HTML.
    <img src="lamp-lig.png" align="left" alt="Lâmpada Ligada">
  </body>
</html>
```

imagem.html

Na Figura 5 podemos observar a exibição da imagem devidamente alinhada à esquerda.



Figura 5: Imagem

Links

Os links, também chamados de hyperlinks, permitem realizar a ligação entre as páginas HTML e para diferentes tipos de conteúdo, por exemplo, imagens, documentos ou programas para download. Além disso, é possível realizar a ligação entre conteúdos que estão em uma mesma página HTML. A etiqueta a (âncora) deverá ser usada para definir o link.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Sistema Solar</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Sistema Solar</h1>
    <p align="justify">O Sistema Solar consiste no conjunto
constituído pelo Sol e todos os corpos celestes que estão sob sua ação
gravitacional.</p>
    <a href="#sol">Sol</a><br>
    <a href="mercurio.html">Mercúrio</a><br>
    Vênus<br>
    Terra<br>
    Marte<br>
    Júpiter<br>
    Saturno<br>
    Urano<br>
    Netuno<br>

    <h1><a id="sol">Sol</a></h1>
    <p align="justify">O Sol é a estrela central do Sistema Solar. Seu
volume é composto por 92%
    hidrogênio, 7% de hélio e 1% de outros elementos.</p>
    <img src="sol.jpg"><br>
    <a target="_blank"
href="https://pt.wikipedia.org/wiki/Sol">
Mais informações</a>
  </body>
</html>
```

sistema-solar.html

Observe na página que o primeiro link criado, destacado a seguir, faz referência à uma âncora criada na mesma página. Neste caso devemos começar com o caractere # e colocar o nome.

```
<a href="#sol">Sol</a><br>
```

O local da página que será mostrado deverá ser definido usando o atributo id da âncora, conforme destacamos a seguir.

```
<a id="sol">Sol</a>
```

Quando desejamos referenciar uma outra página devemos colocar o endereço completo de onde a página se encontra localizada. Outra possibilidade, destacada a seguir, é indicar somente o nome da página. Neste caso, ela deverá estar hospedada exatamente na mesma pasta onde a página que a chamou está localizada.

```
    <a href="mercurio.html">Mercúrio</a><br>
```

Na Figura 6 temos a página criada, onde podemos observar os links que foram criados para o Sol e Mercúrio.



Figura 6: Links

Ao clicar no link "Sol" a página terá seu conteúdo exibido a partir da âncora sol, conforme ilustra a Figura 7.

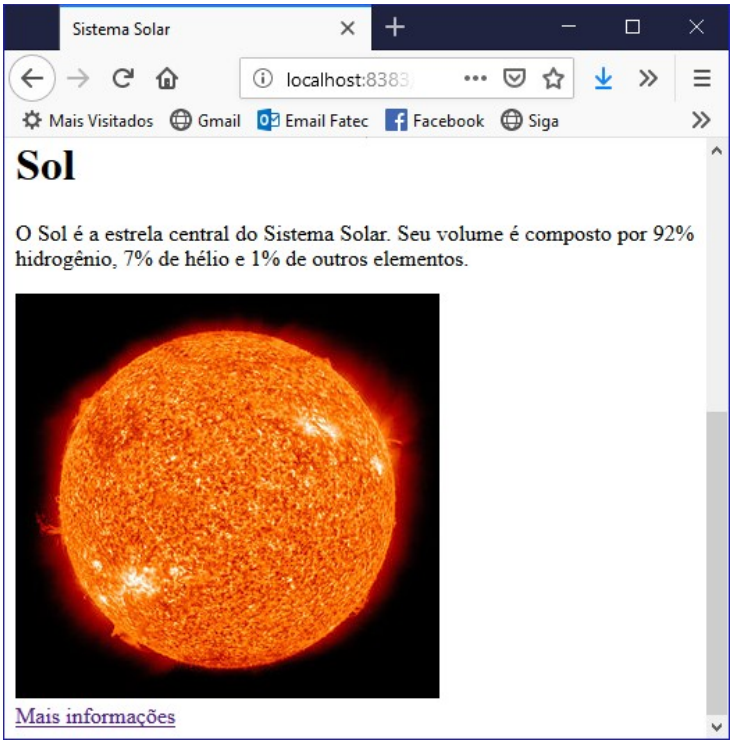


Figura 7: Página mostrada a partir da âncora

No trecho da página destacado a seguir vamos criar um link para um endereço (URL) localizado em outro servidor. Note, neste caso, que o endereço completo foi utilizado. Também observe o uso do atributo target, com o valor "_blank", que determina que a página, indicada no link, deverá ser aberta em uma nova aba do navegador.

```
    <a target="_blank"
href="https://pt.wikipedia.org/wiki/Sol">
Mais informações</a>
```

A página sobre o planeta Mercúrio apresentará o seguinte conteúdo.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Mercúrio</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Mercúrio</h1>
    <p align="justify">Mercúrio é o menor planeta do Sistema Solar,
também é o mais próximo do Sol. Visualmente é parecido com a Lua,
apresentando crateras de impacto e planícies lisas, não possui
satélites naturais</p>
    <a target="_blank"
href="https://pt.wikipedia.org/wiki/Merc%C3%BArio_(planeta)">Mais
informações</a>
  </body>
</html>
```

mercurio.html


Pratique!

Adotando como referência o conteúdo disponível na Wikipedia, crie as páginas para os demais planetas, implementando também os respectivos links na página sistema-solar.html.

Listas

Uma página HTML pode ter, em seu conteúdo, listas ordenadas (numeradas) e listas não ordenadas. O primeiro exemplo vai implementar uma lista não ordenada, observe o uso das etiquetas ul para definir a lista e li para identificar cada item que compõe a lista.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Lista de Compras</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Lista de Compras</h1>
    <ul>
      <li>Banana</li>
      <li>Café</li>
      <li>Batata</li>
      <li>Frango</li>
      <li>Pão de Forma</li>
      <li>Manteiga</li>
    </ul>
  </body>
</html>
```

lista-nao-ordenada.html

Na Figura 8 temos a exibição da página criada com a respectiva lista não ordenada que é caracterizada pelo uso de marcadores iniciando cada item da lista.

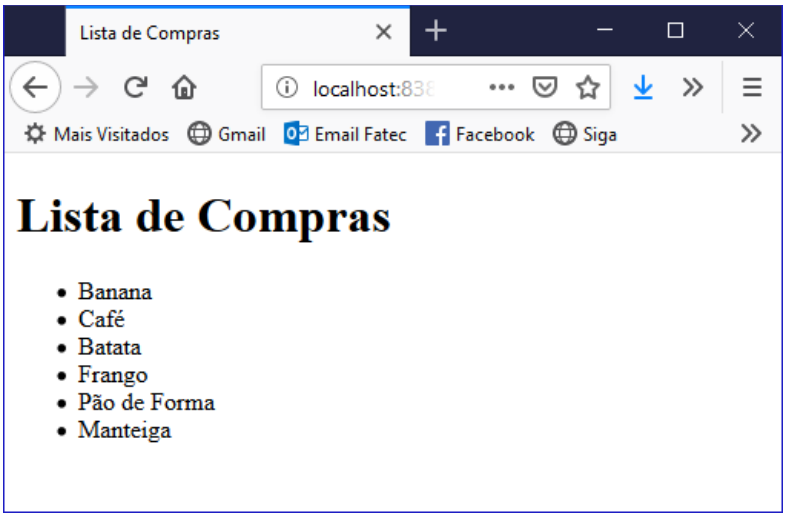


Figura 8: Lista não ordenada

Pratique! Adicione três itens à lista de compras.

No exemplo seguinte vamos criar uma lista ordenada (numerada). Note que, neste caso, devemos usar a etiqueta ol para delimitar a lista e li para identificar cada item.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Como trocar uma lâmpada?</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Como trocar uma lâmpada?</h1>
    <ol>
      <li>Desligar a energia elétrica</li>
      <li>Pegar uma escada</li>
      <li>Pegar uma lâmpada nova</li>
      <li>Colocar a escada sob a lâmpada queimada</li>
      <li>Subir na escada</li>
      <li>Desrosquear a lâmpada queimada</li>
      <li>Rosquear a nova lâmpada</li>
      <li>Descer da escada</li>
      <li>Ligar a energia elétrica</li>
    </ol>
  </body>
</html>
```

lista-ordenada.html

A lista ordenada será exibida na página, conforme ilustra a Figura 9.

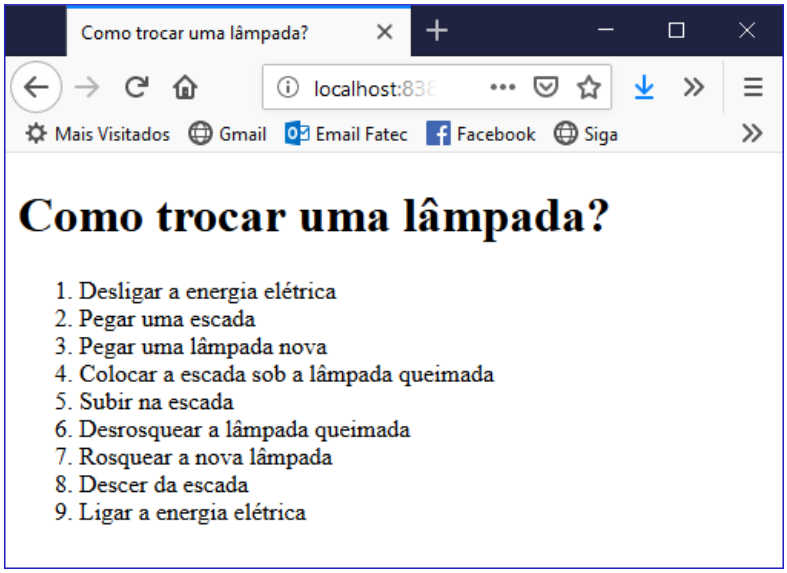


Figura 9: Lista ordenada

Pratique!

Remova o item 3 e adicione três itens à lista numerada.

Tabelas

É muito simples criar tabelas usando HTML. Um conjunto de etiquetas permitem especificar a tabela com suas respectivas linhas e colunas.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Relatório de Vendas</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Relatório de Vendas</h1>
    <table border="1" cellspacing="0" cellpadding="2">
      <tr>
        <td align="center">Produto</td>
        <td align="center">Quantidade</td>
        <td align="center">Preço (R$)</td>
      </tr>
      <tr>
        <td>Televisor</td>
        <td align="center">2</td>
        <td align="right">3.000,00</td>
      </tr>
      <tr>
        <td>Geladeira</td>
        <td align="center">4</td>
        <td align="right">1.990,00</td>
      </tr>
      <tr>
        <td>Fogão</td>
        <td align="center">3</td>
        <td align="right">600,00</td>
      </tr>
    </table>
  </body>
</html>
```

tabela.html

Analisando a página, inicialmente devemos identificar a etiqueta table e os atributos que permitirão definir a borda e o espaçamento entre as células. Em seguida, note o uso da etiqueta tr para especificar uma linha da tabela e a etiqueta td para determinar uma coluna. É importante salientar que se trata de uma estrutura hierárquica, ou seja, as colunas devem estar dentro de uma linha que, por sua vez, devem estar dentro da tabela. A Figura 10 mostra a exibição da página.

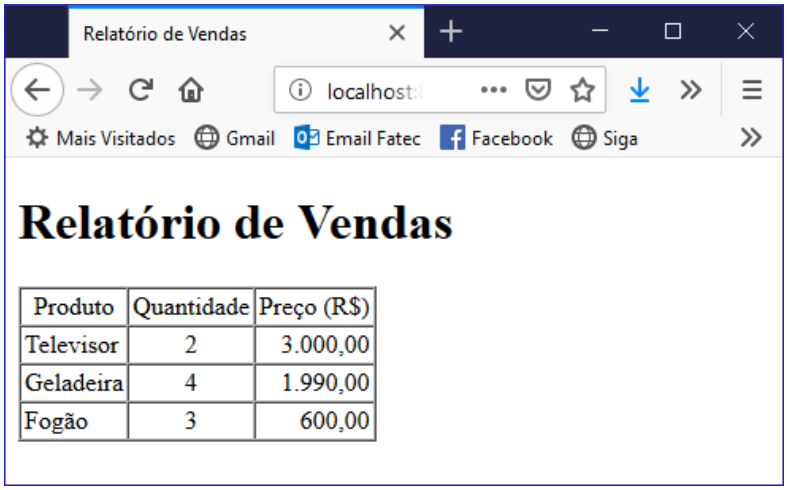


Figura 10: Tabela



Pratique!

- Adicione uma linha contendo o produto Notebook, com quantidade 6 e preço R$ 3.900,00.
- Adicione a coluna Desconto, com os valores 10% para Televisor, 5% para Geladeira, sem desconto para Fogão e 15% para Notebook.
- Adicione a coluna Imagem inserindo uma fotografia representativa para cada produto.

Exercícios

1) Criar uma página HTML, com conteúdo definido a seu critério, que mostre um título formatado com a etiqueta <H2>, um parágrafo com alinhamento justificado e fonte tamanho 8.
2) Desenvolver uma página HTML, com conteúdo definido a seu critério, que apresente o título formatado com a etiqueta <h1>, dois subtítulos formatados com a etiqueta <h2>. O conteúdo do primeiro subtítulo deverá ser apresentado na cor azul, fonte tamanho 4 e alinhado à direita. O conteúdo do segundo subtítulo deverá estar alinhado à esquerda.
3) Elaborar uma página HTML que mostre em uma tabela de duas colunas e três linhas imagens de aparelhos domésticos comuns, por exemplo, fogão, geladeira e televisor entre outros. Cada imagem deverá ser colocada em uma célula, juntamente com sua respectiva descrição formatada na cor verde, tamanho 6 e centralizada.
4) Criar uma página HTML, que mostre o título “Notícias” formatado com a etiqueta <H1> e apresente um conjunto de links para, pelo menos, 5 (cinco) sites de notícias.
5) Elaborar um site com, no mínimo, três páginas HTML e conteúdo definido a seu critério, sendo que a primeira página deverá possuir links para as outras duas páginas criadas. Este Web Site deverá apresentar os seguintes recursos:

    a) Mostrar imagens (HTML).
    b) Exibir uma lista de tópicos (HTML).
    c) Utilizar tabela (HTML).
    d) Utilizar links (HTML).
    e) Realizar a formatação de fontes (cores, tamanho, negrito e itálico).
    f) Realizar a formatação de parágrafos (esquerda, direita, centralizado e justificado).

Cascading Style Sheets

As Folhas de Estilo em Cascata (CSS - Cascading Style Sheets) permitem customizar a formatação dos elementos HTML. Desta forma, ampliam as possibilidades de exibição das páginas oferecendo bastante versatilidade para a diagramação das páginas e para a construção da identidade visual de um site. Outra vantagem de adotarmos CSS em nossos sites é a possibilidade de separar o conteúdo das páginas de sua formatação, o que facilita o processo de manutenção e atualização das páginas.

Vamos abordar as três maneiras que podemos utilizar para usar o CSS em nossas páginas HTML.

CSS Inline

A primeira opção mostrada consiste em utilizar o atributo style das etiquetas para inserir diretamente o elemento CSS. Esta opção é interessante apenas quando temos pouco conteúdo no site ou para fazer algum teste rápido para avaliar alguma alteração na diagramação da página. Mas, de forma geral, deva ser evitada nos sites que estão publicados.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Introdução ao CSS</title>
    <meta charset="UTF-8">
  </head>
  <body style="background-color:navy;">
    <h1 style="font-family:sans-serif; color:yellow; padding:20px;">
      Introdução ao CSS
    </h1>
    <p style="color:white; padding:20px;">
      CSS Inline.
    </p>
  </body>
</html>
```

css-inline.html

Conforme mostra o exemplo, o CSS trabalha sempre através de pares de atributos (propriedades) e valores, sendo que os pares devem ser separados por ponto e vírgula. Enquanto atributos e valores estão unidos pelo caractere dois pontos, por exemplo, color:white, onde color é o atributo e white é o seu valor. Na Figura 11 temos a página HTML customizada com os atributos do CSS.

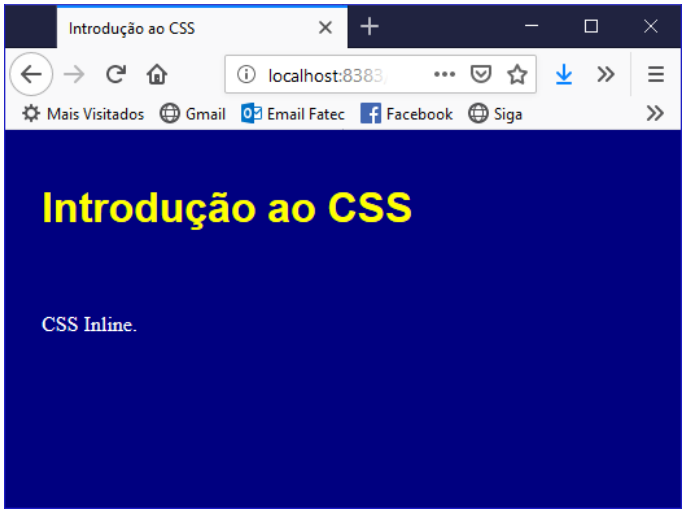


Figura 11: Exemplo de CSS Inline

CSS Interno

Esta opção permite separar o conteúdo da página dos elementos de formatação através do emprego da etiqueta style, onde será definida o conjunto de atributos que serão customizados pela folha de estilos em cascata.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Introdução ao CSS</title>
    <meta charset="UTF-8">
    <style>
    body {
      background-color: black;
    }
    h1 {
      color: cyan;
      text-align: center;
      padding: 30px;
    }
    p {
      color: white;
      padding: 30px;
    }
    </style>
  </head>
  <body>
    <h1>Introdução ao CSS</h1>
    <p>CSS Interno</p>
  </body>
</html>
```

css-interno.html

Observe que dentro da etiqueta style serão definidas os atributos que serão aplicados ao elemento HTML, conforme destaca o trecho de código-fonte a seguir, onde a etiqueta h1 terá a sua cor alterada para cyan, o texto será exibido alinhado ao centro da página e terá uma margem de 30 pixels em relação aos demais elementos HTML. Também é importante salientar que o grupo de atributos para determinado elemento HTML deverá estar delimitado por chaves.

{CSS}

```
h1 {
  color: cyan;
  text-align: center;
  padding: 30px;
}
```

Na Figura 12 temos a exibição da página criada.

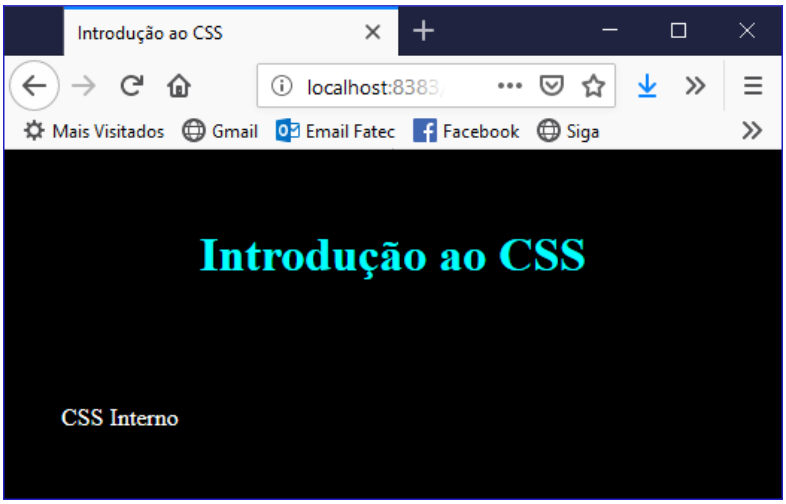


Figura 12: Exemplo de CSS Interno

CSS Externo

Nesta opção iremos, além de separar o conteúdo das páginas de formatação, possibilitar que uma única folha de estilos seja utilizada por diversas páginas do site. Desta maneira, é possível criar uma identidade visual para todo o site que é simples de manter, pois qualquer alteração dentro da folha de estilos irá automaticamente afetar todas as páginas do site. Observe que será criado um arquivo (estilo.css) que vai conter todos os elementos CSS. A página HTML apenas deverá referenciar este arquivo através da etiqueta link.

</>

```
<!DOCTYPE html>
<html>
  <head>
    <title>Introdução ao CSS</title>
    <meta charset="UTF-8">
    <link rel="stylesheet" type="text/css" href="estilo.css" />
  </head>
  <body>
```

```
    <h1>Introdução ao CSS</h1>
    <p>CSS Externo</p>
  </body>
</html>
```

css-externo.html

Como já mencionado anteriormente, o arquivo CSS, mostrado a seguir, contém apenas os elementos de formatação da página.

{CSS}

```
body {
  background-color: navy;
}

h1 {
  color: yellow;
  text-align: center;
}

p {
  color: white;
}
```

estilo.css

Na Figura 13 apresentamos a página devidamente customizada pela folha de estilos.

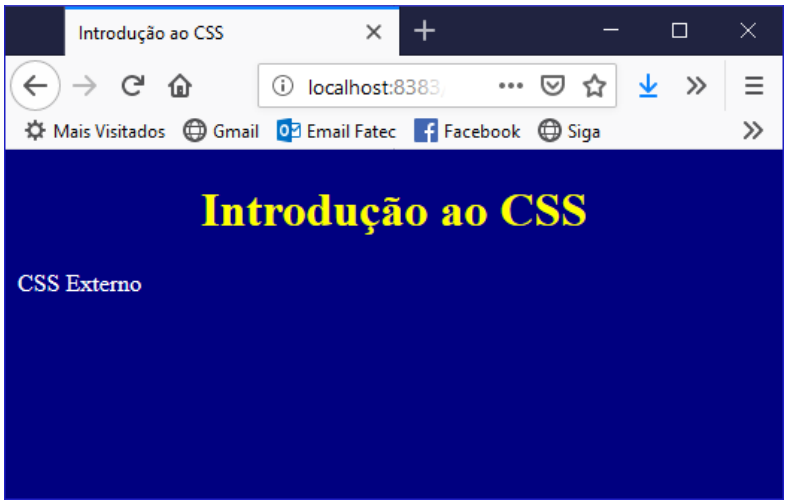


Figura 13: Exemplo de CSS Externo

Seletores e classes

No exemplo anterior podemos notar que a formatação sobre determinado elemento HTML afetará todos os elementos da página. Por exemplo, a formatação da etiqueta de parágrafo (p), mostrada em detalhes no trecho de código-fonte a seguir, formatará todos os parágrafos da página.

{CSS}

```
p {
  color: white;
}
```

Existem situações dentro do projeto que um web site que precisamos tratar dos elementos de forma individual ou através de grupos. Nestes casos devemos aplicar os conceitos de seletores quando precisamos atuar sobre um elemento individualmente ou classes quando desejamos trabalhar com um grupo de elementos.

</>

```
<!DOCTYPE html>
<html>
  <head>
    <title>Introdução ao CSS</title>
    <meta charset="UTF-8">
    <link rel="stylesheet" type="text/css" href="estilo.css" />
  </head>
  <body id="fundo">
    <h1 id="titulo">Introdução ao CSS</h1>
    <p class="texto">Seletores</p>
    <p class="texto">Classes</p>
  </body>
</html>
```

seletores-classes.html

Observe no código-fonte apresentado a seguir que um seletor será especificado usando o caractere # precedendo o identificador (id) do elemento que será afetado, o identificador deve ser único, ou seja, não podemos em uma mesma página definir mais do que um elemento HTML com o mesmo id. Por exemplo, o elemento body que possui o atributo id definido como fundo, será referenciado no CSS como #fundo. Uma classe, por sua vez, permite referenciar um conjunto de elementos através do atributo HTML class, que será referenciado no CSS, com o caractere de ponto final antecedendo o nome da classe. Como exemplo temos a formatação do texto que será usado pelos parágrafos que pertencem a classe identificada como texto.

{CSS}

```
#fundo {
  background-color: #808080;
  padding: 20px;
}

#titulo {
```

```
  color: lime;
}

.texto {
  color: lightcyan;
  font-weight: bold;
}
```

estilo.css

Na Figura 14 apresentamos a página.

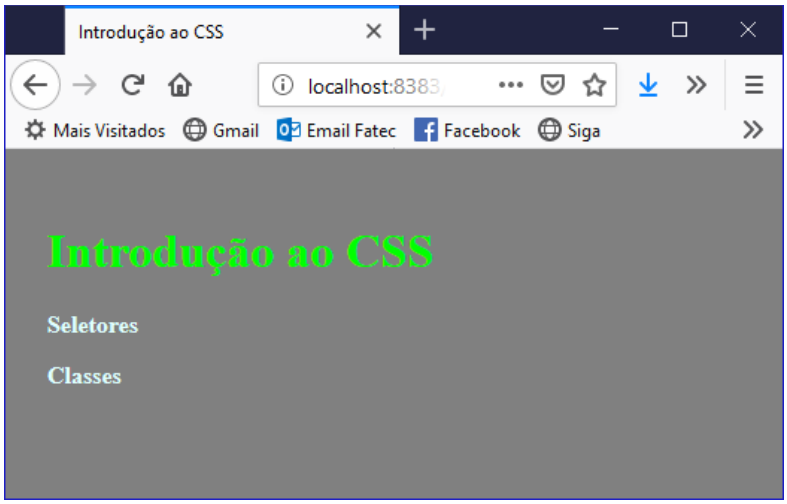


Figura 14: Seletores e classes



Dica!

Uma ótima referência é a **Mozilla Developer Network**, acesse https://developer.mozilla.org/pt-BR/docs/Web/CSS

Layout: Dimensão e posicionamento

Entre seus inúmeros recursos, CSS permite a criação de layout de páginas que poderão se ajustar automaticamente à diferentes dimensões de janelas. Este é conceito de Web Site responsivo, bastante utilizado atualmente, pois, diferentes dispositivos (Computadores, SmartPhones e SmartTVs entre outros), com diferentes tamanhos e resoluções de tela acessam a Internet. Neste exemplo criaremos a estrutura básica de um site responsivo que consistirá no arquivo HTML apresentado a seguir.

```
<!DOCTYPE html>
<html>

<head>
  <title>Layout: Dimensão e posicionamento</title>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
```

```
  <link rel="stylesheet" type="text/css" href="css-site.css" />
</head>

<body>
  <div class="cabecalho">
    <h1>Layout: Dimensão e posicionamento</h1>
    <p>Este é um web site responsivo. Verifique o seu funcionamento em
diferentes tamanhos de janela.</p>
  </div>

  <div class="menu">
    <a href="#">Link A</a>
    <a href="#">Link B</a>
    <a href="#">Link C</a>
    <a href="#">Link D</a>
  </div>

  <div class="conteudo">
    <div class="coluna lateral">
      <h2>Esquerda</h2>
      <p>A ligeira raposa marrom salta sobre o cão preguiçoso.</p>
    </div>

    <div class="coluna principal">
      <h2>Principal</h2>
      <p>A ligeira raposa marrom salta sobre o cão preguiçoso.
        A ligeira raposa marrom salta sobre o cão preguiçoso.
        A ligeira raposa marrom salta sobre o cão preguiçoso.
        A ligeira raposa marrom salta sobre o cão preguiçoso.
        A ligeira raposa marrom salta sobre o cão preguiçoso.
      </p>
      <p>A ligeira raposa marrom salta sobre o cão preguiçoso.
        A ligeira raposa marrom salta sobre o cão preguiçoso.
        A ligeira raposa marrom salta sobre o cão preguiçoso.
        A ligeira raposa marrom salta sobre o cão preguiçoso.
        A ligeira raposa marrom salta sobre o cão preguiçoso.
        A ligeira raposa marrom salta sobre o cão preguiçoso.</p>
    </div>

    <div class="coluna lateral">
      <h2>Direita</h2>
      <p>A ligeira raposa marrom salta sobre o cão preguiçoso.</p>
    </div>
  </div>

  <div class="rodape">
    <p>Rodapé</p>
  </div>
</body>
</html>
```

css-site.html

Também devemos implementar uma folha de estilos externa que será responsável pelo layout.

{CSS}

```
* {
  box-sizing: border-box;
  font-family: Arial, Helvetica, sans-serif;
}

body {
  margin: 0;
}

.cabecalho {
  background-color: #7FFFD4;
  padding: 20px;
  text-align: center;
}

/* Barra de menu */
.menu {
  overflow: hidden;
  background-color: #008B8B;
}

/* Item do menu */
.menu a {
  float: left;
  display: block;
  color: #f2f2f2;
  border-radius: 15px;
  text-align: center;
  padding: 14px 16px;
  text-decoration: none;
}

/* Mudar a cor quando o ponteiro do mouse estiver sobre o item */
.menu a:hover {
  background-color: #20B2AA;
  color: black;
}

.coluna {
  float: left;
  padding: 10px;
}

.coluna.lateral {
  width: 25%;
}
```

```
.coluna.principal {
  width: 50%;
}

/* Limpa a flutuação após exibir as colunas */
.conteudo::after {
  content: "";
  display: table;
  clear: both;
}

/* Layout responsivo - ajusta para três colunas ou empilhadas */
@media screen and (max-width: 600px) {
  .coluna.lateral, .coluna.principal {
    width: 100%;
  }
}

.rodape {
  background-color: #7FFFD4;
  padding: 10px;
  text-align: center;
}
```

css-site.css

Exercícios

1) Considerando o layout definido no exemplo anterior (Layout: Dimensão e posicionamento) desenvolver um site com tema e conteúdo a sua escolha. Este site deve conter obrigatoriamente: a) Uma imagem na coluna conteúdo principal. b) Definir pelo menos 4 links no menu que devem carregar páginas que usam o mesmo cabeçalho, menu e rodapé da página principal, mas com apenas uma coluna no conteúdo. c) Utilizar a família de fontes Arial, Helvetica, Sans-serif em todas as páginas. d)Toda a formatação deve ser realizada apenas com CSS.
2) Criar uma página HTML, com conteúdo definido a seu critério, que mostre um título formatado com a etiqueta <H2> e, através de CSS inline, um parágrafo com alinhamento justificado e fonte 14 px.
3) Desenvolver uma página HTML, com conteúdo definido a seu critério, que apresente título formatado com a etiqueta <h1>, dois subtítulos formatados com a etiqueta <h2>. O texto relacionado ao primeiro subtítulo deverá ser apresentado na cor azul, fonte tamanho 12px e alinhado à direita. O texto relacionado ao segundo subtítulo deverá estar alinhado à esquerda. Utilizar CSS interno para realizar as formatações solicitadas.
4) Elaborar uma página HTML que mostre em uma tabela 6 imagens de aparelhos domésticos comuns, por exemplo, fogão, geladeira e televisor entre outros. Cada imagem deverá ser apresentada centralizada em uma célula, com sua respectiva descrição formatada na cor verde, tamanho 24px e centralizada. Adotar CSS interno para realizar as formatações solicitadas.

5) Elaborar um site com três páginas HTML, com conteúdo definido a seu critério, sendo que a primeira página deverá possuir links para as outras duas páginas criadas. Utilizar um CSS externo de modo que todas as páginas apresentem um fundo preto, os títulos deverão ser mostrados em amarelo, os textos na cor branca, fonte tamanho 14px e os links na cor cian.
6) Aplicando o conceito de CSS externo, seletores e classes, criar uma página que exiba no nome das cores vermelho, verde, amarelo, azul, cian, rosa, roxo e marrom. Cada item deverá ser mostrado formatado na cor que representa e em tamanho 28px.
7) Crie um cartão de perfil que contenha uma foto circular, um nome em negrito e uma breve biografia, sendo que: a) O cartão deve ter um fundo branco, bordas arredondadas e uma sombra (box-shadow). b) A imagem deve ser perfeitamente redonda (border-radius: 50%). c) Use padding para o espaçamento interno e margin para centralizar o cartão na página. d) O fundo da página deve ser em um tom de cinza médio.

Linguagens de programação para a Web

A linguagem HTML é a base de criação de qualquer página para a Web, porém trata-se de uma linguagem que é composta por elementos estáticos e está fortemente focada na composição e formatação de documentos.

O grande crescimento e popularização da Internet nas últimas décadas trouxe a necessidade da criação de novas linguagens que conseguissem agregar conteúdo dinâmico as páginas, além de recursos como, por exemplo, a validação de dados e acesso a banco de dados entre outras funcionalidades.

Neste contexto, surgem linguagens como o PHP, ASP.net, JSP e JavaScript entre outras. Estas linguagens não substituem o HTML, que é a base de qualquer documento para a Web, mas atuam em conjunto suprindo os recursos que não estão disponíveis no HTML.

Estas linguagens são classificadas em Server Side, ou seja, são executadas no servidor Web ou Client Side que são as linguagens que são executadas no cliente (Navegador Web).

Nas linguagens Server Side os programas criados são executados no servidor e os resultados são enviados, em HTML, para o cliente que está acessando o servidor através de um programa navegador, conforme ilustra a Figura 15. Como principais linguagens Server Side podemos citar PHP, ASP, ASP.Net, JSP (Java), Node.js e Python.

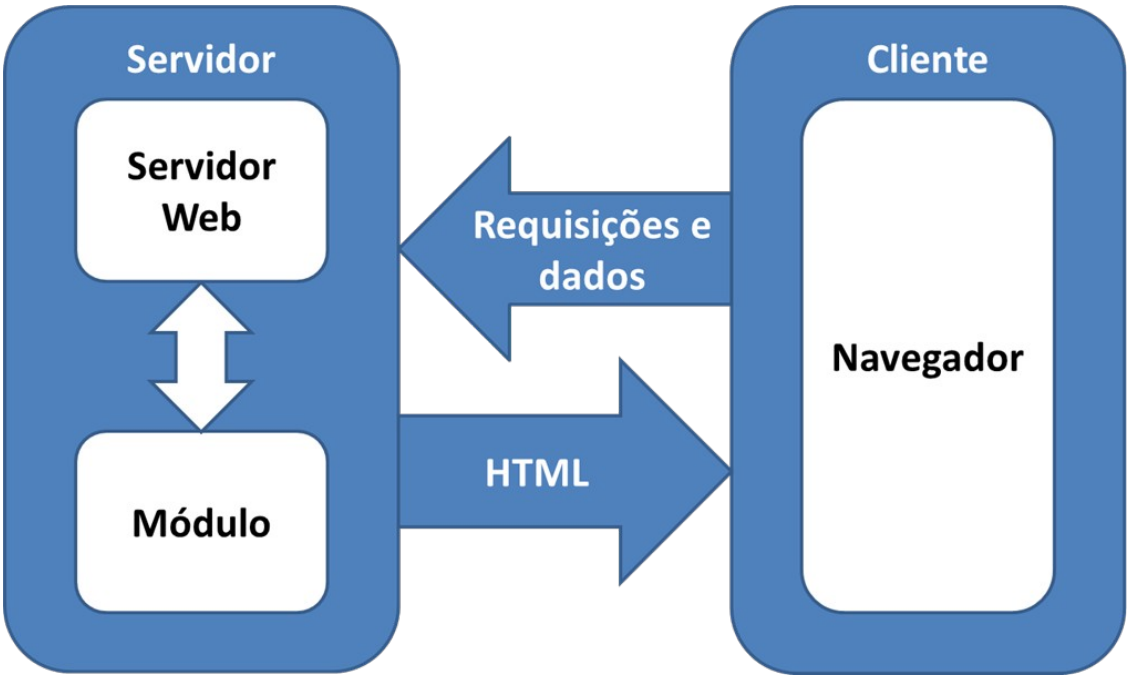


Figura 15: Estrutura básica de uma aplicação Web

Por outro lado, nas linguagens Client Side os scripts são enviados, com o HTML, para o navegador Web, o qual será o responsável pela execução. A principal linguagem Client Side, se tornando um padrão de mercado, é o JavaScript.

JavaScript

O JavaScript é uma linguagem de programação orientada a objetos que é interpretada e executada pelo navegador Web (client-side script). Apresenta uma sintaxe similar a linguagem Java e tem como objetivo principal dar uma maior interatividade às páginas. Uma característica importante da linguagem JavaScript reside no fato de ela não possuir tipos de dados, sendo que qualquer variável definida é do tipo variante, ou seja, o tipo de dados é definido de acordo com a informação de está armazenada naquele momento. O tipo de dados de determinada variável pode ser modificado ao longo da execução da aplicação, conforme o seu conteúdo vai sendo alterado.

Primeiros scripts

Abaixo está o código fonte do script que será criado. Este script é parte de uma página com conteúdo HTML e que deve ser gravada com a respectiva extensão.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Introdução ao JavaScript</title>
    <meta charset="UTF-8">
  </head>
  <body onload="iniciar()">
    <script>
    function iniciar() {
      document.write("<h1>" + document.title + "</h1>");
      document.write("Olá pessoal!<br>");
    }
    </script>
  </body>
</html>
```

ola.html

Este exemplo consiste basicamente em utilizar o evento onload da página HTML para realizar a chamada da função iniciar. A função, por sua vez, utilizará o método write do objeto document para escrever um determinado conteúdo na área de trabalho do navegador. O resultado obtido após a execução da página no navegador é mostrado na Figura 16.



Figura 16: Utilizando o JavaScript

Como podemos observar na Figura 17, assim como qualquer linguagem orientada a objetos, o JavaScript está fundamentado sobre os conceitos objetos, eventos, atributos (propriedades ou características) e métodos (ações), além de incluir o uso de funções. Por exemplo:




```
<!DOCTYPE html>
<html>
  <head>
    <title>I                    JavaScript</title>
    <meta char                  -8">
    ad>
     onload="iniciar()"
    script type="text/javascript">
    function iniciar() {
      document.write("<h1>" + document.title
        + "</h1>");
      document.write("Olá pessoal!<br>");
    }
    </script>
  </body>
</html>
```

ola.html

Figura 17: Funções, objetos, eventos, atributos e métodos

Vários objetos estão disponíveis para serem utilizados, com o intuito de ilustrar esse conceito, vamos criar uma página. A função iniciar vai utilizar o objeto navigator e, através dos atributos appName e appVersion, identificar o nome e a versão do navegador que o usuário está utilizando.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Bem-vindo ao JavaScript</title>
    <meta charset="UTF-8">
  </head>
  <body onload="iniciar()">
    <script>
    function iniciar() {
      document.write("<h1>" + document.title + "</h1>");
      document.write ("Você está utilizando o navegador " +
navigator.appName + ", versão " + navigator.appVersion);
    }
    </script>
  </body>
</html>
```

navegador.html

A Figura 18 mostra o resultado da execução da página que foi criada, sendo importante observar a exibição do nome do navegador e a respectiva versão.



Figura 18: Exibição dos atributos de um objeto

Caixas de mensagem e diálogo

Neste exemplo são mostradas as funções prompt e alert que permitem, respectivamente, receber e exibir informações em janelas (caixas de diálogo e mensagem) destacadas da página principal. Também é mostrado o método toUpperCase que realizar a conversão de uma determinada cadeia de caracteres em letras maiúsculas.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Caixas de Diálogo e Mensagem</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <script>
    function login() {
      let nome = prompt("Qual o seu nome?", "Zé Ninguém");
      alert("Olá, " + nome.toUpperCase() + ".");
    }
    </script>
    <h1>Identificação</h1>
    <input type="button" value="Login" onclick="login()">
  </body>
</html>
```

caixas.html

Na Figura 19, temos um exemplo da execução da página após o pressionamento do botão Login:

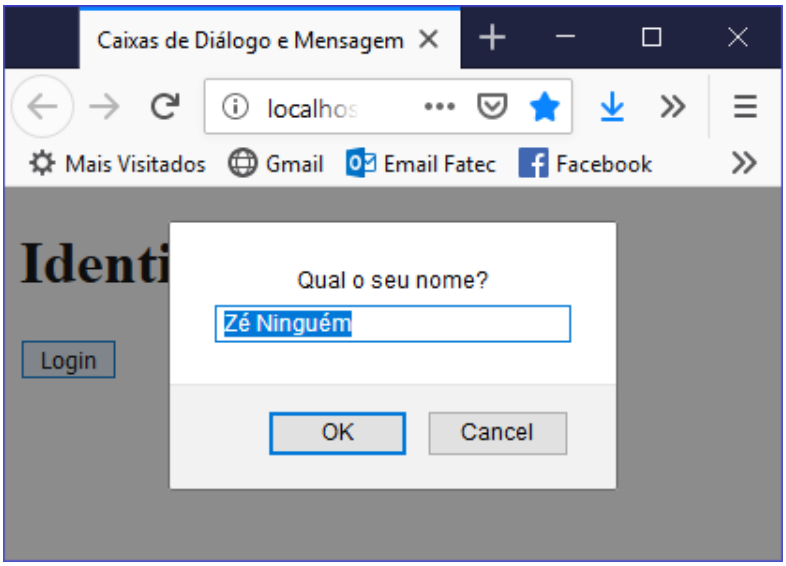


Figura 19: Uso da função prompt

Depois de preencher o campo nome e pressionar o botão OK, podemos observar na Figura 20 que será exibido o nome da pessoa através do emprego da função alert.

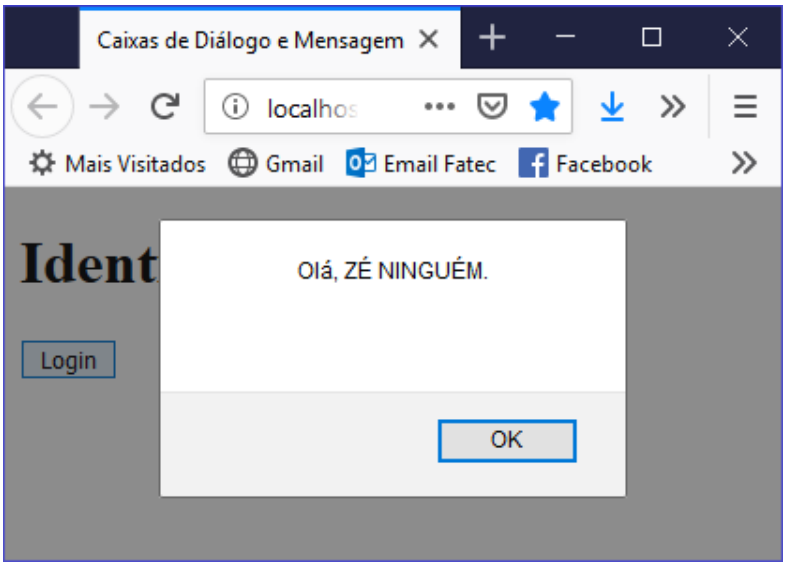


Figura 20: Uso da função alert

As funções prompt e alert podem ser muito úteis no processo de desenvolvimento das páginas, porém, o uso delas deve ser evitado nas versões finais. Pois, como se trata de elementos que não apresentam recursos de formatação, prejudicam a sua integração com o padrão visual (design) desenvolvido para a página. Neste contexto, o conceito de formulário, que será abordado a seguir, deve ser usado para permitir a interação com o usuário.

Formulários: Introdução

O HTML implementa o conceito de formulário através de um conjunto de etiquetas que possibilitam a interação do usuário com uma página web. Em seguida, podemos através de funções associadas à eventos, obter e processar os dados fornecidos nos elementos dos formulários. No exemplo seguinte utilizamos os elementos caixa de texto e botão para interagir com a página. Os elementos do HTML que precisarão ser acessados pelo JavaScript devem possuir obrigatoriamente o atributo id (identificação). Desta forma, o texto digitado pelo usuário na caixa de texto será acessado através do seu id seguido pelo atributo value, então, neste exemplo, devemos usar nome.value.

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Formulários</title>
</head>
<body>
  <h1>Formulários</h1>
  Digite o nome:
  <input type="text" id="nome">
  <input type="button" value="Login" onclick="login()">
  <script>
  function login() {
    alert("Olá, " + nome.value.toUpperCase() + ".");
  }
  </script>
</body>
</html>
```

formulario.html

Eventos

Conforme abordado nos exemplos anteriores, em JavaScript podemos associar funções aos eventos através dos atributos colocados diretamente nas etiquetas HTML como, por exemplo, onload, onclick ou onblur entre diversos outros. Outra forma, mais elegante de associar os eventos às funções é realizar isso usando apenas JavaScript mantendo os códigos-fonte HTML e JavaScript totalmente isolados.

O método addEventListener é usado para determinar a função de callback, ou seja, a função que será executada quando o evento ocorrer. Note que o objeto precisa ter uma id (identificação) definida e, no JavaScript adicionamos o EventListener a ele. Neste caso é importante que a etiqueta <script> seja carregada após a criação do objeto, caso contrário, o programa vai apresentar um erro se tentarmos adicionar o EventListener ao objeto, pois, este ainda não existirá. No exemplo a seguir, temos a página *formulario.html* reescrita a partir destes conceitos.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Caixas de Diálogo e Mensagem</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Identificação</h1>
    Digite o nome:
    <input type="text" id="nome">
    <input type="button" value="Login" id="btLogin">
    <script>
      btLogin.addEventListener("click", login);

      function login() {
        alert("Olá, " + nome.value.toUpperCase() + ".");
      }
      </script>
  </body>
</html>
```

eventos.html

Containers

Apesar da linguagem HTML não permitir a inserção (e ou modificação) do seu conteúdo após o carregamento da página, ela provê alguns elementos (TAGs) que atuam como containers de dados. Elementos containers são aqueles que permitem marcar uma determinada porção do texto e/ou outros elementos HTML e, posteriormente, através de uma linguagem de script, esse conteúdo pode ser modificado de maneira dinâmica, ou seja, após a página ser carregada.

```
<!DOCTYPE html>
<html>
  <head>
    <title> Elementos Containers</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Identificação</h1>
    Digite o nome:
    <input type="text" id="nome">
    <input type="button" value="Login" id="btLogin">
    <hr>
    <div id="resultado">Olá.</div>
    <script>
```

```
      btLogin.addEventListener("click", login);

      function login() {
        resultado.innerHTML = "Olá, " + nome.value.toUpperCase() +
".";
      }
    </script>
  </body>
</html>
```

containers.html

Na Figura 21 temos um exemplo de como a página poderá ser exibida após a execução do script:

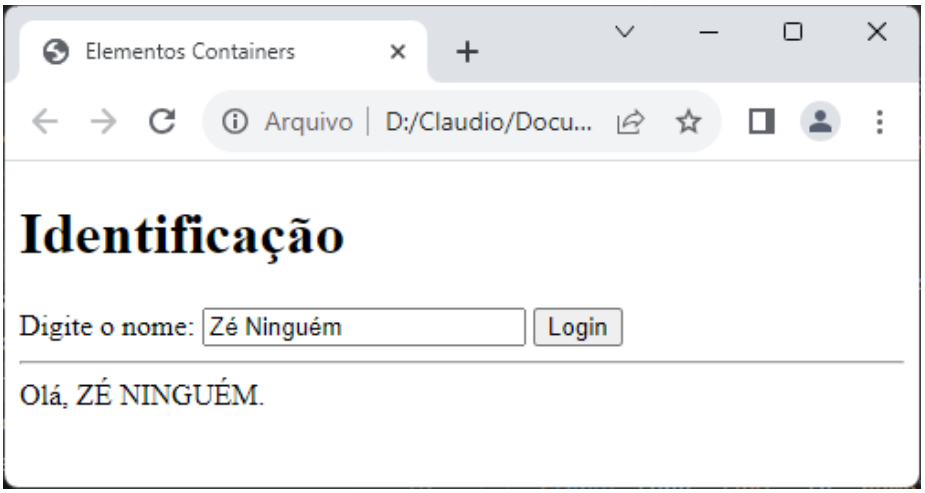


Figura 21: Texto exibido no elemento container

Variáveis em JavaScript

Conforme abordado anteriormente, as variáveis em JavaScript não possuem um tipo de dados no momento de sua declaração, elas passam a assumir o tipo de dados dos valores que estão armazenados naquele instante. No exemplo a seguir, será possível observar que a variável "x" recebe inicialmente uma cadeia de caracteres, ou seja, neste momento, ela é do tipo de dados String. Posteriormente ela recebe um valor numérico assumindo, a partir deste momento, o tipo de dados Number contendo um valor inteiro (int). Neste exemplo, a variável "x" como sendo do tipo de dados Number, mas apresentando um valor real (float).

```
<!DOCTYPE html>
<html>
  <head>
    <title>Variáveis em JavaScript</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <script>
```

```
    window.addEventListener("load", iniciar);

    function iniciar() {
      document.write("<h1>" + document.title + "</h1>");
      let x = "Olá!";
      document.write("A variável é String: " + x + "<br>");
      x = 12 + 13;
      document.write("Agora é inteira: " + x + "<br>");
      x = 4.50 + 5.25;
      document.write("Neste momento é real: " + x + "<br>");
    }
    </script>
  </body>
</html>
```

variaveis0.html

Na Figura 22 é mostrada a página criada.

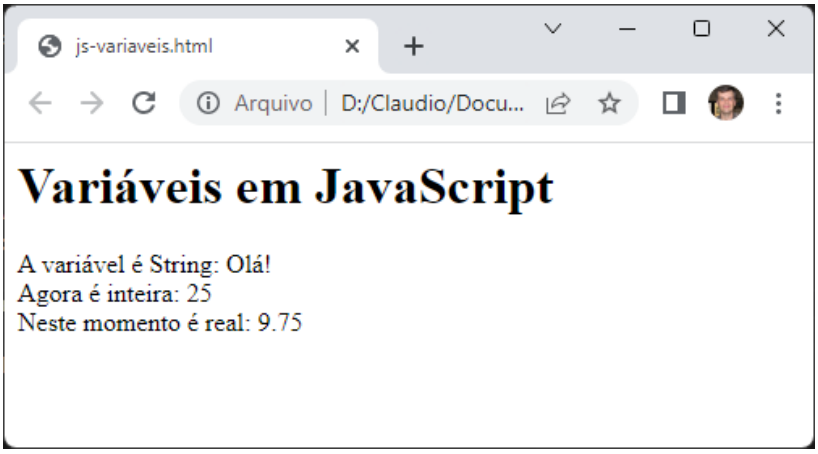


Figura 22: Variáveis

As variáveis podem ser declaradas através das instruções let ou var. A principal diferença entre elas é o escopo. Variáveis declaradas com var possuem escopo de função ou global, além de permitir re-declaração. Por outro lado, a declaração com let possui escopo de bloco ({ }), limitando-se ao local onde foram definidas, não permite re-declaração tornando o seu uso mais seguro. Observe que os valores para o tipo de dados String devem ser especificados entre aspas simples (') ou duplas ("), por exemplo:

```
{JS}
let x = "Olá";
let y = 'amigo!';
```

ou

```
{JS}
var x = "Olá";
var y = 'amigo!';
```

Valores numéricos, por sua vez, não possuem delimitadores e como separador decimal se deve utilizar o caractere de ponto final (.), conforme mostrado a seguir:

```
{JS}
let x = 12 + 13;
let y = 4.50 + 5.25;
```

ou

```
{JS}
var x = 12 + 13;
var y = 4.50 + 5.25;
```

Quando uma variável é definida dentro de uma função ela assumirá um **escopo local**, ou seja, existe apenas para a função. Por outro lado, quando declarada fora das funções, a variável terá **escopo global**, podendo ser acessada por qualquer função.

```
</>
<!DOCTYPE html>
<html>
  <head>
    <title>Variável Global</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Variável Global</h1>
    <input type="text" id="txNum" value="0">
    <input type="button" value="+" id="mais">
    <input type="button" value="-" id="menos">
    <script>
    mais.addEventListener("click", inc);
    menos.addEventListener("click", dec);
    let num = 0;

    function inc() {
      num++;
      txNum.value = num;
    }

    function dec() {
      num--;
      txNum.value = num;
    }
    </script>
  </body>
</html>
```

variaveis1.html

Neste programa observe que a variável num tem escopo global e, desta forma, é acessível pelas funções inc e dec.


Pratique!

Adicione ao programa um botão “Zerar” que, ao ser clicado, ajuste o valor da variável num para 0 (zero).

Também existem as constantes que são declaradas pela instrução const, sendo utilizadas em cenários que necessitam armazenar um valor fixo e que será imutável durante toda a execução do script, não podendo ser reatribuídas. Obrigatoriamente devem ser inicializadas no momento da declaração e apresentam escopo de bloco, similar ao let. Por exemplo, podemos definir o valor de π (Pi):

{JS}

```
const PI = 3.14159;
```

Operadores

De mesma maneira que outras linguagens de programação, JavaScript possui um grande conjunto de operadores, vamos apresentar aqui os mais utilizados.

Grupo	Operador	Descrição
Atribuição	=	Atribuição simples
	+=	Atribuição de adição
	-=	Atribuição de subtração
	*=	Atribuição de multiplicação
	/=	Atribuição de divisão
	%=	Atribuição de resto
	**=	Atribuição de exponenciação
Relacional	==	Igual
	===	Exatamente igual (conteúdo e tipo de dado)
	!=	Diferente
	!==	Exatamente diferente (conteúdo e tipo de dado)
	<	Menor
	<=	Menor ou igual
	>	Maior
	>=	Maior ou igual
Aritméticos	+	Adição
	-	Subtração
	*	Multiplicação
	/	Divisão
	%	Resto da Divisão
	**	Exponenciação
	++	Incremento
	--	Decremento
Lógicos	&&	E (AND)
	||	OU (OR)
	!	NÃO (NOT)

No trecho de programa a seguir vamos ilustrar o uso de alguns operadores.

{JS}

```
let x = 10;
x -= 4;
let y = x ** 2;
```

Na primeira linha temos o operador de atribuição (=), isto é, a varíavel x receberá o valor 10. Na linha seguinte usamos o operador de atribuição de subtração (-=), neste caso, x receberá

o valor de 10 (que é o valor atual de x) menos 4. Ou seja, ao final da operação x armazenará o valor 6.

Na terceira linha usamos o operador de exponenciação (**) e elevamos o valor de x, que é 6, ao quadrado (6^2), desta forma, y vai armazenar 36. No exemplo seguinte, podemos visualizar a diferença entre os operadores de igualdade (==) e exatamente igual (===).

{JS}

```
let x = 5;
let y = "5";
let z = (x == y);  // z receberá true (verdadeiro)
z = (x === y);     // z receberá false (falso)
```

Analisando o exemplo, na igualdade será verificado apenas o valor, sem considerar o tipo de dado, sendo a expressão avaliada como verdadeira. Por outro lado, quando usamos o operador exatamente igual, o valor e o tipo de dados devem ser iguais, para que a expressão seja avaliada como verdadeira, o que não ocorreu neste exemplo.



Dica!

A **Mozilla Developer Network**, apresenta a relação completa dos operadores, acesse
https://developer.mozilla.org/pt-BR/docs/Web/JavaScript

Depuração de Scripts

Os navegadores implementam ferramentas que nos permite monitorar a execução de das instruções JavaScript e emitir diretamente comandos para avaliar o seu funcionamento. Aqui vamos ilustrar o uso destas ferramentas no navegador Firefox. Em outros navegadores as ferramentas podem ser acessadas na maneira parecida. Por exemplo, no Chrome devemos clicar no ícone ⋮ (três pontos verticais), localizado no canto superior direito da janela, escolher "Mais ferramentas" e depois "Ferramentas do desenvolvedor".

Quando estiver usando o Firefox, clique na barra de ferramentas no botão "Open menu" que fica localizado no canto superior direito da janela do navegador, conforme mostra a Figura 23. Então, selecione a opção "Web Developer" e depois escolha "Web Console".

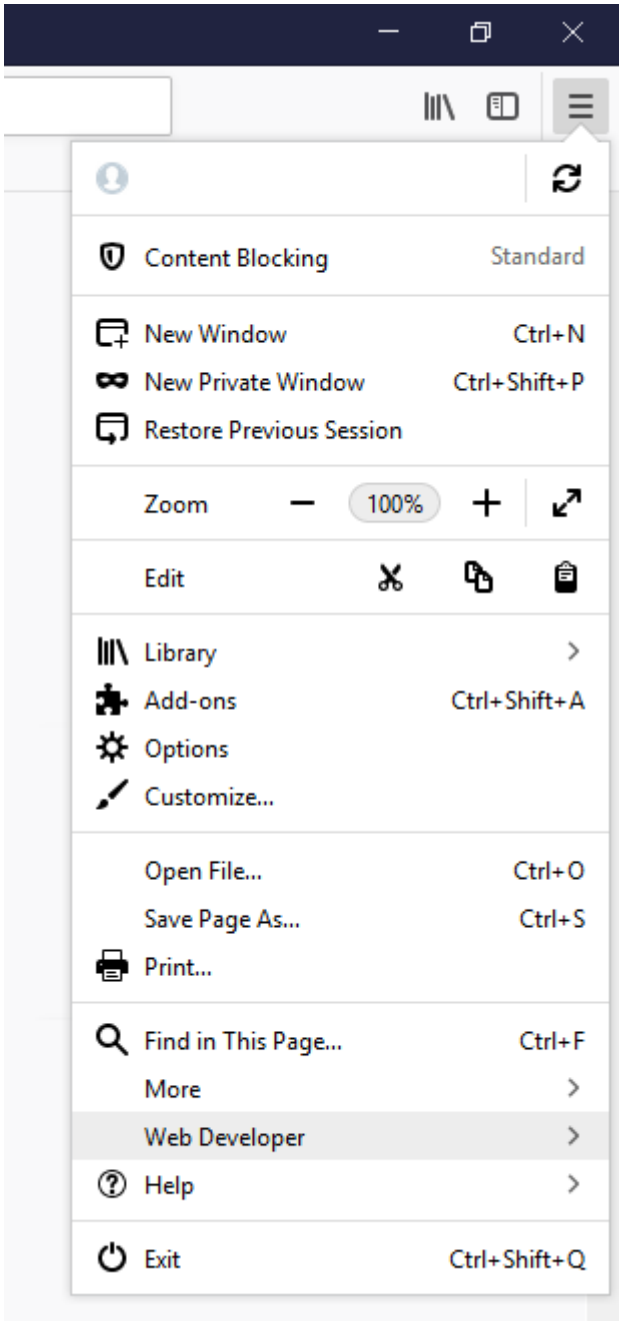


Figura 23: Menu

Na parte inferior da janela do navegador será mostrado o “Web Console” (Figura 24). Nele é possível enviar comandos ele também aponta as linhas de um script que apresentaram erro durante a execução do script.

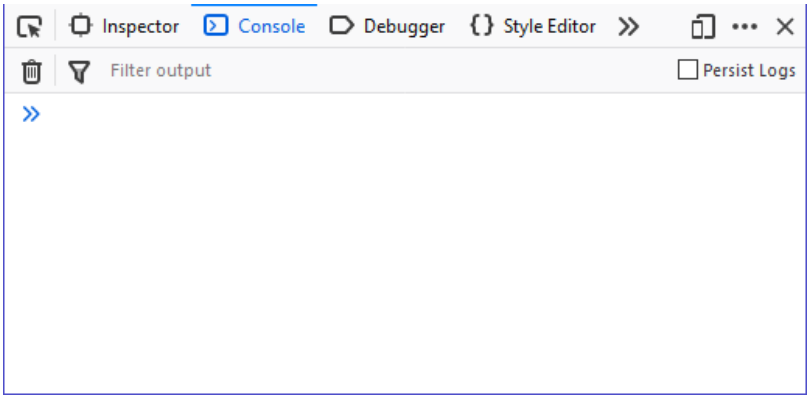


Figura 24: Console

Observe na Figura 25 que vamos utilizar o console para verificar o funcionamento de alguns dos operadores apresentados anteriormente.

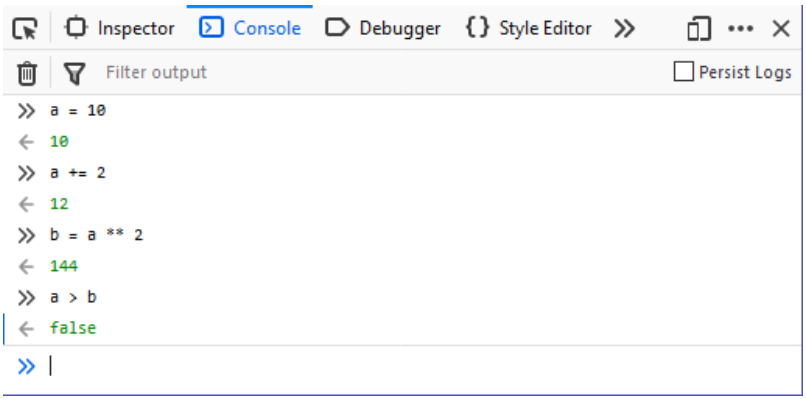


Figura 25: Avaliando o uso dos operadores

Esta é uma maneira muito rápida e fácil de verificar como determinado comando ou operação irá se comportar. O próximo exemplo mostra como podemos identificar erros nos scripts de uma página HTML. Observe no código-fonte a seguir que intencionalmente foi deixado um erro, pois a string atribuída à variável nome não foi adequadamente delimitada por aspas.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Caixas de Diálogo e Mensagem</title>
    <meta charset="UTF-8">
  </head>
  <body onload="iniciar()">
    <script>
    function iniciar() {
      var nome = "Zé Ninguém;
    }
    </script>
    <h1>Erro</h1>
```

```
  </body>
</html>
```

erro.html

Ao carregar a página no navegador, observe na Figura 26 que o console vai indicar que houve o erro a apontando a linha do arquivo, que, no nosso é exemplo, se trada da linha 10.

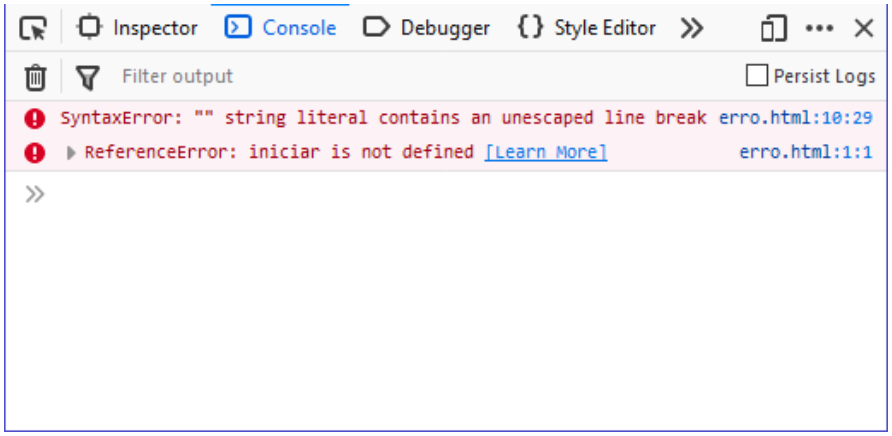


Figura 26: Identificando erros

Ao clicar na linha indicada o navegador exibirá o código-fonte da página, deixando a linha que apresenta o erro em destaque, conforme ilustra a Figura 27.

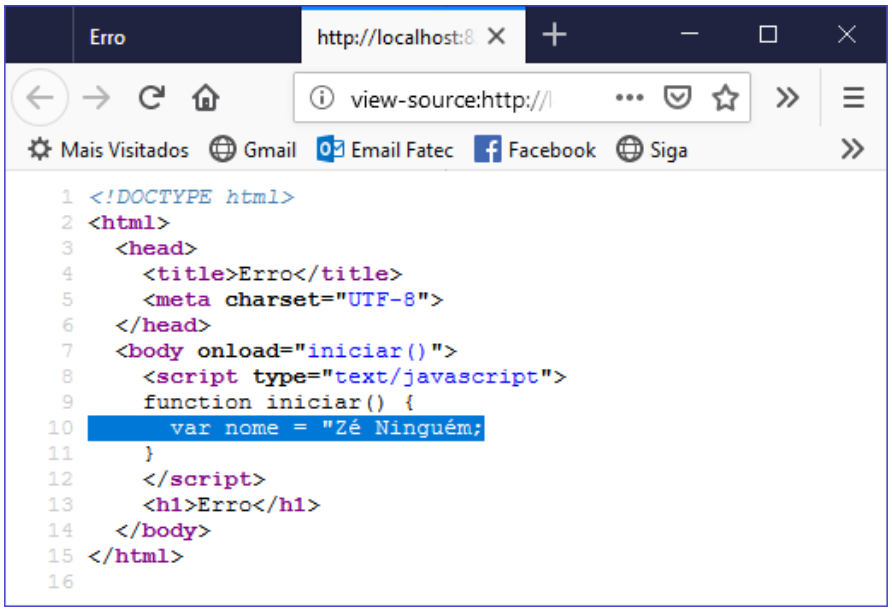


Figura 27: Código-fonte da página com o erro em destaque

Formulários: Caixas de texto e botões

Em JavaScript, como já abordado anteriormente, é possível realizar o acesso aos elementos de formulários definidos em HTML. Desta forma, consegue-se realizar verificações e alterações no conteúdo dos elementos, proporcionando maiores possibilidades de interação com o usuário final da aplicação. Neste exemplo uniremos os conceitos de formulários, variáveis, tipos de dados e operadores para construir uma página que realiza a soma de dois números.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Calculadora Simples</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Calculadora Simples</h1>
    Valor 1: <input type="text" id="v1" value="0"><br>
    Valor 2: <input type="text" id="v2" value="0"><br>
    <input type="button" value="Somar" id="btSomar"><br>
    Resultado: <input type="text" id="res" value="0" readonly>
    <script>
      btSomar.addEventListener("click", somar);

      function somar() {
        res.value = parseInt(v1.value) + parseInt(v2.value);
      }
      </script>
  </body>
</html>
```

somar.html

Na Figura 28 podemos observar o resultado da página após a interação do usuário:



Figura 28: Formulário

Estruturas de seleção e repetição

A linguagem JavaScript implementa as tradicionais estruturas para controle de fluxo do programa, disponíveis na grande maioria das linguagens de programação, são elas:

if: Permite definir a execução ou não de determinado bloco de código de acordo com a condição especificada. No exemplo abaixo se testa o valor da variável x e, de acordo com o seu valor, o comando if determina qual bloco de código deverá ser executado.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Estruturas de Controle</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Estruturas de Controle</h1>
    Digite um número:
    <input type="text" id="txNum" value="0">
    <input type="button" value="Ok" id="btOk">
    <div id="res"></div>
    <script>
    btOk.addEventListener("click", maiorMenorIgual);

    function maiorMenorIgual() {
      let num = parseFloat(txNum.value);
      if (num > 5) {
        res.innerHTML = "O valor de X é maior que cinco.";
      }
      else if (num < 5) {
        res.innerHTML = "O valor de X é menor que cinco.";
      }
      else {
        res.innerHTML = "O valor de X é igual a cinco.";
      }
    }
    </script>
  </body>
</html>
```

comando-if.html

switch: De acordo com o valor da variável testada pelo comando, é determinado o bloco de código que será executado. Neste exemplo, o número que representa o dia da semana é obtido e, com base em seu valor, o nome que representa o dia é armazenado em uma variável, sendo posteriormente exibido.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Dia da Semana</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Dia da Semana</h1>
    <input type="button" value="Obter" id="btObter">
    <div id="res"></div>
    <script>
    btObter.addEventListener("click", obterDiaSemana);

    function obterDiaSemana() {
      let hoje = new Date();
      let diaSemana = hoje.getDay();
      let nomeDiaSemana = "";
      switch(diaSemana) {
        case 0:
          nomeDiaSemana = "Domingo";
          break;
        case 1:
          nomeDiaSemana = "Segunda-feira";
          break;
        case 2:
          nomeDiaSemana = "Terça-feira";
          break;
        case 3:
          nomeDiaSemana = "Quarta-feira";
          break;
        case 4:
          nomeDiaSemana = "Quinta-feira";
          break;
        case 5:
          nomeDiaSemana = "Sexta-feira";
          break;
        case 6:
          nomeDiaSemana = "Sábado";
      }
      res.innerHTML = "Hoje é " + nomeDiaSemana;
    }
    </script>
  </body>
</html>
```

comando-switch.html

for: O comando permite a repetição de um bloco de instruções enquanto a condição especificada for verdadeira, sendo muito indicado quando se deseja trabalhar na solução de problemas que envolvem sequências numéricas. No exemplo abaixo, são mostrados os números pares entre 2 e 20.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Pares</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Pares</h1>
    <input type="button" value="Iniciar" id="btIniciar">
    <div id="res"></div>
    <script>
    btIniciar.addEventListener("click", pares);

    function pares() {
      res.innerHTML = "Contando: ";
      for (let i = 2; i <= 20; i+=2) {
        res.innerHTML += i + ", ";
      }
    }
    </script>
  </body>
</html>
```

comando-for.html

while: Similar ao comando for, executa a repetição de um bloco de programa, enquanto a condição for verdadeira. Neste exemplo são mostrados os números inteiros que estão em um intervalo digitado pelo usuário.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Intervalo</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Intervalo</h1>
    Início: <input type="text" id="txIni">
    Fim: <input type="text" id="txFim">
    <input type="button" value="Iniciar" id="btIniciar">
    <div id="res"></div>
    <script>
    btIniciar.addEventListener("click", contar);
```

```
    function contar() {
      let ini = parseInt(txIni.value);
      let fim = parseInt(txFim.value);
      let i = ini;
      res.innerHTML = "Contando: ";
      while (i <= fim) {
        res.innerHTML += (i++) + ", ";
      }
    }
    </script>
  </body>
</html>
```

comando-while.html

do-while: Similar aos comandos for e while apresentados anteriormente, porém, apresenta uma diferença fundamental, como a verificação da condição ocorre apenas após a execução do bloco de instruções a ser repetido, ele será executado pelo menos uma vez, independente se a condição for verdadeira ou falsa. Quando se utiliza for e while se a condição inicial resultar em falso o bloco a ser repetido nunca é executado. No exemplo abaixo, são mostrados os números pares entre 2 e 20.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Pares</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Pares</h1>
    <input type="button" value="Iniciar" id="btIniciar">
    <div id="res"></div>
    <script>
    btIniciar.addEventListener("click", pares);

    function pares() {
      res.innerHTML = "Contando: ";
      let i = 2;
      do {
        res.innerHTML += (i++) + ", ";
      } while (i <= 20);
    }
    </script>
  </body>
</html>
```

comando-do.html

Calculadora básica

Uma vez que os conceitos básicos sobre a interação entre formulários em HTML e a linguagem JavaScript foram abordados, é possível realizar algumas melhorias na página, desenvolvida anteriormente, que realizava a soma de dois números. O primeiro passo consiste em implementar as outras três operações aritméticas básicas. O modo mais fácil para fazer isso, consiste em criar um botão para cada operação junto ao respectivo evento associado à função. As outras alterações realizadas serão o suporte aos números reais (float) e evitar a divisão de um determinado número por zero.



```
<!DOCTYPE html>
<html>
  <head>
    <title>Calculadora Simples</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Calculadora Simples</h1>
    Valor 1: <input type="text" id="v1" value="0"><br>
    Valor 2: <input type="text" id="v2" value="0"><br>
    <input type="button" value="Somar" id="btSomar">
    <input type="button" value="Subtrair" id="btSubtrair">
    <input type="button" value="Multiplicar" id="btMultiplicar">
    <input type="button" value="Dividir" id="btDividir"><br>
    Resultado: <input type="text" id="res" value="0" readonly>
    <script>
      btSomar.addEventListener("click", somar);
      btSubtrair.addEventListener("click", subtrair);
      btMultiplicar.addEventListener("click", multiplicar);
      btDividir.addEventListener("click", dividir);

      function somar() {
        res.value = parseFloat(v1.value) + parseFloat(v2.value);
      }

      function subtrair() {
        res.value = parseFloat(v1.value) -parseFloat(v2.value);
      }

    function multiplicar() {
        res.value = parseFloat(v1.value) * parseFloat(v2.value);
      }

      function dividir() {
        if (parseFloat(v2.value) === 0)
          res.value = "Erro!";
        else
          res.value = parseFloat(v1.value) / parseFloat(v2.value);
      }
      </script>
```

```
  </body>
</html>
```

calculadora.html

Na Figura 29 mostramos o resultado da subtração dos dois valores digitados pelo usuário.

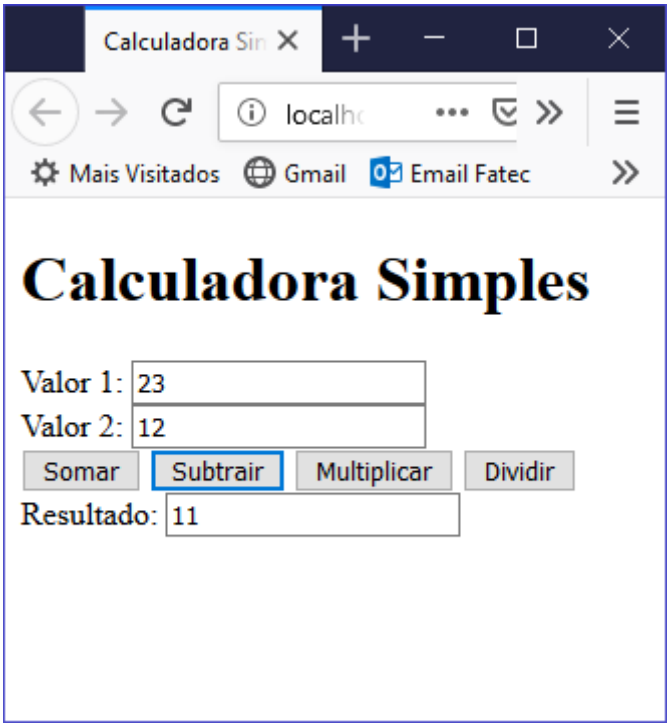


Figura 29: Formulário

Outra implementação possível consiste em criar uma única função que realizará todas as operações. Para que isso seja possível, a função deverá receber um parâmetro indicando qual operação será executada, como ilustrado no trecho de código-fonte a seguir considerando o botão Somar.

{JS}

```
btSomar.addEventListener("click", calcular);
btSomar.op = "+";
```

Em seguida a função deverá, com base no valor do parâmetro, executar a operação solicitada, conforme mostrado no seguinte código-fonte.

</>

```
<!DOCTYPE html>
<html>
  <head>
    <title>Calculadora Simples</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Calculadora Simples</h1>
    Valor 1: <input type="text" id="v1" value="0"><br>
```

```
    Valor 2: <input type="text" id="v2" value="0"><br>
    <input type="button" value="Somar" id="btSomar">
    <input type="button" value="Subtrair" id="btSubtrair">
    <input type="button" value="Multiplicar" id="btMultiplicar">
    <input type="button" value="Dividir" id="btDividir"><br>
    Resultado: <input type="text" id="res" value="0" readonly>
    <script>
      btSomar.addEventListener("click", calcular);
      btSomar.op = "+";
      btSubtrair.addEventListener("click", calcular);
      btSubtrair.op = "-";
      btMultiplicar.addEventListener("click", calcular);
      btMultiplicar.op = "*";
      btDividir.addEventListener("click", calcular);
      btDividir.op = "/";

      function calcular(evt) {
        let op = evt.currentTarget.op;
        if (op === '+')
          res.value = parseFloat(v1.value) + parseFloat(v2.value);
        else if (op === '-')
          res.value = parseFloat(v1.value) - parseFloat(v2.value);
        else if (op === '*')
          res.value = parseFloat(v1.value) * parseFloat(v2.value);
        else {
          if (parseFloat(v2.value) === 0)
            res.value = 'Erro!';
          else
            res.value = parseFloat(v1.value) / parseFloat(v2.value);
        }
      }
      </script>
  </body>
</html>
```

calculadora-parametro0.html

Outro modo de realizar a passagem de parâmetros para a função, consiste em declarar uma função no momento da criação do eventListener e realizar a chamada da função calcular a partir desta função, conforme ilustra o programa apresentado a seguir.

</>

```
<!DOCTYPE html>
<html>
  <head>
    <title>Calculadora Simples</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Calculadora Simples</h1>
    Valor 1: <input type="text" id="v1" value="0"><br>
    Valor 2: <input type="text" id="v2" value="0"><br>
```

```
    <input type="button" value="Somar" id="btSomar">
    <input type="button" value="Subtrair" id="btSubtrair">
    <input type="button" value="Multiplicar" id="btMultiplicar">
    <input type="button" value="Dividir" id="btDividir"><br>
    Resultado: <input type="text" id="res" value="0" readonly>
    <script>
      btSomar.addEventListener("click", function()
{ calcular("+"); });
      btSubtrair.addEventListener("click", function() { calcular("-");
});
      btMultiplicar.addEventListener("click", function()
{ calcular("*"); });
      btDividir.addEventListener("click", function()
{ calcular("/"); });

      function calcular(op) {
        if (op === '+')
          res.value = parseFloat(v1.value) + parseFloat(v2.value);
        else if (op === '-')
          res.value = parseFloat(v1.value) - parseFloat(v2.value);
        else if (op === '*')
          res.value = parseFloat(v1.value) * parseFloat(v2.value);
        else {
          if (parseFloat(v2.value) === 0)
            res.value = 'Erro!';
          else
            res.value = parseFloat(v1.value) / parseFloat(v2.value);
        }
      }
      </script>
  </body>
</html>
```

calculadora-parametro1.html

Neste exemplo, podemos também empregar o conceito arrow functions (funções de seta) para declarar a função de callback, conforme mostrado a seguir.

{JS}

```
btSomar.addEventListener("click", () => { calcular("+"); });
btSubtrair.addEventListener("click", () => { calcular("-"); });
btMultiplicar.addEventListener("click", () => { calcular("*"); });
btDividir.addEventListener("click", () => { calcular("/"); });
```

Arrow functions em JavaScript consistem em uma sintaxe concisa para escrever funções, substituindo a instrução function por =>. Elas devem ser sempre anônimas, sendo muito úteis em funções de callback.

Vetores

O objeto Array permite a definição de vetores cujos elementos poderão ser acessados posteriormente através do respectivo índice.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Montadoras</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Montadoras</h1>
    <input type="button" value="Exibir" id="btExibir">
    <div id="res"></div>
    <input type="button" value="Esconder" id="btEsconder">
    <script>
    btExibir.addEventListener("click", exibir);
    btEsconder.addEventListener("click", esconder);
    btEsconder.style.visibility = 'hidden';

    function exibir() {
      let montadora = new Array("Fiat", "Ford", "General Motors",
        "Honda", "Nissan", "Renault", "Volkswagen");
      res.innerHTML = "";
      for (var i = 0; i < montadora.length; i++)
        res.innerHTML += montadora[i] + "<br>";
      btExibir.disabled = true;
      btEsconder.style.visibility = 'visible';
    }

    function esconder() {
      res.innerHTML = "";
      btExibir.disabled = false;
      btEsconder.style.visibility = 'hidden';
    }
    </script>
  </body>
</html>
```

vetor-0.html

Neste exemplo um vetor é declarado e inicializado com alguns itens, posteriormente, a rotina utiliza o comando de repetição for para acessar os elementos do vetor e os exibir na página HTML, conforme ilustrado pela Figura 30.

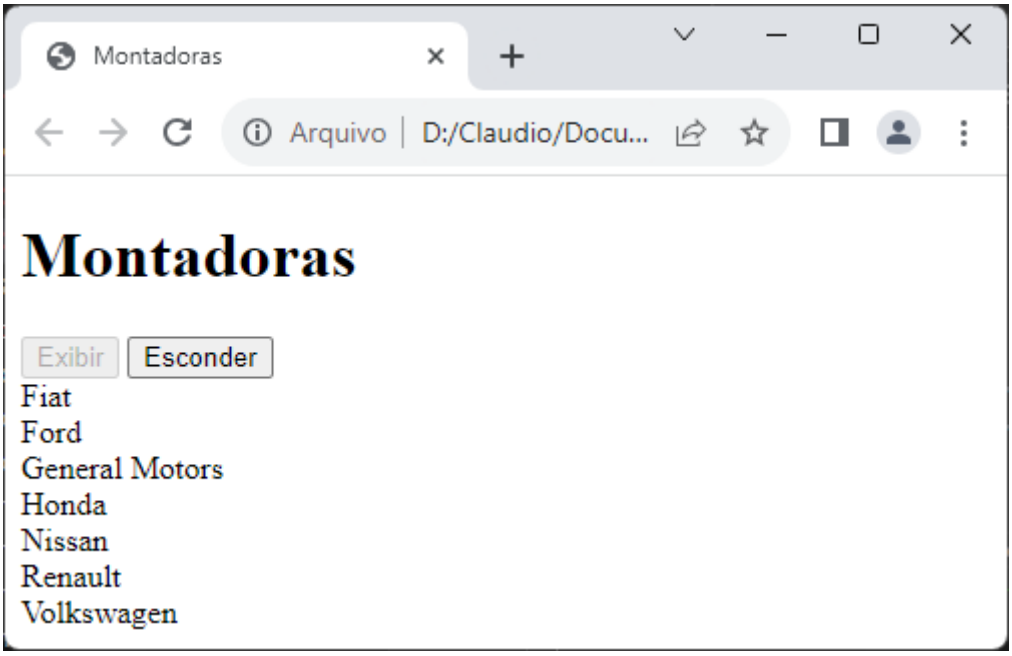


Figura 30: Exemplo de uso de vetores

Um vetor pode ser manipulado de maneira dinâmica, permitindo a inserção, alteração, remoção e busca dos itens. A página apresentada a seguir implementa uma lista de compra, exemplificando algumas operações que podem ser realizadas em um vetor.

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="UTF-8">
  <title>Lista de Compras</title>
</head>

<body>
  <h1>Lista de Compras</h1>
  Item:
  <input type="text" id="item">
  <input type="button" value="+" id="btInserir">
  <input type="button" value="-" id="btRemover">
  <h2>Sua lista:</h2>
  <div>
    <ol id="lista">
    </ol>
  </div>
  <script>
    btInserir.addEventListener('click', inserir);
    btRemover.addEventListener('click', remover);

    let itens = new Array(); // Escopo GLOBAL

    function exibir() {
      lista.innerHTML = '';
```

```
      for (let i = 0; i < itens.length; i++) {
        lista.innerHTML += '<li>' + itens[i] + '</li>';
      }
    }

    function inserir() {
      itens[itens.length] = item.value;
      exibir();
    }

    function remover() {
      for (let i = 0; i < itens.length; i++) {
        if (itens[i] == item.value) {
          itens.splice(i, 1);
        }
      }
      exibir();
    }
  </script>
</body>

</html>
```

vetor-1.html

Na página desenvolvida, notamos que a função inserir adiciona um elemento ao final da lista. Para isso, basta acessar o índice do próximo elemento disponível da lista através do atributo length. A exclusão, realizada através da função remover, percorre toda a lista a procura do item digitado na caixa de texto. Ao encontrar, o método splice é usado para apagar o item. Após cada inserção ou remoção a função exibir atualiza a exibição dos elementos da lista (Figura 31).



Figura 31: Lista de compras

Exibição de imagens

A linguagem JavaScript é bastante versátil no acesso a arquivos contendo imagens, permitindo que uma imagem possa ser trocada por outra, sem que com isso, a página HTML original precise ser recarregada. Esta característica, aliada aos eventos de um objeto, permite dar dinamismo a páginas, sem a necessidade de scripts com código complexo.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Imagens</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <script>
    function mudar(objeto, imagem) {
      objeto.src = imagem;
    }
    </script>
    <h1>Imagens</h1>
    <a onmouseover="mudar(lampada, 'lamp-lig.png')"
      onmouseout="mudar(lampada, 'lamp-desl.png')">
      <img src="lamp-desl.png" border="0" id="lampada">
    </a>
  </body>
</html>
```

imagem-0.html

Neste exemplo, mostrado na Figura 32, a imagem exibida na página é trocada sempre que o ponteiro do mouse entra ou sai da área ocupada pela imagem.



Figura 32: Substituindo uma figura através do JavaScript

A seguir o mesmo exemplo foi reescrito, separando os eventos do código-fonte HTML, ou seja, não utilizando os atributos onmouseover e onmouseout da etiqueta <a> e adicionando os respectivos EventListeners.



```
<!DOCTYPE html>
<html>
  <head>
    <title>Imagens</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Imagens</h1>
    <a id="ancora">
      <img src="./img/lamp-desl.png" border="0" id="lampada">
    </a>
    <script>
    ancora.addEventListener('mouseout', function() { mudar(lampada,
"./img/lamp-desl.png"); });
    ancora.addEventListener('mouseover', function() { mudar(lampada,
"./img/lamp-lig.png"); });

    function mudar(objeto, imagem) {
      objeto.src = imagem;
    }
    </script>
  </body>
</html>
```

imagem-1.html



Pratique!

Alterar o programa para utilizar Arrow Functions nas funções de callback.

Interação com CSS

Os atributos da folha de estilo de um objeto podem ser acessados pelo atributo style. Na página seguinte definimos margem (margin), espaçamento (padding) e cor de fundo (backgroundColor) do objeto area, que corresponde ao elemento DIV definido no HTML. Além disso, usamos os métodos da classe Math para gerar um número aleatório (random). O número sorteado será um valor real maior ou igual a 0 (zero) e menor do que 1 (um). Na sequência, o número é multiplicado por dois, arredondado pelo método floor e adicionado 1. Desta forma, obteremos o valor 1 ou 2, que correspondem a uma das faces da moeda.

```
<!DOCTYPE html>
<html lang="pt-br">

<head>
  <meta charset="UTF-8">
  <title>Cara ou coroa</title>
</head>

<body>
  <h1>Cara ou coroa</h1>
  <input type="button" value="Sortear" id="btSortear">
  <div id="area"></div>
  <script>
    btSortear.addEventListener('click', sortear);

    function sortear() {
      let face = Math.floor(Math.random() * 2) + 1;
      area.innerHTML += '<img src="./img/moeda_' + face + '.jpg">';
      area.style.margin = '25px 0px';
      area.style.padding = '10px';
      area.style.backgroundColor = 'rgb(176, 224, 230)';
    }
  </script>
</body>

</html>
```

moedas-0.html

Cada vez que o botão Sortear for pressionado será sorteada e exibida uma imagem correspondente a cara ou coroa (Figura 33).

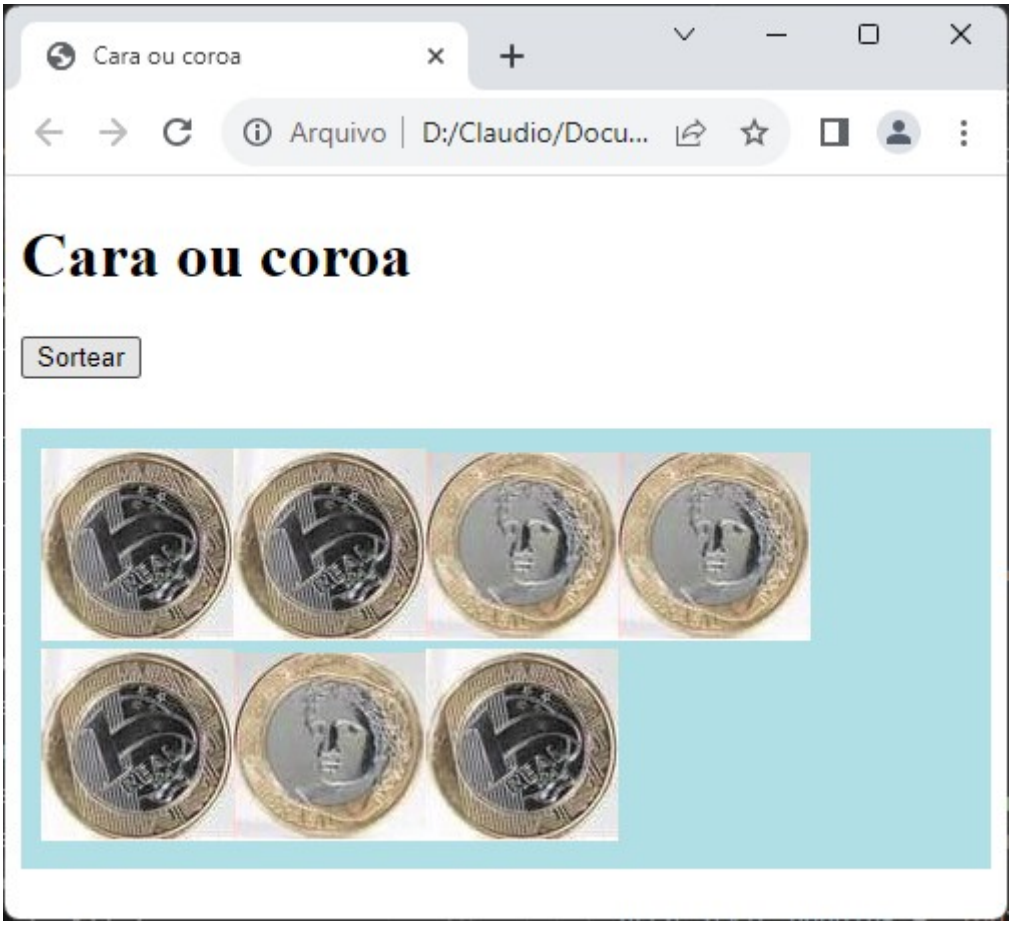


Figura 33: Cara ou coroa


Pratique!

Alterar o programa para exibir a porcentagem de caras e coroas e a quantidade de sorteios que foram realizados.

Exercícios

1) A Mega-Sena é a maior loteria do Brasil, sendo organizada pela Caixa Econômica Federal. Para jogar, você pode escolher de 6 a 20 números entre 60 disponíveis (1 até 60). Co base nestas informações, criar uma página que realize a sugestão de números para a Mega-Sena. O usuário deverá informar quantos números ele quer sortear (6 a 20) e também quantos jogos pretende realizar.

Formulários: Botões de rádio

Os botões de rádio são amplamente utilizados em formulário quando é necessário restringir os valores de uma determinada entrada dados a um conjunto de opções possíveis. Tomando como referência o exemplo da calculadora será utilizado um conjunto de botões de rádio para que o usuário possa selecionar a operação a ser realizada.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Calculadora Simples</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Calculadora Simples</h1>
    Valor 1: <input type="text" id="v1" value="0"><br>
    Valor 2: <input type="text" id="v2" value="0"><br>
    Operação:
    <input type="radio" id="somar" name="op" checked> Somar
    <input type="radio" id="subtrair" name="op"> Subtrair
    <input type="radio" id="multiplicar" name="op"> Multiplicar
    <input type="radio" id="dividir" name="op"> Dividir<br>
    <input type="button" value="Calcular" id="btCalc"><br>
    Resultado: <input type="text" id="res" value="0" readonly>
    <script>
      btCalc.addEventListener("click", calcular);

      function calcular() {
        if (somar.checked === true)
          res.value = parseFloat(v1.value) + parseFloat(v2.value);
        else if (subtrair.checked === true)
          res.value = parseFloat(v1.value) - parseFloat(v2.value);
        else if (multiplicar.checked === true)
          res.value = parseFloat(v1.value) * parseFloat(v2.value);
        else {
          if (parseFloat(v2.value) === 0)
            res.value = "Erro!";
          else
            res.value = parseFloat(v1.value) / parseFloat(v2.value);
        }
      }
      </script>
  </body>
</html>
```

calculadora-radio.html

A Figura 34 mostra a implementação da página da calculadora básica usando os botões de rádio.



Figura 34: Botões de Rádio

Formulários: Caixas de seleção

Outro elemento disponível em formulário que possibilita limitar a entrada de dados a um conjunto pré-determinado de valores são as caixas de seleção. Observe que no formulário HTML devemos utilizar a etiqueta select e acrescentando também as respectivas opções, conforme podemos notar no trecho de código-fonte a seguir.

```
Operação: <select id="op">
  <option value="">(Escolha uma opção)</option>
  <option value="+">Somar</option>
  <option value="-">Subtrair</option>
  <option value="*">Multiplicar</option>
  <option value="/">Dividir</option>
</select>
```

Observe no seguinte trecho de código-fonte que a função calcular deverá determinar o valor que foi escolhido pelo usuário na caixa de seleção através do atributo value.

```
function calcular() {
  if (op.value === '+')
    res.value = parseFloat(v1.value) + parseFloat(v2.value);
  else if (op.value === '-')
    res.value = parseFloat(v1.value) - parseFloat(v2.value);
  // ...
}
```

A seguir apresentamos a implementação completa da página.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Calculadora Simples</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Calculadora Simples</h1>
    Valor 1: <input type="text" id="v1" value="0"><br>
    Valor 2: <input type="text" id="v2" value="0"><br>
    Operação:
    <select id="op">
      <option value="">(Escolha uma opção)</option>
      <option value="+">Somar</option>
      <option value="-">Subtrair</option>
      <option value="*">Multiplicar</option>
      <option value="/">Dividir</option>
    </select><br>
    <input type="button" value="Calcular" id="btCalc"><br>
```

```
    Resultado: <input type="text" id="res" value="0" readonly>
    <script>
      btCalc.addEventListener("click", calcular);

      function calcular() {
        if (op.value === '+')
    res.value = parseFloat(v1.value) + parseFloat(v2.value);
        else if (op.value === '-')
    res.value = parseFloat(v1.value) - parseFloat(v2.value);
        else if (op.value === '*')
          res.value = parseFloat(v1.value) * parseFloat(v2.value);
        else if (op.value === '/') {
          if (parseFloat(v2.value) === 0)
            res.value = 'Erro!';
          else
            res.value = parseFloat(v1.value) / parseFloat(v2.value);
        }
        else
          alert('Escolha a operação desejada!');
      }
      </script>
  </body>
</html>
```

calculadora-selecao.html

O resultado da execução do script produzirá um resultado semelhante ao mostrado na Figura 35.



Figura 35: Utilização da Caixa de Seleção

JavaScript e CSS externos

Muitas vezes a solução de um determinado problema computacional acaba por gerar longos arquivos de código-fonte. A modularização é utilizada para permitir agrupar melhor o código-fonte facilitando o entendimento e as tarefas de manutenção. Desta forma, é possível criarmos arquivos externos apenas com o código-fonte em CSS e outro com JavaScript mantendo-os separados do conteúdo HTML. Este conceito será mostrado no próximo exemplo, onde será desenvolvida uma calculadora com uma interface gráfica "estilo" Windows. Assim sendo, em primeiro lugar, criaremos apenas a página com o conteúdo HTML.



```
<!DOCTYPE html>
<html>
  <head>
    <title>Calculadora</title>
    <meta charset="UTF-8">
    <link rel="stylesheet" type="text/css" href="calculadora.css" />
  </head>
  <body>
    <h2>Calculadora</h2>
    <table>
      <tr>
        <td colspan="4">
          <input type="text" id="res" value="0" size="16" readonly>
          <input type="hidden" id="v1" value="0">
          <input type="hidden" id="op" value="">
        </td>
      </tr>
      <tr>
        <td><input type="button" id="off" value="Off"></td>
        <td><input type="button" id="lim" value=" C "></td>
        <td> </td>
        <td> </td>
      </tr>
      <tr>
        <td><input type="button" id="d7" value=" 7 "></td>
        <td><input type="button" id="d8" value=" 8 "></td>
        <td><input type="button" id="d9" value=" 9 "></td>
        <td><input type="button" id="div" value=" &divide; "></td>
      </tr>
      <tr>
        <td><input type="button" id="d4" value=" 4 "></td>
        <td><input type="button" id="d5" value=" 5 "></td>
        <td><input type="button" id="d6" value=" 6 "></td>
        <td><input type="button" id="mul" value=" &times; "></td>
      </tr>
      <tr>
        <td><input type="button" id="d1" value=" 1 "></td>
        <td><input type="button" id="d2" value=" 2 "></td>
        <td><input type="button" id="d3" value=" 3 "></td>
        <td><input type="button" id="sub" value=" &minus; "></td>
```

```
      </tr>
      <tr>
        <td><input type="button" id="d0" value=" 0 "></td>
        <td><input type="button" id="sep" value="  , "></td>
        <td><input type="button" id="tot" value=" = "></td>
        <td><input type="button" id="adi" value=" &plus; "></td>
      </tr>
    </table>
    <script type="text/javascript" src="calculadora.js"></script>
  </body>
</html>
```

calculadora-completa.html

Na página HTML deve ocorrer a chamada aos arquivos externos. A etiqueta LINK irá realizar a referência ao arquivo CSS que deverá ser usado.

</>

```
<link rel="stylesheet" type="text/css" href="calculadora.css" />
```

Observe também que a chamada ao arquivo externo JavaScript será efetuada dentro da etiqueta SCRIPT através do atributo src.

</>

```
<script type="text/javascript" src="calculadora.js"></script>
```

Em seguida implementamos o arquivo CSS.

{CSS}

```
table {
  background-color: rgb(176, 224, 230);
  border: 0px;
  padding: 4px;
  border-spacing: 0px;
}

#res {
  display: inline-block;
  text-align: right;
}
```

calculadora.css

Na sequência devemos criar o arquivo calculadora.js que deve conter as funções em JavaScript.

{JS}

```
d0.addEventListener('click', function() { digito('0');});
d1.addEventListener('click', function() { digito('1');});
d2.addEventListener('click', function() { digito('2');});
d3.addEventListener('click', function() { digito('3');});
```

```
d4.addEventListener('click', function() { digito('4');});
d5.addEventListener('click', function() { digito('5');});
d6.addEventListener('click', function() { digito('6');});
d7.addEventListener('click', function() { digito('7');});
d8.addEventListener('click', function() { digito('8');});
d9.addEventListener('click', function() { digito('9');});
div.addEventListener('click', function() { operacao("/");});
mul.addEventListener('click', function() { operacao("X");});
sub.addEventListener('click', function() { operacao("-");});
adi.addEventListener('click', function() { operacao("+");});
off.addEventListener('click', retornar);
lim.addEventListener('click', limpar);
sep.addEventListener('click', separador);
tot.addEventListener('click', total);

function string2Number(valor) {
  valor = valor.replace(",", ".");
  return(parseFloat(valor));
}

function number2String(valor) {
  valor = valor.toString().replace('.', ',');
  return (valor);
}

function digito(dig) {
  if (res.value.length < 12) {
    if (res.value != "0")
      res.value = res.value + dig;
    else
      res.value = dig;
  }
}

function total() {
  let n1 = string2Number(v1.value);
  let n2 = string2Number(res.value);
  if (op.value === "+")
    res.value = number2String(n1 + n2);
  else if (op.value === "-")
    res.value = number2String(n1 - n2);
  else if (op.value === "X")
    res.value = number2String(n1 * n2);
  else if (op.value === "/") {
    if (res.value !== 0)
      res.value = number2String(n1 / n2);
    else
      res.value = "Erro!";
  }
}
```

```
function operacao(ope) {
  v1.value = res.value;
  op.value = ope;
  res.value = "0";
}

function separador() {
  if (res.value.indexOf(",") < 0)
    res.value = res.value + ",";
}

function limpar() {
  res.value = "0";
}

function retornar() {
}
```

calculadora.js

Na Figura 36 podemos observar a aparência da página criada.



Figura 36: Exemplo completo da calculadora

Exercícios

1) Considerando a aplicação calculadora e adotando CSS e JavaScript externos, adicionar as operações de raiz quadrada, exponenciação e também uma área de texto sobre o "visor" que registre todas as operações efetuadas, conforme ilustra a Figura 37.

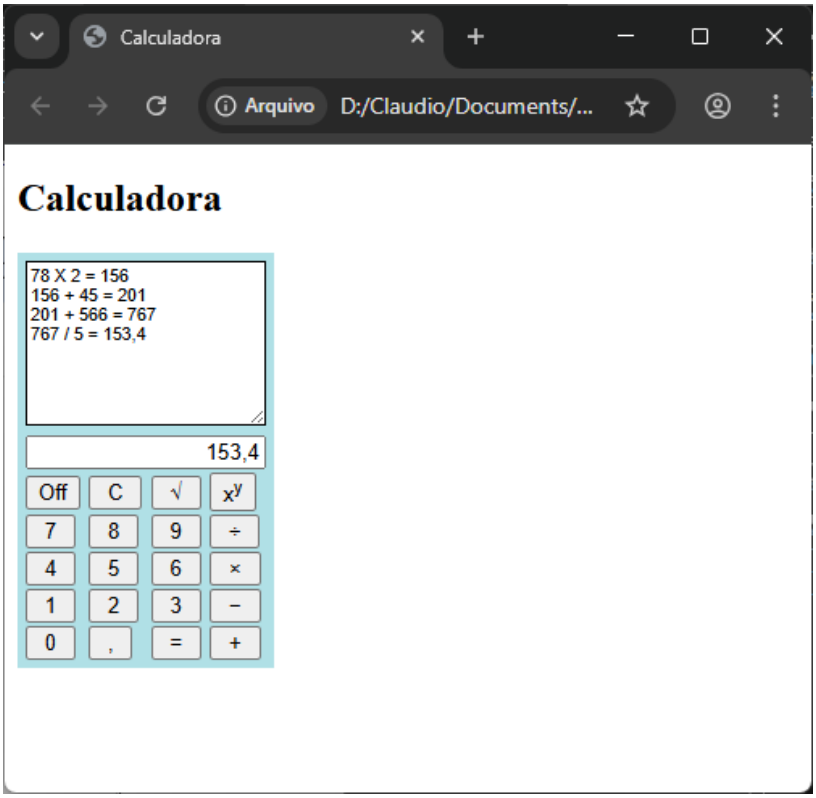


Figura 37: Alterações propostas para a calculadora

Interação entre páginas

A linguagem JavaScript possui objetos que permitem realizar a interação entre diferentes páginas HTML. Neste exemplo, será mostrado o código necessário para que a calculadora, desenvolvida anteriormente, preencha um campo do tipo texto existente em uma outra página HTML.

A página principal vai conter uma função em JavaScript que abre a página HTML que contém o programa da calculadora. Esta função será chamada quando o usuário clicar no botão calculadora, também implementaremos o campo texto que deverá receber o valor disponível na calculadora. Nesta página, será utilizado o método open do objeto window que permite criar uma instância do navegador, controlando parâmetros como tamanho, localização, barras de menu entre outros.

```
<!DOCTYPE html>
<html>

<head>
  <title>Interação entre páginas</title>
  <meta charset="UTF-8">
</head>

<body>
  <h1>Interação entre páginas</h1>
  Digite um valor ou utilize a
  <input type="button" value="calculadora" id="calc">:
  <input type="text" id="valor" value="0">
  <script>
    calc.addEventListener('click', abrirCalc);
    window.addEventListener("message", function (evento) {
      valor.value = evento.data;
    }, false);

    function abrirCalc() {
      var janela = window.open('calculadora-completa.html',
        'Calculadora', 'toolbar=no, \
        location=no,directories=no,status=no,menubar=no, \
        scrollbars=no,resizable=no,menubar=no,top=250,left=450, \
        width=200,height=300');
    }
  </script>
</body>

</html>
```

interacao.html

A função retornar que está implementada no arquivo *calculadora.js* deverá enviar uma mensagem (postMessage) para a página que a chamou. Esta mensagem contém o valor de que está caixa de texto res e, em seguida, esta mesma função vai fechar a janela (close).

{JS}

```
function retornar() {
  window.opener.postMessage(res.value, "*");
  window.close();
}
```

O funcionamento é simples, basta carregar a página principal e, ao clicar sobre o botão calculadora, a página *calculadora-completa.html* é exibida. Após usar a calculadora, ao clicar no botão Off, o conteúdo que está no visor é transferido para o campo texto da página anterior e a janela que contém a calculadora é fechada, conforme podemos notar na Figura 38.

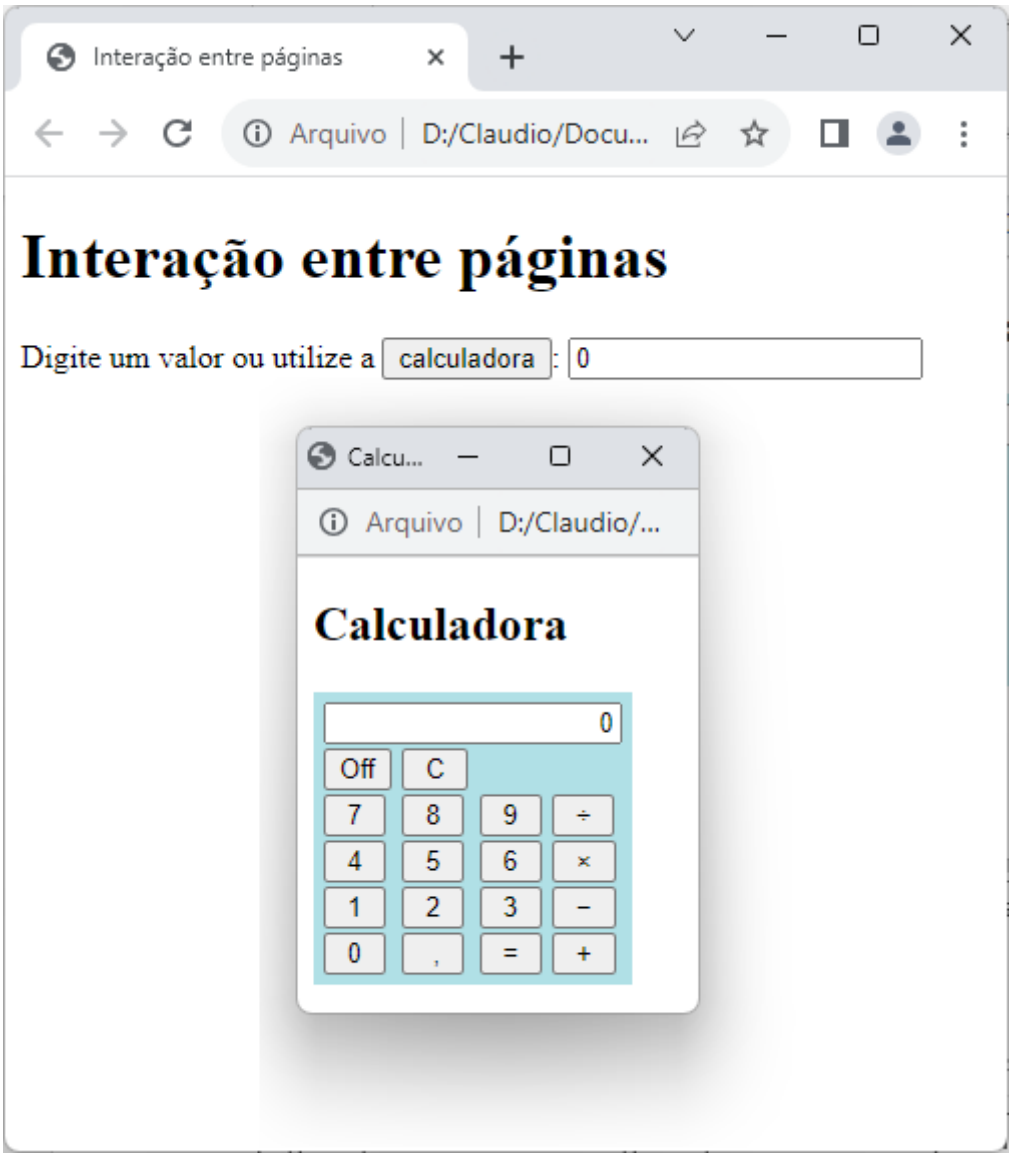


Figura 38: Interação entre as páginas

Formulários: Validação

Um aspecto importante do JavaScript é permitir a verificação e validação local dos dados de um formulário. Desta forma, não é necessário realizar requisições ao servidor web, o que aumenta significativamente a capacidade de interação com o usuário e o desempenho da página.

A página que será criada deverá obter a descrição, preço unitário e quantidade de um determinado produto para, posteriormente, calcular o preço total. Os dados que o usuário vai digitar no formulário deverão ser validados aplicando as seguintes regras:

a) Todos os campos devem ser obrigatoriamente preenchidos.
b) O campo quantidade deverá aceitar um número inteiro entre 1 e 999.
c) O campo preço unitário deverá aceitar apenas números reais positivos.

Para a realizar a validação existe o evento blur o qual permite executar uma função quando o campo de texto perde o foco, ou seja, o cursor deixa o campo seja através do pressionamento do botão esquerto do mouse em outra área da tela ou pressionando da tecla de tabulação. Desta forma, para validar o campo descrição teríamos, por exemplo:

```
desc.addEventListener("blur", validarDescricao);
```

A função em JavaScript que seria chamada na ocorrência do evento blur, poderia ser escrita da maneira mostrada a seguir, onde inicialmente verificamos se o valor do campo desc (descrição do produto) está vazio. O método trim irá desconsiderar na comparação os espaços em branco que, eventualmente, possam ocorrer no início e/ou no fim do valor digitado no campo.

```
function validarDescricao() {
  if (desc.value.trim() === "") {
    msgErro.innerHTML = "Preencha a descrição do produto!";
    desc.style.background = "yellow";
    desc.focus();
    return false;
  }
  else {
    desc.style.background = "white";
    msgErro.innerHTML = "";
    return true;
  }
}
```

Quando o campo estiver vazio ele será destacado em amarelo e uma mensagem será mostrada, conforme ilustra a Figura 39. Além disso, quando a validação falhar, a função retornará falso, caso contrário, o retorno será verdadeiro.

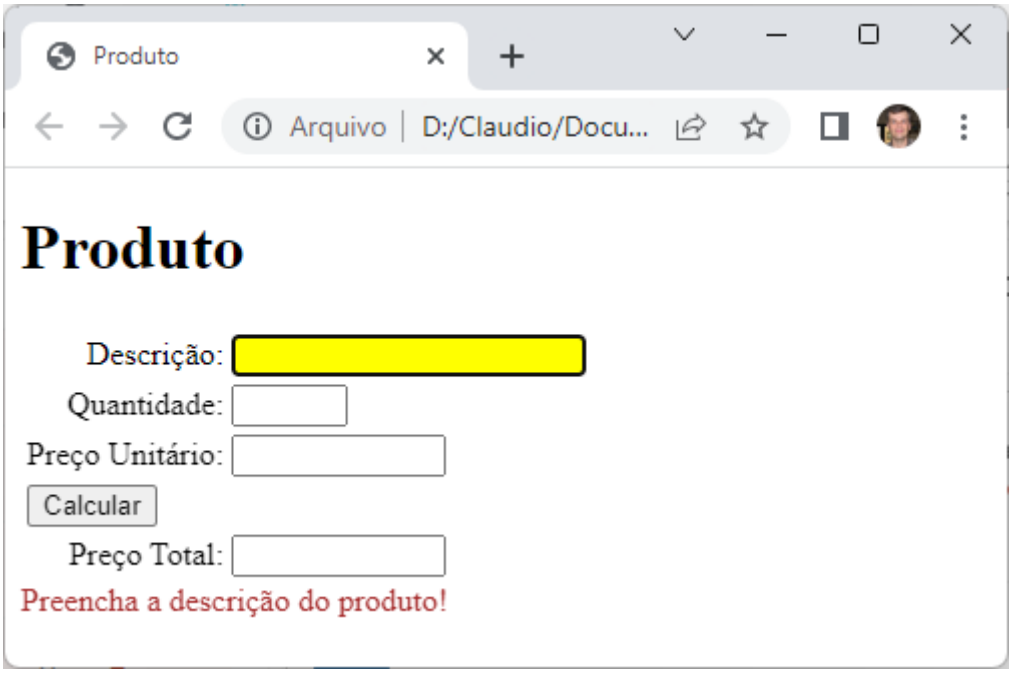


Figura 39: Alertando que o campo não foi preenchido

A seguir apresentamos a implementação da página HTML.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Produto</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Produto</h1>
    <table border="0">
      <tr>
        <td align="right">Descrição:</td>
        <td><input type="text" id="desc" value="" size="20"
maxlength="20" /></td>
      </tr>
      <tr>
        <td align="right">Quantidade:</td>
        <td><input type="text" id="qtd" value="" size="3"
maxlength="3" /></td>
      </tr>
      <tr>
        <td align="right">Preço Unitário:</td>
        <td><input type="text" id="unit" value="" size="10"
maxlength="10" /></td>
      </tr>
      <tr>
        <td colspan="2"><input type="button" value="Calcular"
id="calc"/></td>
      </tr>
```

```
      <tr>
        <td align="right">Preço Total:</td>
        <td><input type="text" id="total" value="" size="10"
readonly /></td>
      </tr>
    </table>
    <div id="msgErro" style="color:brown"></div>
    <script src="produto-0.js"></script>
  </body>
</html>
```

produto-0.html

Como neste exemplo o código-fonte JavaScript é mais longo, vamos organizar as rotinas em um arquivo externo, separadas do HTML, sendo que este arquivo deverá possuir a extensão js e a etiqueta <SCRIPT> faz a referência ao arquivo.

{JS}

```
desc.addEventListener("blur", validarDescricao);
qtd.addEventListener("blur", validarQuantidade);
unit.addEventListener("blur", validarPreco);
calc.addEventListener("click", calcular);

function validarDescricao() {
  if (desc.value.trim()==="") {
    msgErro.innerHTML="Preencha a descrição do produto!";
    desc.style.background="yellow";
    desc.focus();
    return false;
  }
  else {
    desc.style.background="white";
    msgErro.innerHTML="";
    return true;
  }
}

function validarQuantidade() {
  let erro=false;

  if (qtd.value.trim()==="") {
    erro=true;
  }
  else {
    if (isNaN(qtd.value)===true) {
      erro=true;
    }
    else {
      let nQtd=parseInt(qtd.value);

      if (nQtd < 1 || nQtd > 999) {
        erro=true;
```

```
      }
    }
  }

  if (erro===true) {
    msgErro.innerHTML="A quantidade deve ser um número entre 1 e
999!";
    qtd.style.background="yellow";
    qtd.focus();
  }
  else {
    qtd.style.background="white";
    msgErro.innerHTML="";
  }
  return (!erro);
}

function validarPreco() {
  let erro=false;

  if (unit.value.trim()==="") {
    erro=true;
  }
  else {
    if (isNaN(unit.value)===true) {
      erro=true;
    }
    else {
      let nUnit=parseFloat(unit.value);

      if (nUnit <=0.0) {
        erro=true;
      }
      else {
        unit.value=nUnit.toFixed(2);
      }
    }
  }

  if (erro===true) {
    msgErro.innerHTML="O preço unitário deve ser um número maior que
zero!";
    unit.style.background="yellow";
    unit.focus();
  }
  else {
    unit.style.background="white";
    msgErro.innerHTML="";
  }
  return (!erro);
}
```

```
function calcular() {
  if (validarDescricao() && validarQuantidade() && validarPreco()) {
    var nTotal=parseFloat(unit.value) * parseInt(qtd.value);
    total.value=nTotal.toFixed(2);
  }
}
```

produto-0.js

Na Figura 40 temos o resultado devidamente calculado e formatado, após a validação dos dados digitados.

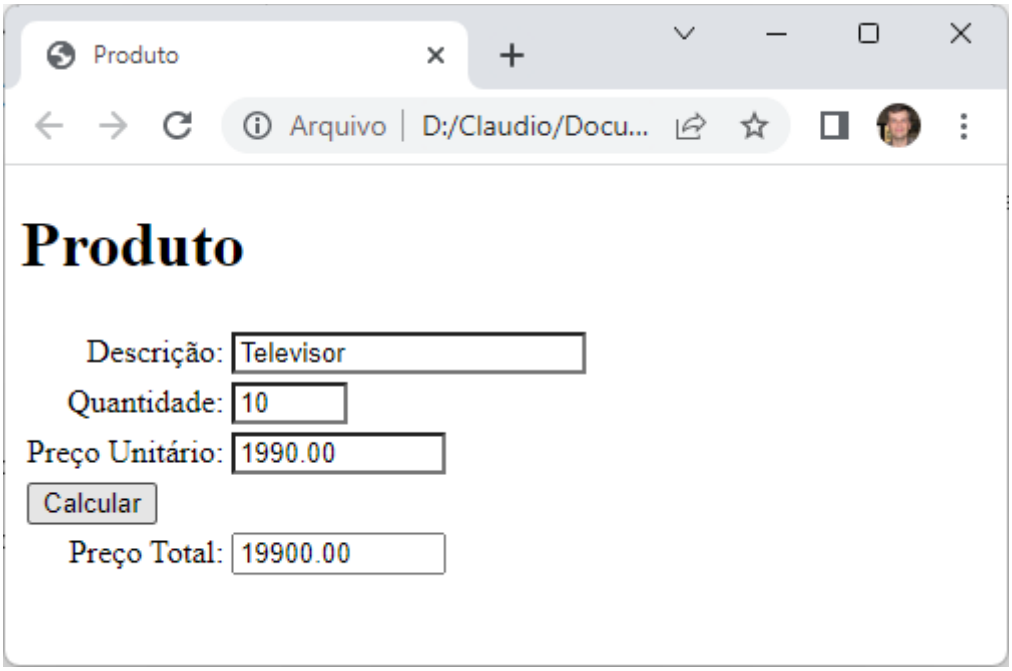


Figura 40: Cálculo do preço total

Outra possibilidade consiste em exibir as mensagens de erro individualizadas em elementos SPAN do HTML, conforme apresentado no seguinte programa que consiste em um arquivo HTML e outro em JavaScript.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Produto</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Produto</h1>
    <table border="0">
      <tr>
        <td align="right">Descrição:</td>
        <td><input type="text" id="desc" value="" size="20"
maxlength="20" /> <span id="eDesc"></span></td>
      </tr>
```

```
      <tr>
        <td align="right">Quantidade:</td>
        <td><input type="text" id="qtd" value="" size="3"
maxlength="3" /> <span id="eQtd"></span></td>
      </tr>
      <tr>
        <td align="right">Preço Unitário:</td>
        <td><input type="text" id="unit" value="" size="10"
maxlength="10" /> <span id="eUnit"></span></td>
      </tr>
      <tr>
        <td colspan="2"><input type="submit" value="Calcular"
id="calc"/></td>
      </tr>
      <tr>
        <td align="right">Preço Total:</td>
        <td><input type="text" id="total" value="" size="10"
readonly /></td>
      </tr>
    </table>
    <script src="produto-1.js"></script>
  </body>
</html>
```

produto-1.html

As funções JavaScript foram desenvolvidas no arquivo a seguir.

{JS}

```
desc.addEventListener("blur", validarDescricao);
qtd.addEventListener("blur", validarQuantidade);
unit.addEventListener("blur", validarPreco);
calc.addEventListener("click", calcular);

function validarDescricao() {
  if (desc.value.trim() === "") {
    desc.style.background = "yellow";
    desc.focus();
    eDesc.style.color = "#FF0000";
    eDesc.innerHTML = "Preencha a descrição do produto!";
    return false;
  }
  else {
    desc.style.background = "white";
    eDesc.innerHTML = "";
    return true;
  }
}

function validarQuantidade() {
  let erro = false;
  if (qtd.value.trim() === "") {
```

```
    erro = true;
  }
  else {
    if (isNaN(qtd.value) === true) {
      erro = true;
    }
    else {
      let nQtd = parseInt(qtd.value);
      if (nQtd < 1 || nQtd > 999) {
        erro = true;
      }
    }
  }

  if (erro === true) {
    qtd.style.background = "yellow";
    qtd.focus();
    eQtd.style.color = "#FF0000";
    eQtd.innerHTML = "A quantidade deve ser um número entre 1 e 999!";
  }
  else {
    qtd.style.background = "white";
    eQtd.innerHTML = "";
  }
  return (!erro);
}

function validarPreco() {
  let erro = false;
  if (unit.value.trim() === "") {
    erro = true;
  }
  else {
    if (isNaN(unit.value) === true) {
      erro = true;
    }
    else {
      let nUnit = parseFloat(unit.value);
      if (nUnit <= 0.0) {
        erro = true;
      }
      else {
        unit.value = nUnit.toFixed(2);
      }
    }
  }

  if (erro === true) {
    unit.style.background = "yellow";
    unit.focus();
    eUnit.style.color = "#FF0000";
```

```
    eUnit.innerHTML = "O preço unitário deve ser um número maior que
zero!";
  }
  else {
    unit.style.background = "white";
    unit.innerHTML = ""
  }
  return (!erro);
}

function calcular() {
  if (validarDescricao() && validarQuantidade() && validarPreco()) {
    let nTotal = parseFloat(unit.value) * parseInt(qtd.value);
    total.value = nTotal.toFixed(2);
  }
}
```

produto-1.js

Exercícios

1) Desenvolver uma página Web que receba quatro números reais positivos digitados pelo usuário, devidamente validados e, em seguida, utilizando JavaScript, calcule e exiba a valor da média desses números.
2) Desenvolver uma página Web para uma determinada loja que precisa calcular o preço de venda de um produto. O cálculo deverá ser efetuado através da multiplicação do preço unitário pela quantidade vendida e, posteriormente, subtrair o valor do desconto. Considerar todas as variáveis do tipo de dado real, que serão digitadas pelo usuário e um arquivo externo de JavaScript contendo as funções necessárias para resolver o problema.
3) A Lei de Ohm define que a resistência (R) de um condutor é obtida através da divisão da tensão aplicada (V) dividida pela intensidade de corrente elétrica (A). Desta forma, a partir de uma tensão e corrente, digitadas pelo usuário em um formulário HTML e validadas considerando números inteiros positivos, utilize o JavaScript para calcular e mostrar o valor da resistência na mesma página Web que contém o formulário.
4) Adotando um arquivo externo de JavaScript contendo as funções necessárias para resolver o problema, elaborar uma página Web que a partir de um valor digitado pelo usuário e o respectivo prefixo mostrar a representação do valor nos demais prefixos, por exemplo:

   Digite o valor: **10.000**
   Digite o prefixo: **M**
   Converter
   Resultado:
   10.000.000 k
   10 G
   0,01 T

   Adotar, como referência a tabela mostrada abaixo:

Prefixo	Valor (Decimal)
k (kilo)	10^3 (1000)
M (mega)	10^6 (1,000,000)
G (Giga)	10^9 (1,000,000,000)
T (Tera)	10^{12} (1,000,000,000,000)

5) Considerando que a aprovação de um aluno em determinada disciplina requer uma média final maior ou igual a 6,0 (seis). Elaborar uma página Web que receba duas notas, que devem ser números reais entre 0,0 e 10,0 devidamente validados, realize o cálculo da média, exiba o valor calculado e uma imagem indicando se o aluno está aprovado ou reprovado.
6) A partir dos lados de um retângulo ou quadrado, digitados pelo usuário, elaborar uma página Web que calcule e exiba o valor da área da figura e informe se é um retângulo ou um quadrado, exibindo uma imagem que represente a respectiva figura geométrica. Lembrando que a área é obtida pela multiplicação da base (L) pela altura (A).
7) Considerando um número inteiro digitado pelo usuário, calcular e exibir o valor da sua fatorial em uma página Web. Por exemplo, se o usuário digitar 4, temos que a fatorial é 4 x 3 x 2 x 1, isto é, 24.
8) Considerando três números inteiros digitados pelo usuário em uma página web, determinar e exibir o maior número.
9) Realizar o desenvolvimento de uma página HTML que irá receber as informações através de um formulário e realizar a validação dos dados digitados, considerando que uma seguradora de veículos precisa calcular o valor da apólice com base nas seguintes informações: nome, sexo e ano de nascimento do segurado, marca, modelo, ano de fabricação, valor do veículo e porcentagem do bônus. As seguintes validações deverão ser realizadas no formulário utilizando, para isso, um arquivo JavaScript externo:
    a) O campo sexo deverá aceitar apenas F (Feminino) ou M (Masculino).
    b) O campo ano de nascimento deve aceitar um valor entre 2001 e 1901.
    c) O campo ano de fabricação deverá ser um valor inteiro positivo.
    d) O campo valor do veículo deve ser um número real positivo.
    e) O campo porcentagem do bônus deverá ser um número real entre 0 e 25.

10) Adotando como referência o formulário desenvolvido no exercício anterior, crie uma função JavaScript para determinar o valor da apólice, a partir dos seguintes critérios para cálculo:
    a) Para veículos 2010 ou mais recentes o valor da apólice é de 1,25% do valor do veículo, veículos entre 2009 e 2000 o valor da apólice é de 1,75% do valor do veículo, veículos entre 1999 e 1980 o valor da apólice é de 2,00% e para os demais anos de fabricação devemos utilizar 2,50% como base de cálculo.
    b) Caso o segurado seja do sexo feminino aplicar um desconto 10% sobre o valor calculado no item a, caso contrário, acrescer 5% ao valor calculado no item a.
    c) Se o segurado possuir menos de 30 anos ou mais de 60 anos, acrescentar 20% ao valor da apólice após os cálculos realizados no item a e no item b.
    d) A partir do valor apurado nos itens a, b e c aplicar o desconto com base na porcentagem de bônus informada pelo usuário.

Requisições

Os valores obtidos através de formulários podem ser enviados para outra página. Neste caso, devemos usar o atributo location do objeto window para criar o caminho (URL) da página que deverá ser carregada quando o usuário pressionar o botão Enviar. Acompanhado do endereço da página, devemos acrescentar os parâmetros que serão enviados, assim como os respectivos valores.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Envio de Dados</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Envio de Dados</h1>
    Nome: <input type="text" id="nome" size="20"><br>
    Email: <input type="text" id="email" size="20"><br>
    <input type="button" value="Enviar" id="enviar">
    <script>
    enviar.addEventListener('click', gerarURL)

    function gerarURL() {
      window.location ="receber.html?nome=" + nome.value + "&email=" +
email.value;
    }
    </script>
  </body>
</html>
```

enviar.html

A página que vai receber os dados deverá usar o atributo window.location.href para obter a cadeia de caracteres (string) que contém o endereço (URL) da página e todos os valores que foram enviados. Em seguida, torna-se necessário realizar a manipulação da cadeia de caracteres para a fim de separar e obter os valores desejados de cada um dos parâmetros.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Dados Recebidos</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Dados Recebidos</h1>
    <div id="res"></div>
    <script>
      window.addEventListener('load', lerFormulario);
```

```
      function obterValor(chave) {
        let url = window.location.href;
        let chave = chave.replace(/[\[\]]/g, '\\$&');
        let regex = new RegExp('[?&]' + chave + '(=([^&#]*)|&|#|$)');
        let res = regex.exec(url);
        if (!res)
          return null;
        else if (!res[2])
          return '';
        else
          return decodeURIComponent(res[2].replace(/\+/g, ' '));
      }

      function lerFormulario() {
        let nome = obterValor("nome");
        let email = obterValor("email");
        let texto = "Nome: <b>" + nome + "</b><br>" +
          "E-mail: <b>" + email + "</b><br>";
        res.innerHTML = texto;
      }
      </script>
  </body>
</html>
```

receber.html

A Figura 41 apresenta o resultado do processamento dos parâmetros que foram recebidos pela página.



Figura 41: Dados recebidos



Dica!

Uma ótima referência para ampliar os conhecimentos em JavaScript é a **Mozilla Developer Network**, acesse https://developer.mozilla.org/pt-BR/docs/Web/JavaScript

Primitivas de Desenho: Objetos básicos

Usando JavaScript é possível interagir com o elemento CANVAS do HTML, possibilitando o desenho de objetos gráficos no contexto da página. No exemplo apresentado a seguir será desenhado um quadrado na cor vermelha. Inicialmente a página HTML será criada, contendo o tag CANVAS.

```
<!DOCTYPE html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Primitivas Gráficas</title>
    <link rel="stylesheet" href="./css/canvas0.css">
  </head>

  <body>
    <h1>Primitivas Gráficas</h1>
    <canvas id="desenho"></canvas>
    <script src="./js/canvas0.js"></script>
  </body>
</html>
```

canvas0.html

O CSS utilizado pela página é simples e define uma borda para a área de desenho e a fonte de letra empregada no tag H1.

```
#desenho {
  border: 1px solid black;
}

h1 {
  font-family: Verdana, Geneva, Tahoma, sans-serif;
}
```

canvas0.css

No JavaScript é implementada a função desenhar, onde é definido o tamanho da área ocupada pelo canvas. Em seguida, o método getContext deve ser usado para obter o contexto gráfico que, basicamente, pode ser entendido como a área onde podemos desenhar. Neste contexto, o método fillStyle define a cor de preenchimento a ser usada e o método fillRect desenha o retângulo nas dimensões especificadas, sendo os parâmetros do método a posição no eixo X, posição em Y, comprimento e altura.

```
function desenhar() {
  desenho.width = 400;
```

```
  desenho.height = 300;
  const ctx = desenho.getContext("2d");
  ctx.fillStyle = 'rgb(200 0 0)';
  ctx.fillRect(10, 10, 50, 50);
}
desenhar();
```

canvas0.js

Ainda sobre o código-fonte em JavaScript apresentado no exemplo e considerando a Figura 42, observe que as coordenadas da área de desenho se iniciam do canto superior esquerdo (0, 0), o eixo X define o posicionamento horizontal enquanto o Y realiza o posicionamento vertical. Além disso, o atributo width define o comprimento e o atributo height a altura.

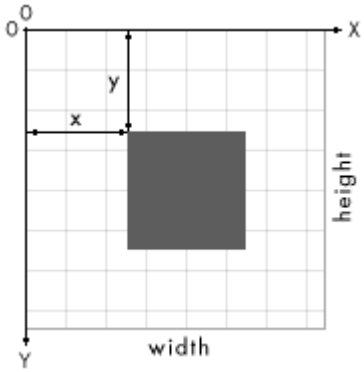


Figura 42: Sistema de coordenadas

Na Figura 43 é possível observar o resultado obtido após a página web ser carregada e renderizada no navegador.

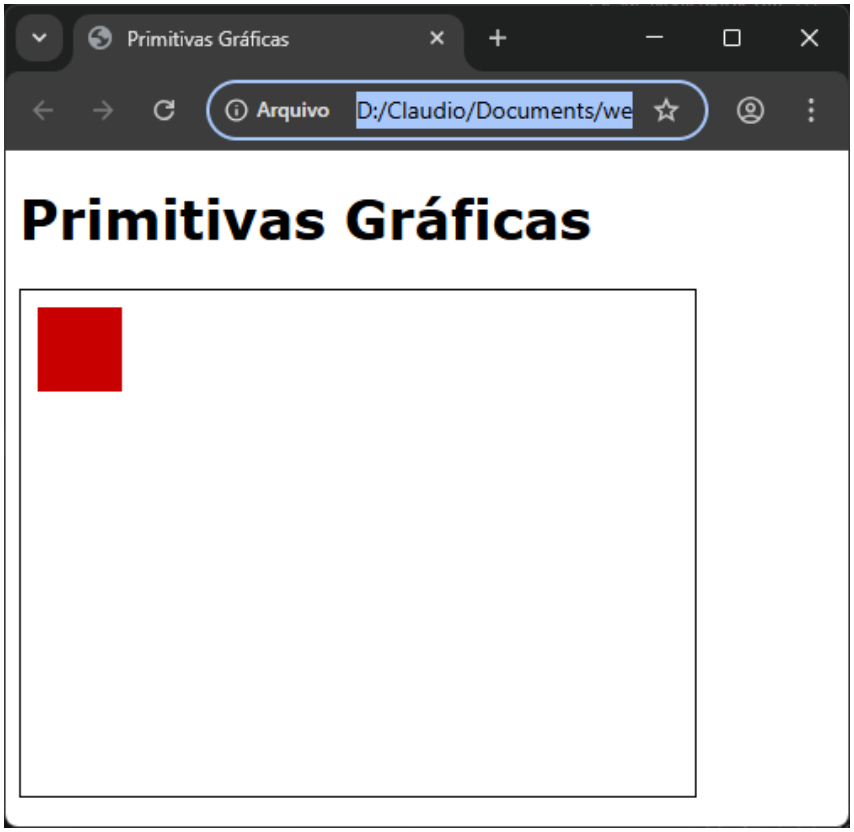


Figura 43: Desenho de um quadrado

A seguir, no arquivo canvas0.js, serão acrescentadas as instruções necessárias para desenhar uma linha na cor verde e com espessura de 5 pixels. No método strokeStyle é definida a cor do contorno, o método moveTo move (sem desenhar) a caneta para as coordenadas X e Y passadas como parâmetros. Por outro lado, o método lineTo traça (desenha) a linha da posição atual da caneta até as coordenadas X e Y especificadas. Completando o exemplo, o método stroke irá, efetivamente, realizar o desenho.

{JS}

```
function desenhar() {
  desenho.width = 400;
  desenho.height = 300;
  const ctx = desenho.getContext("2d");
  ctx.fillStyle = 'rgb(200 0 0)';
  ctx.fillRect(10, 10, 50, 50);

  ctx.strokeStyle = 'rgb(0 150 0)';
  ctx.lineWidth = 5;
  ctx.moveTo(10, 200);
  ctx.lineTo(390, 200);
  ctx.stroke();
}
desenhar();
```

canvas0.js

Na Figura 44 temos o desenho realizado.

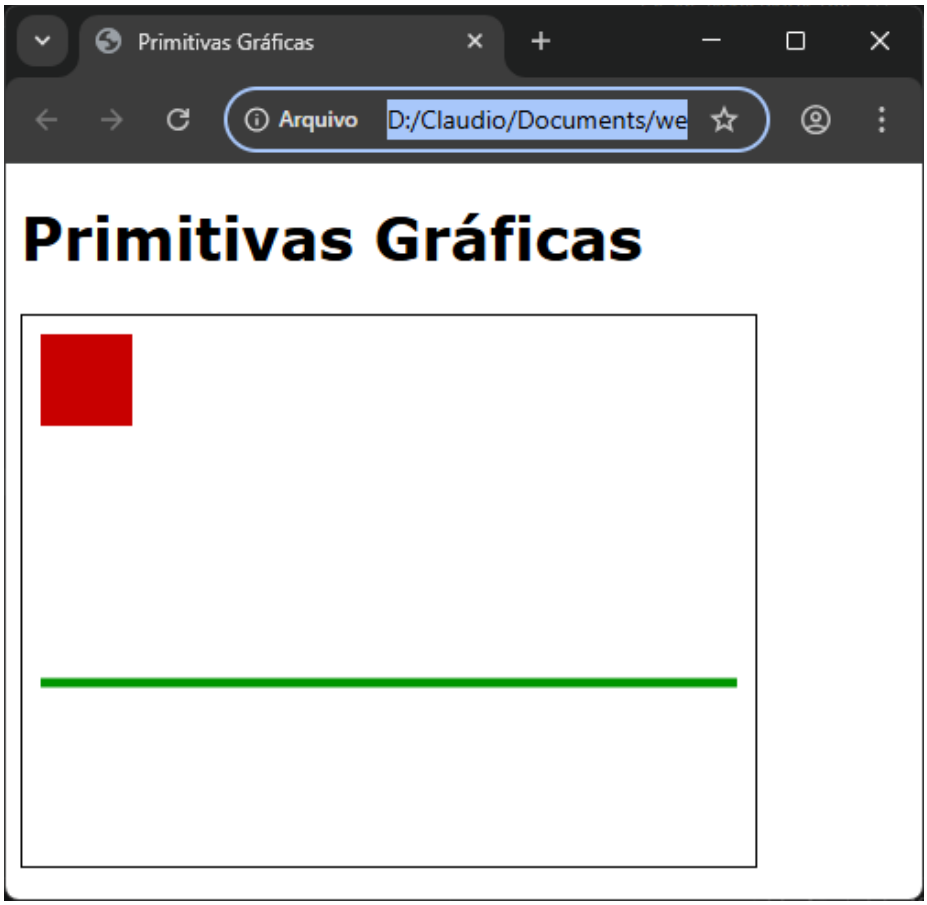


Figura 44: Desenhando uma linha

Agora será realizado o desenho de um círculo na cor azul, conforme é possível observar no código-fonte a seguir. Em primeiro lugar, precisamos usar os métodos beginPath e closePath para delimitar a área do desenho. Caso isso não ocorra, o desenho será fundido ao desenho anterior que, neste exemplo, foi a linha verde.

{JS}

```
function desenhar() {
  desenho.width = 400;
  desenho.height = 300;
  const ctx = desenho.getContext("2d");
  ctx.fillStyle = 'rgb(200 0 0)';
  ctx.fillRect(10, 10, 50, 50);

  ctx.strokeStyle = 'rgb(0 150 0)';
  ctx.lineWidth = 5;
  ctx.moveTo(10, 200);
  ctx.lineTo(390, 200);
  ctx.stroke();

  ctx.beginPath();
  ctx.arc(300, 120, 50, 0, Math.PI * 2);
  ctx.fillStyle = 'blue';
  ctx.closePath();
  ctx.fill();
}
desenhar();
```

canvas0.js

Analisando o programa, o método arc desenhará um arco na cor definida pelo método fillStyle. Os parâmetros do método arc são, respectivamente, coordenadas X e Y do centro do objeto, o raio, ângulo inicial e final. Na Figura 45 é possível ver a área do desenho atualizada.

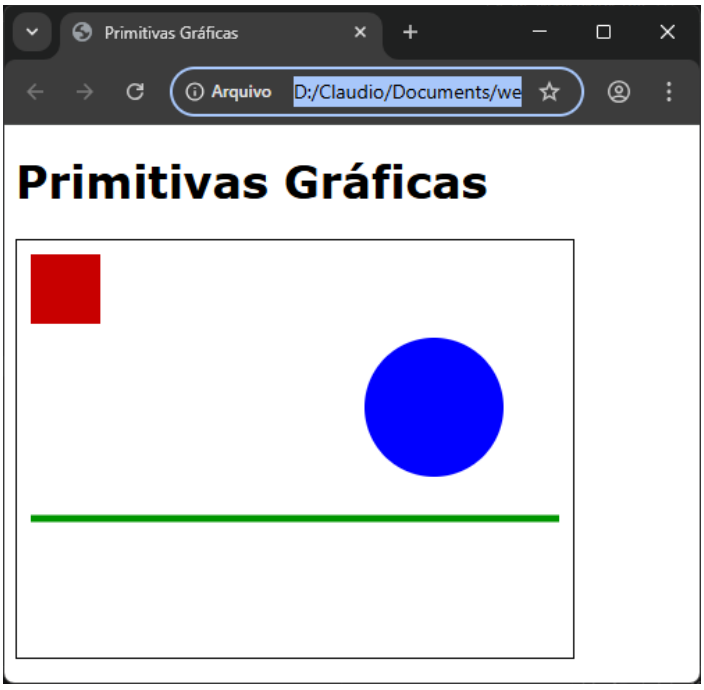


Figura 45: Acrescentando um círculo ao desenho

A área de desenho pode exibir textos, sendo que os métodos fillText e strokeText devem ser usados, respectivamente, para renderizar um texto com preenchimento ou apenas o seu contorno.

{JS}

```
function desenhar() {
  desenho.width = 400;
  desenho.height = 300;
  const ctx = desenho.getContext("2d");
  ctx.fillStyle = 'rgb(200 0 0)';
  ctx.fillRect(10, 10, 50, 50);

  ctx.strokeStyle = 'rgb(0 150 0)';
  ctx.lineWidth = 5;
  ctx.moveTo(10, 200);
  ctx.lineTo(390, 200);
  ctx.stroke();

  ctx.beginPath();
  ctx.arc(300, 120, 50, 0, Math.PI * 2);
  ctx.fillStyle = 'blue';
  ctx.closePath();
  ctx.fill();

  ctx.lineWidth = 1;
  ctx.strokeStyle = 'rgb(0 0 255)';
  ctx.font = "32px arial";
  ctx.strokeText("Olá pessoal!", 10, 130);
}
desenhar();
```

canvas0.js



Figura 46: Inserção de texto no contexto gráfico

Observando o script, a espessura da linha (lineWidth) é definida em 1 pixel, a cor do contorno será azul e a fonte será Arial com tamanho de 32 pixels. Por último, o método strokeText vai exibir o texto no contexto gráfico, conforme mostra a Figura 46.


Pratique!

Inserir um quadrado verde, com o dobro do tamanho ao lado do quadrado vermelho. Desenhar um círculo concêntrico ao círculo azul e com metade do seu tamanho.

Primitivas de Desenho: Imagens e animações

O contexto gráfico possibilita a inserção de imagens e realizar animações. No exemplo a seguir mostraremos, de forma ilustrativa, sem considerar distâncias e proporções reais, a rotação da Lua em torno do nosso planeta. Inicialmente a página HTML será criada, contendo o tag CANVAS.

```
<!DOCTYPE html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Primitivas Gráficas</title>
    <link rel="stylesheet" href="./css/canvas1.css">
  </head>
  <body>
    <h1>Imagens e animações</h1>
    <canvas id="desenho"></canvas>
    <script src="./js/canvas1.js"></script>
  </body>
</html>
```

canvas1.html

O CSS utilizado pela página define um fundo de página na cor preta, a borda para a área de desenho e a fonte de letra e cor usados na exibição do conteúdo da tag H1.

```
#desenho {
  border: 1px solid black;
}

body {
  background-color: black;
}

h1 {
  font-family: Verdana, Geneva, Tahoma, sans-serif;
  color: rgb(0 255 255);
}
```

canvas1.css

No JavaScript temos inicialmente a criação dos objetos lua e terra que serão responsáveis pela exibição das respectivas imagens e a definição da dimensão de cada imagem e da área de exibição. A função iniciar será responsável pela carga dos arquivos de imagem e também definir a função que realizará a animação, a qual é realizada pelo método requestAnimationFrame do objeto window. Este método solicita ao navegador a execução da função de animação a cada atualização da janela, sincronizando com a taxa de atualização do monitor, gerando movimentos suaves e processamento otimizado.

{JS}

```
const lua = new Image();
const terra = new Image();
const tam_lua = 24;
const tam_terra = 96;
const orb_lua = 125;
const largura = 400;
const altura = 300;

function iniciar() {
  lua.src = './img/lua.png';
  terra.src = './img/terra.jpg';
  window.requestAnimationFrame(desenhar);
}

function desenhar() {
  desenho.width = largura;
  desenho.height = altura;
  desenho.style.backgroundColor = 'black';

  const ctx = desenho.getContext("2d");
  ctx.globalCompositeOperation = 'destination-over';
  ctx.clearRect(0, 0, largura, altura);

  ctx.fillStyle = 'rgba(0, 0, 0, 0.4)';
  ctx.strokeStyle = 'rgba(0, 153, 255, 0.4)';
  ctx.save();
  ctx.translate(largura / 2, altura / 2);

  // Lua
  const agora = new Date();
  ctx.rotate(((2 * Math.PI) / 60) * agora.getSeconds() + ((2 *
Math.PI) / 60000) * agora.getMilliseconds());
  ctx.translate(orb_lua, 0);
  ctx.drawImage(lua, (tam_lua / 2) * -1, (tam_lua / 2) * -1, tam_lua,
tam_lua);
  ctx.restore();

  // Órbita da lua
  ctx.beginPath();
  ctx.arc(largura / 2, altura / 2, orb_lua, 0, Math.PI * 2, false);
  ctx.stroke();
```

```
  ctx.drawImage(terra, (largura / 2) - (tam_terra / 2), (altura / 2) -
(tam_terra / 2), tam_terra, tam_terra);

  window.requestAnimationFrame(desenhar);
}

iniciar();
```

canvas1.js

Prosseguindo com a análise do código-fonte, na função desenhar é usado o método globalCompositeOperation, do objeto ctx, para definir como os elementos são desenhados sobre o conteúdo já existente (composição) no contexto gráfico. O método save armazena o estado atual do contexto gráfico em uma pilha, possibilitando alterações e depois retornar ao estado original com o método restore.

O método translate move a origem (0, 0) do sistema de coordenadas para uma nova posição. Os objetos já desenhados não são movidos, mas desloca a grade onde novos objetos serão desenhados, facilitando reposicionar objetos, rotações e escalas. Por sua vez, o método rotate gira o sistema de coordenadas em um determinado ângulo, especificado em radianos ao redor da origem.

Concluindo a função, o método requestAnimationFrame, do objeto window, deve ver executado novamente, iniciando o desenho do próximo quadro (frame) da animação. Na Figura 47 temos a apresentação da página criada.

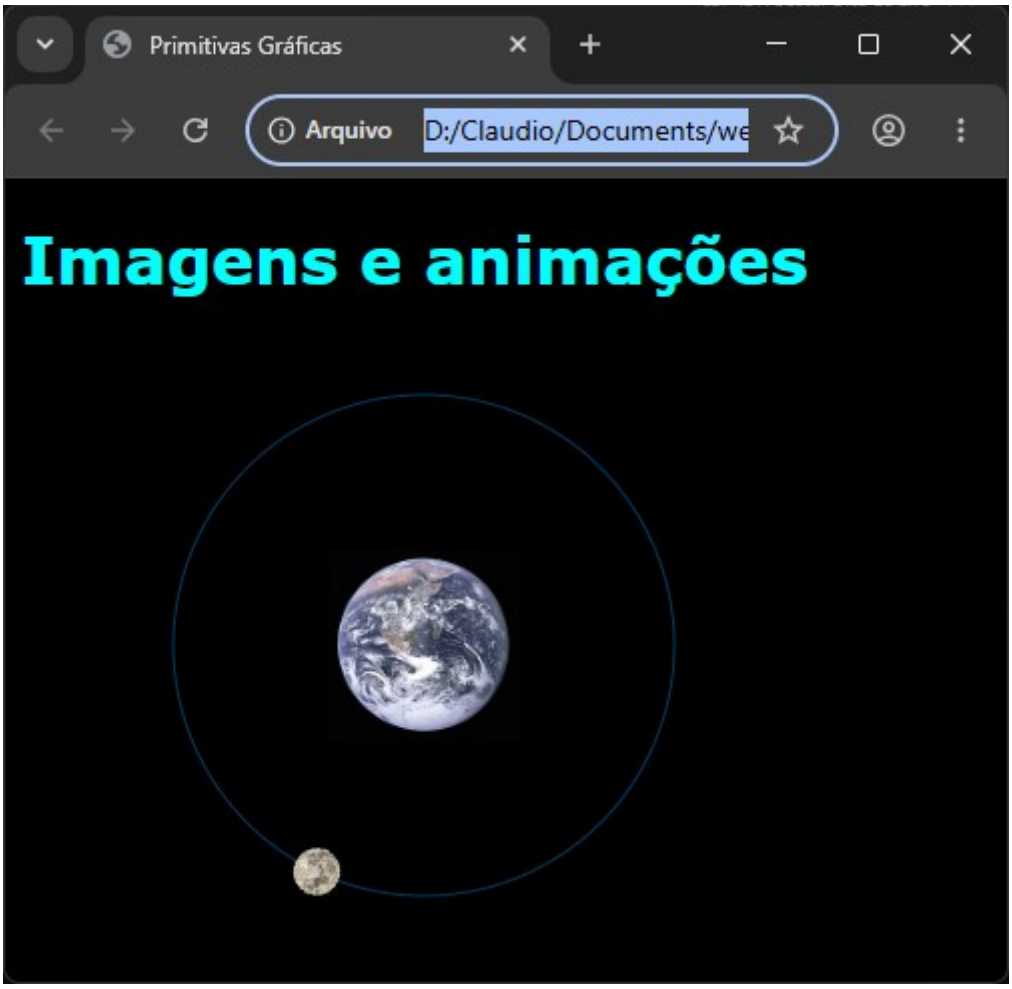


Figura 47: Imagens e animação

Controle do mouse

A programação dos eventos associados ao mouse possibilita muitas possibilidades de interação do usuário com a página web, além de ser um dos fundamentos para a construção de jogos. Com o intuito de ilustrar estes conceitos será criado um jogo de tênis (Figura 48), parecido com o que era disponibilizado nos primeiros consoles de videogames.

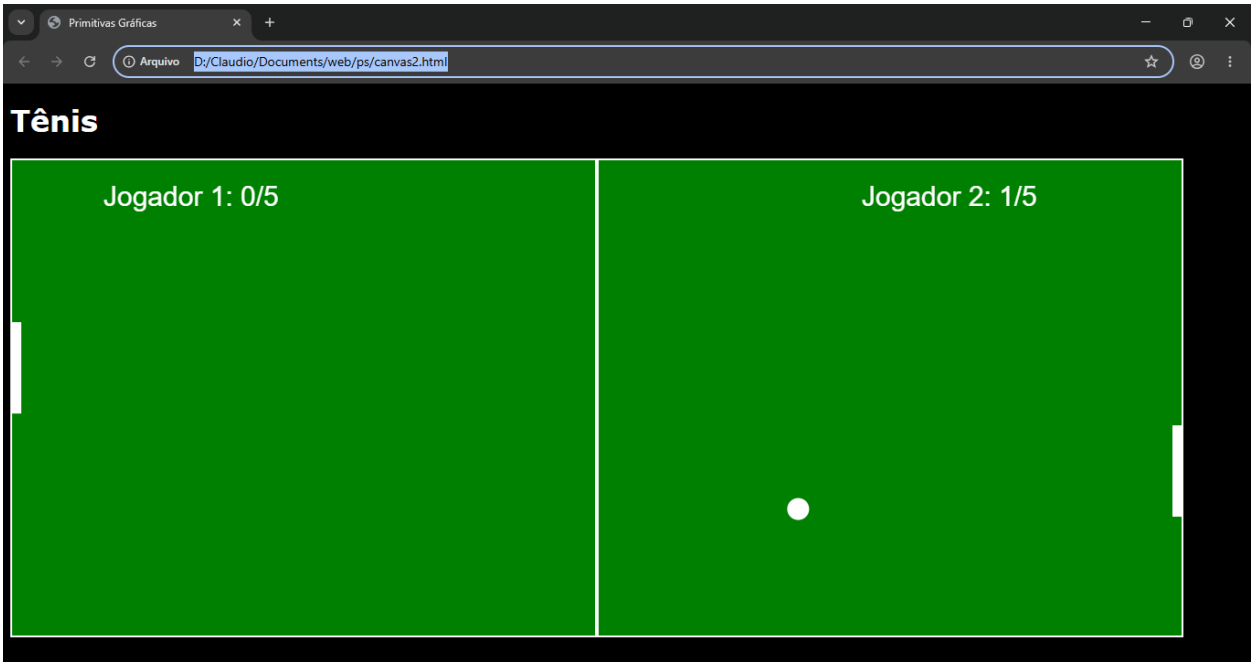


Figura 48: Jogo de tênis

A primeira etapa consiste em criar a página HTML, contendo o tag CANVAS.

```
<!DOCTYPE html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Primitivas Gráficas</title>
    <link rel="stylesheet" href="./css/canvas2.css">
  </head>
  <body>
    <h1>Tênis</h1>
    <canvas id="desenho"></canvas>
    <script src="./js/canvas2.js"></script>
  </body>
</html>
```

canvas2.html

O CSS utilizado pela página define um fundo de página na cor preta, a borda para a área de desenho e a fonte de letra e cor usados na exibição do conteúdo da tag H1.

{CSS}

```
#desenho {
  border: 2px solid white;
}

body {
  background-color: black;
}

h1 {
  font-family: Verdana, Geneva, Tahoma, sans-serif;
  color: white;
}
```

canvas2.css

A implementação deste exemplo é extensa, então, vamos mostrando passo-a-passo a construção do programa e, ao final, apresentamos o arquivo completo. Iniciamos o script com a definição das variáveis e constantes globais, onde temos a especificação da quadra, bola e dos jogadores, além do placar e controle do final do jogo.

{JS}

```
let ctx;

const QUADRA_ALTURA = 520;
const QUADRA_LARGURA = 1280;

let bolaX = 50;
let bolaY = 50;
let bolaVelocX = 15;
let bolaVelocY = 8;

let jogador1Y = 250;
let jogador2Y = 250;
const JOGADOR_ALTURA = 100;
const JOGADOR_LARGURA = 10;

let placarJogador1 = 0;
let placarJogador2 = 0;
const PLACAR_VENCEDOR = 5;

let fimJogo = false;
```

A função posicaoMouse calcula e retorna as coordenadas (X e Y) do mouse no jogo, levando em consideração a posição do ponteiro do mouse (evento), a área do jogo (quadra) e seu posicionamento na página. Para isso é usado o método getBoundingClientRect que retorna o tamanho de um elemento e sua posição relativa a parte visível da janela do navegador (viewport). Também é importante observar o uso do atributo documentElement, do objeto document, ele obterá os atributos do elemento raiz da página HTML (documento).

{JS}

```
function posicaoMouse(evento) {
  let quadra = desenho.getBoundingClientRect();
  let pagina = document.documentElement;
  let mouseX = evento.clientX - quadra.left - pagina.scrollLeft;
  let mouseY = evento.clientY - quadra.top - pagina.scrollTop;
  return {
    x: mouseX,
    y: mouseY
  }
}
```

Quando uma partida termina (determinada pela constante PLACAR_VENCEDOR), a função clickMouse tem como objetivo permitir que uma nova partida seja iniciada, zerando o placar e alterando o valor da variável fimJogo.

{JS}

```
function clickMouse(evento) {
  if (fimJogo) {
    placarJogador1 = 0;
    placarJogador2 = 0;
    fimJogo = false;
  }
}
```

Conforme mostra a função seguinte, o jogo tem início quando a página é carregada. Nela são definidas o tamanho da área do jogo, isto é, o tamanho do elemento CANVAS. Em seguida, é criado um temporizador, usando o método setInterval. Ele será responsável pela movimentação da bola e dos jogadores na quadra. Na sequência, o evento mousemove será tratado pela função de callback que tem como objetivo obter a posição do mouse, usando a função posicaoMouse definida anteriormente. A partir daí ocorre a definição da posição vertical (Eixo Y) do Jogador 1. Completando a função, o evento mousedown (click) será tratado pela função de callback clickMouse, também já especificada.

{JS}

```
window.onload = function () {
  desenho.width = QUADRA_LARGURA;
  desenho.height = QUADRA_ALTURA;
  ctx = desenho.getContext('2d');
  ctx.font = '30px arial';

  var framesPerSecond = 60;
  setInterval(function () {
    moverBola();
    desenhar();
  }, 1000 / framesPerSecond);

  desenho.addEventListener('mousemove',
    function (evento) {
```

```
      let mousePos = posicaoMouse(evento);
      jogador1Y = mousePos.y - (JOGADOR_ALTURA / 2);
  });

  desenho.addEventListener('mousedown', clickMouse);
}
```

A função reiniciarBola deve recolocar a bola novamente em jogo após o um dos jogadores marcar um ponto. Observe também que a função, em primeiro lugar, deve determinar se ocorreu ou não o fim de uma partida.

{JS}

```
function reiniciarBola() {
  if (placarJogador1 >= PLACAR_VENCEDOR || placarJogador2 >=
PLACAR_VENCEDOR) {
    fimJogo = true;
  }
  bolaVelocX = -bolaVelocX / 1.1;
  bolaVelocY = -bolaVelocY / 2;
  bolaX = desenho.width / 2;
  bolaY = desenho.height / 2;
}
```

A função jogadaComputador realiza o movimento autônomo do Jogador 2 (computador) levando em consideração a direção em que a bola foi lançada.

{JS}

```
function jogadaComputador() {
  let jogador2YCentro = jogador2Y + (JOGADOR_ALTURA / 2);
  if (jogador2YCentro < bolaY - 35) {
    jogador2Y += 10;
  } else if (jogador2YCentro > bolaY + 35) {
    jogador2Y -= 10;
  }
}
```

Podemos considerar a função moverBola a principal do jogo, pois, é ela que gerencia um ciclo da partida, realizando inicialmente o movimento do Jogador 2 (computador) a da bola. Na sequência, é verificado se a bola ultrapassou a linha defendida pelo Jogador 2, neste caso, o adversário (Jogador 1) recebe um ponto, ou se ela foi rebatida. Quando isto ocorre, o sentido do movimento da bola é invertido e sua velocidade é aumentada. Em seguida, é realizada a mesma coisa para o Jogador 1. Concluindo a função, o sentido do movimento da bola é invertido quando ela toca uma das linhas laterais (superior ou inferior).

{JS}

```
function moverBola() {
  if (!fimJogo) {
    jogadaComputador();
    bolaX += bolaVelocX;
```

```
    bolaY += bolaVelocY;

    if (bolaX > desenho.width - 10) {
      if (bolaY > jogador2Y - 10 && bolaY < jogador2Y + JOGADOR_ALTURA
+ 10) {
        bolaVelocX = -bolaVelocX * 1.01;
        let deltaY = bolaY - (jogador2Y + JOGADOR_ALTURA / 2);
        bolaVelocY = deltaY * 0.4;
      }
      else {
        placarJogador1++;
        reiniciarBola();
      }
    }
    if (bolaX < 0 + 10) {
      if (bolaY > jogador1Y - 10 && bolaY < jogador1Y + JOGADOR_ALTURA
+ 10) {
        bolaVelocX = -bolaVelocX * 1.1;
        let deltaY = bolaY - (jogador1Y + JOGADOR_ALTURA / 2);
        bolaVelocY = deltaY * 0.4;
      }
      else {
        placarJogador2++;
        reiniciarBola();
      }
    }

    if (bolaY > desenho.height - 12) {
      bolaVelocY = -bolaVelocY;
    }
    if (bolaY < 0 + 12) {
      bolaVelocY = -bolaVelocY;
    }
  }
}
```

O desenho da quadra, do placar, dos jogadores e da bola é realizado pela função desenhar. Sendo que os jogadores e bola são desenhados nas coordenadas definidas dentro da função moverBola, especificada anteriormente. Além disso, quando ocorre o fim do jogo, ocorre a exibição do jogador que venceu partida. Observe que o método measureText é usado para determinar o tamanho do texto a ser exibido, possibilitando, desta maneira, o cálculo para centralizá-lo na área de desenho.

{JS}

```
function desenhar() {
  ctx.fillStyle = 'green';
  ctx.fillRect(0, 0, desenho.width, desenho.height);
  ctx.fillStyle = 'white';
  ctx.fillRect(desenho.width / 2 - 2, 0, 4, desenho.height);

  if (fimJogo) {
```

```
    ctx.fillStyle = 'orange';
    let tamanho = ctx.measureText('Jogador N venceu!');
    if (placarJogador1 >= PLACAR_VENCEDOR) {
      ctx.fillText("Jogador 1 venceu!", (desenho.width -
tamanho.width) / 2, 200);
    }
    else if (placarJogador2 >= PLACAR_VENCEDOR) {
      ctx.fillText("Jogador 2 venceu!", (desenho.width -
tamanho.width) / 2, 200);
    }
    tamanho = ctx.measureText('Click para jogar novamente!');
    ctx.fillText('Click para jogar novamente!', (desenho.width -
tamanho.width) / 2, 400);
  }
  else {
    jogador(0, jogador1Y, JOGADOR_LARGURA, JOGADOR_ALTURA, 'white');
    jogador(desenho.width - JOGADOR_LARGURA, jogador2Y,
JOGADOR_LARGURA, JOGADOR_ALTURA, 'white');
    bola(bolaX, bolaY, 12, 'white');
  }
  ctx.fillStyle = 'white';
  ctx.fillText("Jogador 1: " + placarJogador1 + "/5", 100, 50);
  ctx.fillText("Jogador 2: " + placarJogador2 + "/5", desenho.width -
350, 50);
}
```

Completando o programa, as funções jogador e bola, apresentadas a segui, realizam os respectivos desenhos.

{JS}

```
function jogador(eixoX, eixoY, largura, altura, cor) {
  ctx.fillStyle = cor;
  ctx.fillRect(eixoX, eixoY, largura, altura);
}

function bola(eixoX, eixoY, raio, cor) {
  ctx.fillStyle = cor;
  ctx.beginPath();
  ctx.arc(eixoX, eixoY, raio, 0, Math.PI * 2, true);
  ctx.fill();
}
```

A seguir é apresentado o código-fonte completo.

{JS}

```
let ctx;

const QUADRA_ALTURA = 520;
const QUADRA_LARGURA = 1280;
```

```
let bolaX = 50;
let bolaY = 50;
let bolaVelocX = 15;
let bolaVelocY = 8;

let jogador1Y = 250;
let jogador2Y = 250;
const JOGADOR_ALTURA = 100;
const JOGADOR_LARGURA = 10;

let placarJogador1 = 0;
let placarJogador2 = 0;
const PLACAR_VENCEDOR = 5;

let fimJogo = false;

function posicaoMouse(evento) {
  let quadra = desenho.getBoundingClientRect();
  let pagina = document.documentElement;
  let mouseX = evento.clientX - quadra.left - pagina.scrollLeft;
  let mouseY = evento.clientY - quadra.top - pagina.scrollTop;
  return {
    x: mouseX,
    y: mouseY
  }
}

function clickMouse(evento) {
  if (fimJogo) {
    placarJogador1 = 0;
    placarJogador2 = 0;
    fimJogo = false;
  }
}

window.onload = function () {
  desenho.width = QUADRA_LARGURA;
  desenho.height = QUADRA_ALTURA;
  ctx = desenho.getContext('2d');
  ctx.font = '30px arial';

  var framesPerSecond = 60;
  setInterval(function () {
    moverBola();
    desenhar();
  }, 1000 / framesPerSecond);

  desenho.addEventListener('mousemove',
    function (evento) {
      let mousePos = posicaoMouse(evento);
      jogador1Y = mousePos.y - (JOGADOR_ALTURA / 2);
```

```
    });

  desenho.addEventListener('mousedown', clickMouse);
}

function reiniciarBola() {
  if (placarJogador1 >= PLACAR_VENCEDOR || placarJogador2 >=
PLACAR_VENCEDOR) {
    fimJogo = true;
  }
  bolaVelocX = -bolaVelocX / 1.1;
  bolaVelocY = -bolaVelocY / 2;
  bolaX = desenho.width / 2;
  bolaY = desenho.height / 2;
}

function jogadaComputador() {
  let jogador2YCentro = jogador2Y + (JOGADOR_ALTURA / 2);
  if (jogador2YCentro < bolaY - 35) {
    jogador2Y += 10;
  } else if (jogador2YCentro > bolaY + 35) {
    jogador2Y -= 10;
  }
}

function moverBola() {
  if (!fimJogo) {
    jogadaComputador();
    bolaX += bolaVelocX;
    bolaY += bolaVelocY;

    if (bolaX > desenho.width - 10) {
      if (bolaY > jogador2Y - 10 && bolaY < jogador2Y + JOGADOR_ALTURA
+ 10) {
        bolaVelocX = -bolaVelocX * 1.01;
        let deltaY = bolaY - (jogador2Y + JOGADOR_ALTURA / 2);
        bolaVelocY = deltaY * 0.4;
      }
      else {
        placarJogador1++;
        reiniciarBola();
      }
    }
    if (bolaX < 0 + 10) {
      if (bolaY > jogador1Y - 10 && bolaY < jogador1Y + JOGADOR_ALTURA
+ 10) {
        bolaVelocX = -bolaVelocX * 1.1;
        let deltaY = bolaY - (jogador1Y + JOGADOR_ALTURA / 2);
        bolaVelocY = deltaY * 0.4;
      }
      else {
```

```
        placarJogador2++;
        reiniciarBola();
      }
    }

    if (bolaY > desenho.height - 12) {
      bolaVelocY = -bolaVelocY;
    }
    if (bolaY < 0 + 12) {
      bolaVelocY = -bolaVelocY;
    }
  }
}

function desenhar() {
  ctx.fillStyle = 'green';
  ctx.fillRect(0, 0, desenho.width, desenho.height);
  ctx.fillStyle = 'white';
  ctx.fillRect(desenho.width / 2 - 2, 0, 4, desenho.height);

  if (fimJogo) {
    ctx.fillStyle = 'orange';
    let tamanho = ctx.measureText('Jogador N venceu!');
    if (placarJogador1 >= PLACAR_VENCEDOR) {
      ctx.fillText("Jogador 1 venceu!", (desenho.width -
tamanho.width) / 2, 200);
    }
    else if (placarJogador2 >= PLACAR_VENCEDOR) {
      ctx.fillText("Jogador 2 venceu!", (desenho.width -
tamanho.width) / 2, 200);
    }
    tamanho = ctx.measureText('Click para jogar novamente!');
    ctx.fillText('Click para jogar novamente!', (desenho.width -
tamanho.width) / 2, 400);
  }
  else {
    jogador(0, jogador1Y, JOGADOR_LARGURA, JOGADOR_ALTURA, 'white');
    jogador(desenho.width - JOGADOR_LARGURA, jogador2Y,
JOGADOR_LARGURA, JOGADOR_ALTURA, 'white');
    bola(bolaX, bolaY, 12, 'white');
  }
  ctx.fillStyle = 'white';
  ctx.fillText("Jogador 1: " + placarJogador1 + "/5", 100, 50);
  ctx.fillText("Jogador 2: " + placarJogador2 + "/5", desenho.width -
350, 50);
}

function jogador(eixoX, eixoY, largura, altura, cor) {
  ctx.fillStyle = cor;
  ctx.fillRect(eixoX, eixoY, largura, altura);
}
```

```
function bola(eixoX, eixoY, raio, cor) {
  ctx.fillStyle = cor;
  ctx.beginPath();
  ctx.arc(eixoX, eixoY, raio, 0, Math.PI * 2, true);
  ctx.fill();
}
```

canvas2.js

JavaScript Object Notation (JSON)

JavaScript Object Notation (JSON) ou traduzindo Notação de Objetos JavaScript é uma notação leve para representação e troca de dados. É de fácil compreensão, leitura e escrita. Adota formato texto, é autodescritivo e completamente independente de linguagem.

Uma especificação em JSON está constituída em duas estruturas. A primeira delas, é chamada de objeto e consiste em uma coleção de pares (nome e valor) o que pode caracterizar um objeto, registro, dicionário ou arrays associativos. Um item da coleção deve ser delimitado por chaves. Cada nome é seguido pelo caractere ':' (dois pontos) e, em seguida, o valor deve ser especificado. Vários pares (nome/valor) devem ser separados por vírgula, conforme ilustra o próximo exemplo.

{JSON}

```
{ codigo: 10, descricao: "Televisor", preco: 1990.00 }
```

Desta forma, analisando a estrutura, podemos entender que estamos especificando um par que apresenta o nome 'codigo' que possui o valor 10. Depois temos o par cujo nome é 'descricao' e que tem o valor "Televisor" e o par chamado 'preco' que tem o valor 1990.00.

Manipulação em JavaScript

Utilizar JSON com JavaScript é muito simples, basta referenciar o objeto e a respectiva chave, conforme podemos ver no programa a seguir.

</>

```
<!DOCTYPE html>
<html>
  <head>
    <title>Produtos</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h2>Produtos</h2>
    <div id="res"></div>
    <script>
      window.addEventListener('load', carregar);

      function carregar() {
        var produto = {
          codigo: 10,
          descricao: "Televisor",
          preco: 1990.00
        };
        res.innerHTML = "Dados:" +
          "<br>Código: " + produto.codigo +
          "<br>Descrição: " + produto.descricao +
          "<br>Preço: " + produto.preco;
      }
```

```
    </script>
  </body>
</html>
```

json-basico.html

A notação JSON permite obter informações de maneira dinâmica através do uso de funções. Note no exemplo a seguir que o valor da chave date será obtido através da função hoje que, por sua vez, possibilitará obter a data atual.



```
<!DOCTYPE html>
<html>
  <head>
    <title>Produtos</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h2>Produtos</h2>
    <div id="res"></div>
    <script>
      window.addEventListener('load', carregar);

      function hoje() {
        var agora = new Date();
        return (agora.getDate() + "/" +
          (agora.getMonth() + 1) + "/" +
          agora.getFullYear());
      }

      function carregar() {
        var produto = {
          codigo: 10,
          descricao: "Televisor",
          preco: 1990.00,
          data: function () { return hoje(); }
        };
        res.innerHTML = "Dados do produto: " +
          "<br>Código: " + produto.codigo +
          "<br>Descrição: " + produto.descricao +
          "<br>Preço: " + produto.preco +
          "<br>Data do cadastro: " + produto.data();
      }
      </script>
  </body>
</html>
```

json-funcao.html

A segunda estrutura do JSON é chamada de array e consiste em uma lista ordenada de valores que pode caracterizar array, vetor, lista, sequência ou mesmo uma coleção de valores ordenados. Um array é delimitado por colchetes onde os valores são separados por vírgula, conforme podemos observar no exemplo a seguir.

{JSON}

```
[ 10, 20, 30, 40, 50 ]
```

Um array pode conter objetos, desta maneira, podemos realizar a seguinte notação:

{JSON}

```
[ { codigo: 11, descricao: "Geladeira", preco: 2990.00 }, { codigo:
12, descricao: "Fogão", preco: 840.00 }, { codigo: 13, descricao:
"Computador", preco: 3500.00 } ]
```

Manipular objetos e arrays JSON em JavaScript é uma tarefa bastante simples. Neste próximo programa vamos mostrar como percorrer o array de objetos armazenado na variável produtos.

</>

```
<!DOCTYPE html>
<html>
  <head>
    <title>Produtos</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h2>Produtos</h2>
    <div id="res"></div>
    <script>
      window.addEventListener('load', carregar);

      function carregar() {
        var produtos = [
          { codigo: 11, descricao: "Geladeira", preco: 2990.00 },
          { codigo: 12, descricao: "Fogão", preco: 840.00 },
          { codigo: 13, descricao: "Computador", preco: 3500.00 }
        ];
        res.innerHTML = "Produtos cadastrados:";
        for (var i = 0; i < produtos.length; i++) {
          res.innerHTML += "<br><br>Código: " + produtos[i].codigo +
            "<br>Descrição: " + produtos[i].descricao +
            "<br>Preço: " + produtos[i].preco;
        }
      }
      </script>
  </body>
</html>
```

json-colecao.html

Aprofundando um pouco mais na interação entre JSON e JavaScript, vamos implementar um exemplo de aplicação para a Internet que mostrará como podemos inserir e remover items à um array e exibir o seu conteúdo, considerando um conjunto de produtos que apresentam, como características, código e descrição e preço.

A página HTML, mostrada a seguir, conterá três caixas de texto para que o usuário possa digitar as características do produto e um botão para inserção. A DIV identificada como "tabela" irá, posteriormente, mostrar o conjunto de dados que estarão no array JSON.

```
<html>
  <head>
    <title>Produtos</title>
    <meta charset="UTF-8">
    <link rel="stylesheet" type="text/css" href="json-produto.css" />
  </head>
  <body>
    <h1>Produtos</h1>
    Código: <input type="text" id="cod" size="5" />
    Descrição: <input type="text" id="desc" size="20" />
    Preço: <input type="text" id="prec" size="10"/>
    <input type="button" value="Inserir" id="ins" /><br>
    <h2>Lista de Produtos</h2>
    <div id="tabela"></div>
    <script type="text/javascript" src="json-produto.js"></script>
  </body>
</html>
```

json-produto.html

Em seguida desenvolvemos o arquivo CSS que será responsável pela formatação da tabela.

{CSS}

```
table, td {
  border: 1px solid black;
  padding: 4px;
  border-spacing: 0px;
  border-collapse: collapse;
}
```

json-produto.css

As funções JavaScript serão implementadas em um arquivo a parte, cujo conteúdo é mostrado a seguir. A variável produtos conterá o array. Observe na função inserir, que o método push vai adicionar um objeto ao array. Por outro lado, o método splice será responsável pela remoção de objetos do array produtos. Este método deve receber, como parâmetros, a posição do item no array e quantos itens devem ser removidos que, neste exemplo, deixamos fixo o valor 1. Ou seja, apenas será apagado o item que está na posição que foi passada como parâmetro.

{JS}

```
ins.addEventListener('click', inserir);

var produtos = [];

function inserir() {
  produtos.push({codigo: parseInt(cod.value),
    descricao: desc.value,
    preco: parseFloat(prec.value)});
  mostrar();
}

function remover(i) {
  produtos.splice(i, 1);
  mostrar();
}

function mostrar() {
  console.log(JSON.stringify(produtos));
  var conteudo = "<table>" +
    "<tr><td>Código</td>" +
    "<td>Descrição</td>" +
    "<td>Preço</td></tr>";
  for (var i in produtos) {
    conteudo += "<tr><td><button onclick='remover(" + i +
      ")'><image src='./img/deletar.png' height='12px'></button> " +
      produtos[i].codigo +
      "</td><td> " + produtos[i].descricao +
      "</td><td align='right'>" +
      produtos[i].preco.toFixed(2).replace(".", ",") + "</td></tr>";
  }
  conteudo += "</table>";
  tabela.innerHTML = conteudo;
}
```

json-produto.js

A função mostrar montará uma tabela para exibir os dados que estão armazenados no array produtos. Esta função é chamada sempre que ocorre uma inserção ou remoção no array, de modo que as informações exibidas na página web permaneçam sempre atualizadas. Na Figura 49 temos um exemplo da página criada.

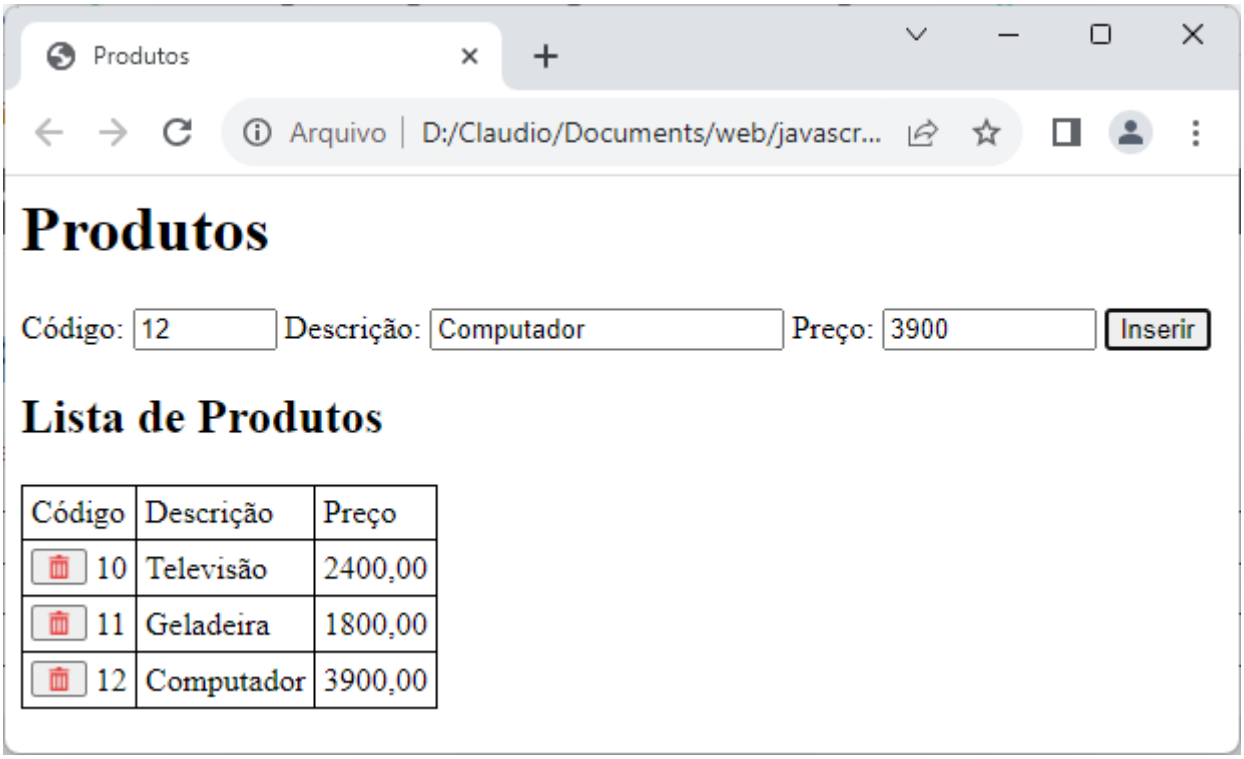


Figura 49: Dados do array JSON



Dica!

Aprenda mais sobre **JSON** acessando https://www.json.org/json-pt.html

Exercícios

1) Considerando um aluno com nome, ra, duas notas e situação acadêmica (aprovado ou reprovado), elaborar uma representação JSON considerando uma sala de aula com 5 alunos.
2) A partir da representação JSON desenvolvida no exercício anterior, elaborar um programa em modo console de texto que mostre o nome dos alunos reprovados.
3) Considerando o estoque de uma loja, elaborar a representação JSON para um conjunto de produtos, onde cada um possui as seguintes características: código, descrição, preço unitário, quantidade e fornecedor.
4) Considerando a representação JSON desenvolvida no exercício anterior, elaborar uma página web com JavaScript no lado cliente que mostre apenas os produtos que apresentem estoque inferior a 10 unidades.

jQuery

Uma biblioteca de funções JavaScript muito utilizada e, provavelmente, a mais popular é a jQuery. Foi criada com o objetivo de simplificar a seleção de elementos, a criação de efeitos visuais e a manipulação de eventos. É uma ferramenta bastante versátil, possibilitando inclusive a criação de plugins.

Os arquivos necessários para o funcionamento do jQuery estão disponíveis em jquery.com e podem ser instalados no seu servidor. Uma página que irá usar o Bootstrap deverá carregar os seguintes arquivos:

```
<!-- jQuery: Carregar através do servidor local ou usar CDN -->
<!-- Versão minimizada do jQuery -->
<script src="js/jquery.min.js"></script>
```

Os arquivos também estão disponíveis através de CDN (Content Delivery Network), desta forma, eles não serão instalados no servidor, sendo acessados diretamente através da conexão com a Internet.

Olá jQuery

O exemplo a seguir apresenta alguns conceitos básicos do jQuery. Em primeiro lugar observe o uso do seletor $() que é utilizado para localizarmos um elemento dentro do HTML. Por exemplo, para identificarmos a caixa de texto que possui o id definido como nome, devemos usar o seletor $("#nome") e para obter o que usuário digitou na caixa de texto, usamos o método val, ou seja, $("#nome").val().

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Exemplo jQuery</title>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <script>
  $(document).ready(function(){
    $('button').click(function() {
      $('#msg').text('Olá, ' + $('#nome').val() + '!');
    });
  });
  </script>
</head>
<body>
  <h1>Exemplo jQuery</h1>
  Qual o seu nome?
  <input type="text" id="nome" /><br />
```

```
  <button>Clique aqui!</button>
  <div id='msg'></div>
</body>
</html>
```

jq-ola.html

Note, neste mesmo exemplo, o uso do evento $(document).ready, ele é gerado após todos os elementos HTML da página serem carregados, sendo que usamos o seletor para referenciar um objeto (document). Desta forma, todas as demais funções do jQuery devem ser executadas a partir dele. Por isso, que o evento associado ao clique do botão "aqui" está inserido dentro de $(document).ready. Também podemos usar o seletor para se referir a tags HTML, por exemplo, $('button'). Na Figura 50 temos um exemplo de página após o usuário preencher o campo nome e pressionar o botão "Clique aqui!".



Figura 50: Olá jQuery

Manipulação de conteúdo e atributo

O jQuery possui métodos que facilitam o acesso e manipulação de conteúdo e atributos de elementos HTML. Os métodos que atuam sobre o conteúdo são: text, que define ou retorna o conteúdo de texto dos elementos selecionados; html, que define ou obtém o conteúdo dos elementos selecionados, incluindo os tags HTML e val, que possibilita definir ou retornar o valor dos campos de formulário. A seguir é demonstrada a utilização dos três métodos.

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Exemplo jQuery</title>
```

```
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <script>
  $(document).ready(function(){
    $('#btTexto').click(function() {
      $('#msg').text($('#conteudo').val());
    });

    $('#btHTML').click(function() {
      $('#msg').html($('#conteudo').val());
    });
  });
  </script>
</head>
<body>
  <h1>Manipulação do conteúdo</h1>
  Conteúdo:
  <input type='text' id="conteudo" value='<b>Negrito</b>' /><br />
  <input type='button' value='Texto' id='btTexto'>
  <input type='button' value='HTML' id='btHTML'>
  <div id='msg'></div>
</body>
</html>
```

jq-conteudo.html

Na Figura 51 temos o conteúdo apresentado na DIV quando o botão Texto é pressionado e na Figura 52 quando o botão HTML foi pressionado.

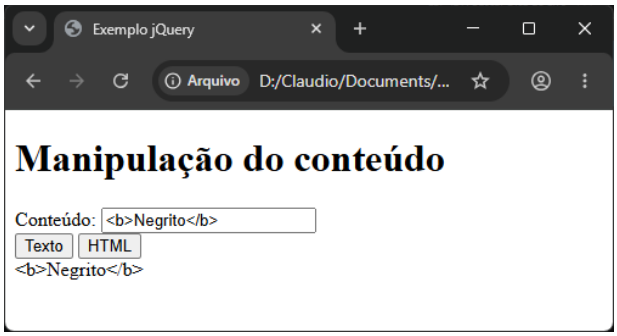


Figura 51: Método text

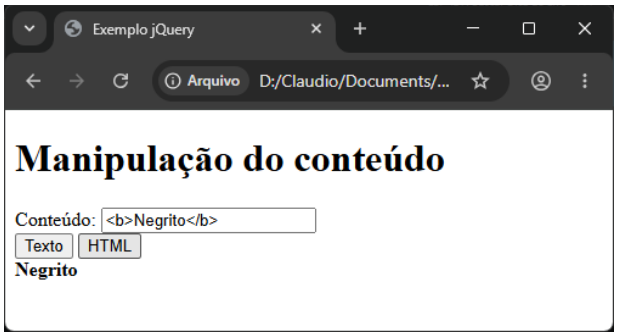


Figura 52: Método html

O método attr permite a alteração dos atributos dos elementos HTML. No exemplo apresentado a seguir, os métodos mouseover e mouseout capturam os eventos de posicionamento do cursor do mouse quando este, respectivamente, está sobre a imagem e quando não está. O método attr será usado para modificar o atributo src do tag IMG, especificando a imagem que será exibida. Note que também será usado o método css para realizar a formatação do tag H1 e do método text para modificar o seu conteúdo.

```
<!DOCTYPE html>
<html lang="pt-br">

<head>
```

```
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Exemplo jQuery</title>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <script>
    $(document).ready(function () {
      $("h1").css("padding", "10px");

      $("#lampada").mouseover(function () {
        $("h1").text("Noite");
        $("h1").css("background-color", "navy");
        $("h1").css("color", "cyan");
        $("#lampada").attr("src", "./img/lamp-lig.png");
      });

      $("#lampada").mouseout(function () {
        $("h1").text("Dia");
        $("h1").css("background-color", "cyan");
        $("h1").css("color", "navy");
        $("#lampada").attr("src", "./img/lamp-desl.png");
      });
    });
  </script>
</head>

<body>
  <h1>Dia e noite</h1>
  <img src="./img/lamp-desl.png" id="lampada">
</body>

</html>
```

jq-atributo.html

O método css possibilita a alteração de múltiplos atributos. Desta forma, a alteração da formação da cor de fundo e do texto do tag H1 do exemplo, também poderia ter sido realizada da maneira apresentada a seguir.



```
$("h1").css({"background-color": "navy", "color", "cyan"});
```

Efeitos

O jQuery apresenta vários efeitos que podem ser aplicados para incrementar a exibição do conteúdo das páginas. No exemplo seguinte aplicaremos os efeitos slideDown e slideUp para, respectivamente, apresentar e esconder o conteúdo da DIV detalhe.

```
<!DOCTYPE html>
<html>

<head>
  <title>Efeitos</title>
  <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"
></script>
  <script>
    $(document).ready(function () {
      $("#titulo").click(function () {
        $("#detalhe").slideDown("slow");
      });

      $("#detalhe").click(function () {
        $("#detalhe").slideUp("slow");
      });

    });
  </script>
  <style>
    #titulo, #detalhe {
      padding: 16px;
      width: 40%;
      text-align: center;
      background-color: #a2ffec;
      border: solid 1px #c3c3c3;
    }

    #detalhe {
      text-align: justify;
      display: none;
    }
  </style>
</head>

<body>
  <h1>Efeitos</h1>
  <div id="titulo">A ligeira raposa marrom. Mais informações...</div>
  <div id="detalhe">A ligeira raposa marrom salta sobre o cão
preguiçoso.
    A ligeira raposa marrom salta sobre o cão preguiçoso.
    A ligeira raposa marrom salta sobre o cão preguiçoso.
    A ligeira raposa marrom salta sobre o cão preguiçoso.
```

```
    A ligeira raposa marrom salta sobre o cão preguiçoso.
    A ligeira raposa marrom salta sobre o cão preguiçoso.
  </div>
</body>

</html>
```

jq-efeitos.html



Pratique!

Altere o exemplo para utilizar os efeitos fadeIn e fadeOut.

AJAX (Asynchronous Javascript and XML)

Podemos entender que AJAX é uma forma de realizar uma requisição ao servidor onde este apenas retornará dados e não uma página HTML completa com marcadores e conteúdo. Desta forma, é possível carregarmos apenas uma parte de uma página, aumentando o desempenho e também possibilitando uma maior interação. O jQuery facilita a implementação de AJAX, oferecendo um método específico para isso, conforme podemos identificar no código-fonte a seguir.

```
<!DOCTYPE html>
<html>
  <head>
    <title>A ligeira raposa</title>
    <meta charset="UTF-8">
    <script src="https://code.jquery.com/jquery-3.4.1.min.js">
    </script>
    <script>
    $(document).ready(function() {
      $("#mais").click(function(){
        $.ajax({url: "jq-ajax.txt", success: function(resultado){
          $("#info").html(resultado);
        }});
      });
    });
    </script>
  </head>
  <body>
    <h1>A ligeira raposa</h1>
    A ligeira raposa marrom salta sobre o cão preguiçoso. <input
type="button" id="mais" value="Mais informações" />
    <div id="info"></div>
  </body>
</html>
```

jq-ajax.html

O arquivo jq-ajax.txt, mostrado a seguir, contém o texto que será acrescentado à página HTML quando o usuário clicar no botão “Mais informações”.

```
A ligeira raposa marrom salta sobre o cão preguiçoso. A ligeira raposa
marrom salta sobre o cão preguiçoso. A ligeira raposa marrom salta
sobre o cão preguiçoso. A ligeira raposa marrom salta sobre o cão
preguiçoso. A ligeira raposa marrom salta sobre o cão preguiçoso. A
ligeira raposa marrom salta sobre o cão preguiçoso.
```

jq-ajax.txt

Na Figura 53 podemos observar a página carregada com o conteúdo inicial, definido no HTML.



Figura 53: Página com o conteúdo inicial

Em seguida, na Figura 54, temos a exibição da página após a requisição AJAX que é responsável por carregar o conteúdo do arquivo-texto na DIV identificada com o id "info".

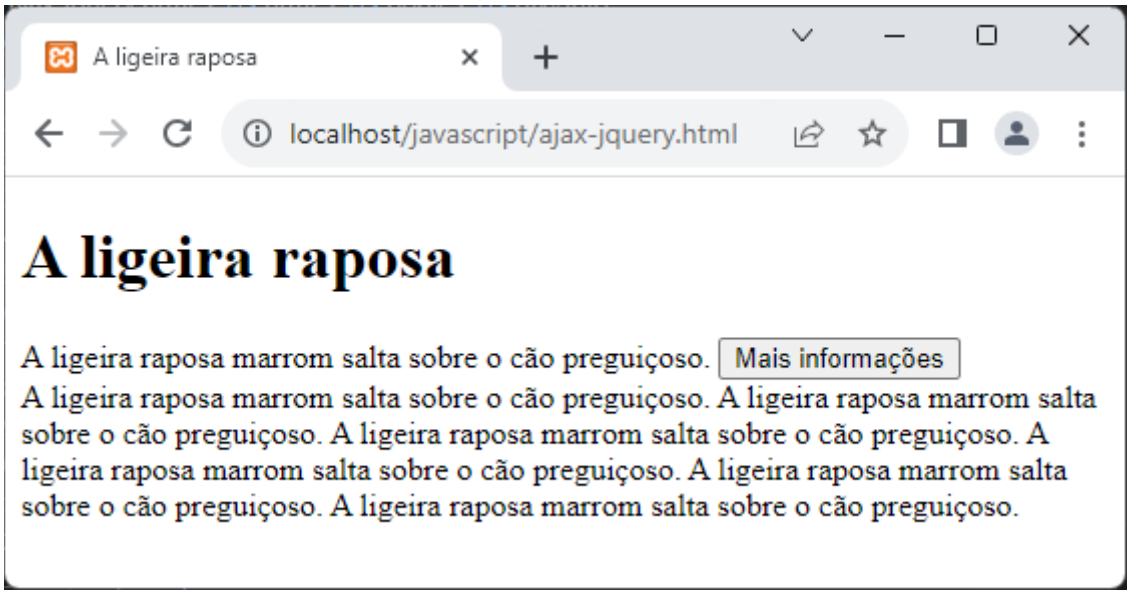


Figura 54: Página após a requisição AJAX

Application Programming Interface (API)

O conceito de Application Programming Interface (API) pode ser entendido como uma maneira de uma aplicação se comunicar com produtos ou serviços, sem precisar saber a forma como eles foram implementados. Desta forma, podemos usar AJAX para realizar requisições para APIs. A página seguinte demonstra a utilização da World Time API, disponível em https://time.now/, para obter as informações de data e hora de determinada região, sendo importante observar que as informações são recebidas em formato JSON.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Data e Hora</title>
    <meta charset="UTF-8">
    <script src="https://code.jquery.com/jquery-3.7.1.min.js">
    </script>
    <script type="text/javascript">
    $(document).ready(function() {
      $("#obter").click(function(){
        $.ajax({
          url:
"https://time.now/developer/api/timezone/America/Sao_Paulo",
          success: function(resultado){
          $("#info").html(JSON.stringify(resultado));
        }});
      });
    });
    </script>
  </head>
  <body>
    <h1>WorldTimeAPI</h1>
    <input type="button" id="obter" value="Obter Data e Hora" />
    <div id="info"></div>
    <a href="https://time.now">World Time API by Time.Now</a>
  </body>
</html>
```

jq-api-0.html

Como o resultado é recebido em JSON, podemos acessar individualmente os seus elementos. Desta maneira, no exemplo seguinte, iremos obter apenas o atributo datetime do objeto resultado. Em seguida, a data e hora são formatadas da maneira que usamos no Brasil.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Data e Hora</title>
    <meta charset="UTF-8">
    <script src="https://code.jquery.com/jquery-3.7.1.min.js">
    </script>
    <script type="text/javascript">
```

```
    $(document).ready(function() {
      $("#obter").click(function(){
        $.ajax({
          url:
"https://time.now/developer/api/timezone/America/Sao_Paulo",
          success: function(resultado){
          var agora = new Date(resultado.datetime);
          var mascara = new Intl.NumberFormat('pt-BR', {
            minimumIntegerDigits: 2,
            minimumFractionDigits: 0
          });
          var data = agora.getDate() + '/' + (agora.getMonth() + 1) +
'/' +agora.getFullYear() + '<br>';
          data += mascara.format(agora.getHours()) + ':' +
mascara.format(agora.getMinutes());
          $("#info").html(data);
        }});
      });
    });
    </script>
  </head>
  <body>
    <h1>World Time API</h1>
    <input type="button" id="obter" value="Obter Data e Hora" />
    <div id="info"></div>
    <a href="https://time.now">World Time API by Time.Now</a>
  </body>
</html>
```

jq-api-1.html

Ampliando os exemplos anteriores, permitiremos que o usuário selecione o fuso horário que deseja obter a data e hora atual. Neste caso, a primeira requisição AJAX busca na API os fusos horários, que são retornados em uma lista e, em seguida, eles são adicionados na caixa de seleção. Após o usuário escolher o fuso horário, ao clicar no botão, nova requisição é realizada à API. Por último, a data e horário são exibidos.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Data e Hora</title>
    <meta charset="UTF-8">
    <script src="https://code.jquery.com/jquery-3.7.1.min.js">
    </script>
    <script type="text/javascript">
    $(document).ready(function() {
      $.ajax({
        url: "https://time.now/developer/api/timezone",
        success: function(resultado){
        for (var item in resultado) {
          var op = new Option(resultado[item]);
          op.text = resultado[item];
```

```
          $("#fuso").append(op);
        }
      }});
      $("#obter").click(function(){
        $.ajax({
          url: "https://time.now/developer/api/timezone/" + $( "#fuso"
).val(),
          success: function(resultado){
          var listaData = resultado.datetime.split(/\D+/);
          var agora = new Date(listaData[0], listaData[1]-1,
listaData[2], listaData[3], listaData[4], listaData[5]);
          var mascara = new Intl.NumberFormat('pt-BR', {
            minimumIntegerDigits: 2,
            minimumFractionDigits: 0
          });
          var data = agora.getDate() + '/' + (agora.getMonth() + 1) +
'/' +agora.getFullYear() + '<br>';
          data += mascara.format(agora.getHours()) + ':' +
mascara.format(agora.getMinutes());
          $("#info").html(data);
        }});
      });
    });
    </script>
  </head>
  <body>
    <h1>World Time API</h1>
    Fuso horário:
    <select id="fuso">
    </select>
    <input type="button" id="obter" value="Obter Data e Hora" />
    <div id="info"></div>
    <a href="https://time.now">World Time API by Time.Now</a>
  </body>
</html>
```

jq-api-2.html

Na Figura 55 temos um exemplo de utilização da página criada.

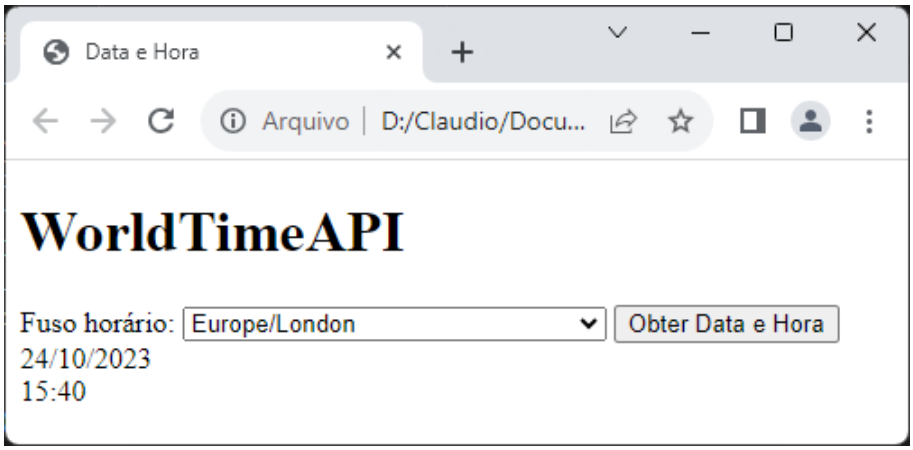


Figura 55: Exibição da data e hora no fuso horário selecionado

O exemplo a seguir vai utilizar a API currency-api, disponível em https://github.com/fawazahmed0/exchange-api, para implementar uma página que realiza conversões entre moedas, sendo que incialmente criamos o arquivo HTML.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Moedas e Criptomoedas</title>
    <meta charset="UTF-8">
    <script src="https://code.jquery.com/jquery-3.7.1.min.js">
    </script>
    <script src="jq-conversao-moeda.js"></script>
  </head>
  <body>
    <h1>Moedas e Criptomoedas</h1>
    <p>Valor: <input type="text" value="1,00" id="valor"></p>
    <p>De:
    <select id="de">
    </select>
    Para:
    <select id="para">
    </select></p>
    <p><input type="button" id="converter" value="Converter" /></p>
    <div id="info"></div>
  </body>
</html>
```

jq-conversao-moeda.html

Na sequência será implementado o arquivo com o código-fonte em JavaScript. Note que serão realizadas duas chamadas à API, sendo que a primeira obtém as moedas disponíveis e as carrega nas duas caixas de seleção. Com base no valor digitado e nas escolhas das moedas, realizadas pelo usuário, a segunda chamada realiza a conversão quando ocorrer o clique no botão converter.

{JS}

```
$(document).ready(function() {
  $.ajax({
    url: "https://cdn.jsdelivr.net/npm/@fawazahmed0/currency-
api@latest/v1/currencies.min.json",
    success: function(resultado){
    for (var item in resultado) {
      var op = new Option(item);
      op.value = item;
      op.text = resultado[item];
      $("#de").append(op);
      op = new Option(item);
      op.value = item;
      op.text = resultado[item];
      $("#para").append(op);
    }
```

```
  }});
  $("#converter").click(function(){
    let endereco =
"https://cdn.jsdelivr.net/npm/@fawazahmed0/currency-api@latest/v1/
currencies/" +
      $("#de").val() + ".json";
    $.ajax({
      url: endereco,
      success: function(resultado){
      let mascara = new Intl.NumberFormat('pt-BR', {
        minimumIntegerDigits: 1,
        minimumFractionDigits: 4
      });
      let cotacao = resultado[$("#de").val()][$("#para").val()];
      let valorDe = parseFloat($("#valor").val().replace(',', '.'));
      let valorPara = valorDe * parseFloat(cotacao);
      let msg = mascara.format(valorDe) + ' ' + $("#de
option:selected").text() +
        ' equivale a ' + mascara.format(valorPara) + ' ' +
        $("#para option:selected").text();
      $("#info").html(msg);
    }});
  });
});
```

jq-conversao-moeda.js

Na Figura 56 temos um exemplo de conversão de um valor em dólares americanos para reais.



Figura 56: Conversão de valores monetários

Exercícios

1) Desenvolver uma página Web, com JavaScript e jQuery, que receba quatro números reais digitados pelo usuário e, em seguida, calcule e exiba a valor da média dos números digitados.
2) Criar uma página Web, com JavaScript e jQuery, para uma determinada loja que precisa calcular o preço de venda de um produto. O cálculo deverá ser efetuado através da multiplicação do preço unitário pela quantidade vendida e, posteriormente, subtrair o valor do desconto. Considerar todas as variáveis do tipo de dado real, que serão digitadas pelo usuário.
3) Desenvolver uma página HTML, com conteúdo definido a seu critério, que apresente o título formatado com a etiqueta <h1>, dois subtítulos formatados com a etiqueta <h2>. O conteúdo associado ao primeiro subtítulo deverá ser apresentado na cor azul, fonte tamanho 12px, alinhado à direita. O conteúdo relacionado ao segundo subtítulo deverá estar alinhado à esquerda. Utilizar jQuery e AJAX para que o conteúdo associado a cada subtítulo possa ser mostrado ou escondido quando se clicar no subtítulo, este conteúdo deverá ser obtido de dois arquivos texto, relacionados aos respectivos subtítulos.
4) Colocar em uma página HTML um botão com o texto “Obter Data e Hora Atuais”. Este botão, quando pressionado, deverá realizar uma requisição AJAX à um servidor da Internet, por exemplo, http://worldclockapi.com/ e exibir em uma <DIV> a data e hora atuais, considerando o fuso horário local.
5) Alterar o exercício anterior de modo que seja possível exibir na <DIV>, além da data e hora atuais, uma imagem que deverá representar dia ou noite, conforme o horário.

Bootstrap

A adoção de frameworks facilita o desenvolvimento de aplicações, pois implementa as funções mais comumente utilizadas em determinada parte do projeto, por exemplo, acesso a banco de dados ou interface com o usuário entre outras.

O Bootstrap é um framework HTML, CSS e JavaScript para desenvolvimento de interfaces responsivas para aplicações Internet e que, desta forma, funciona adequadamente tanto em computadores como em dispositivos móveis.

Os arquivos necessários para o funcionamento do Bootstrap estão disponíveis em getbootstrap.com e podem ser instalados no seu servidor. Uma página que irá usar o Bootstrap deverá carregar os seguintes arquivos:

```
<!—Bootstrap: Carregar através do servidor local ou usar CDN -->
<!-- Versão minimizada do CSS -->
<link rel="stylesheet"
  href="css/bootstrap.min.css">

<!-- Tema CSS opcional -->
<link rel="stylesheet"
  href="css/bootstrap-theme.min.css">

<!-- Versão minimizada do JQuery -->
<script src="js/jquery.min.js"></script>

<!-- Versão minimizada do Boostrap -->
<script src="js/bootstrap.min.js"></script>
```

Os arquivos também estão disponíveis através de CDN (Content Delivery Network), desta forma, eles não serão instalados no seu servidor e serão acessados diretamente através da conexão com a Internet.

Olá Bootstrap

A estrutura básica de uma página com Bootstrap, mostrada no código-fonte a seguir, consiste em realizar algumas definições e carregar as Folhas de Estilo (CSS) e as bibliotecas em JavaScript necessárias.

```
<!DOCTYPE html>
<html lang="pt-br">
  <head>
    <meta charset="utf-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-
scale=1">
    <title>Olá Bootstrap</title>
```

```
    <!—Bootstrap: Carregar através do servidor local ou usar CDN -->
    <!-- Versão minimizada do CSS -->
    <link rel="stylesheet"
      href="css/bootstrap.min.css">

    <!-- Tema CSS opcional -->
    <link rel="stylesheet"
      href="css/bootstrap-theme.min.css">

    <!-- Versão minimizada do JQuery -->
    <script src="js/jquery.min.js"></script>

    <!-- Versão minimizada do Boostrap -->
    <script src="js/bootstrap.min.js"></script>
  </head>
  <body>
    <h1>Olá Pessoal!</h1>
  </body>
</html>
```

bs-ola.html

Analisando a página que foi criada, observe o uso da etiqueta meta para definir a compatibilidade do Bootstrap com o Internet Explorer:

```
<meta http-equiv="X-UA-Compatible" content="IE=edge">
```

A etiqueta meta seguinte define a viewport, ou seja, a área para a exibição da página. Neste caso, o atributo device-width define que toda a área disponível deverá ser usada tanto para smartphones, tablets ou computadores. Esta etiqueta é fundamental para que a página se comporte de maneira responsiva.

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Note que, na sequência, são carregadas as folhas de estilo “bootstrap.min.css” e “bootstrap-theme.min.css”. Também devem ser carregadas as bibliotecas JavaScript JQuery (jquery.min.js) e Bootstrap (bootstrap.min.js).

Na Figura 57 podemos observar a página que foi criada após ela ser carregada no navegador.

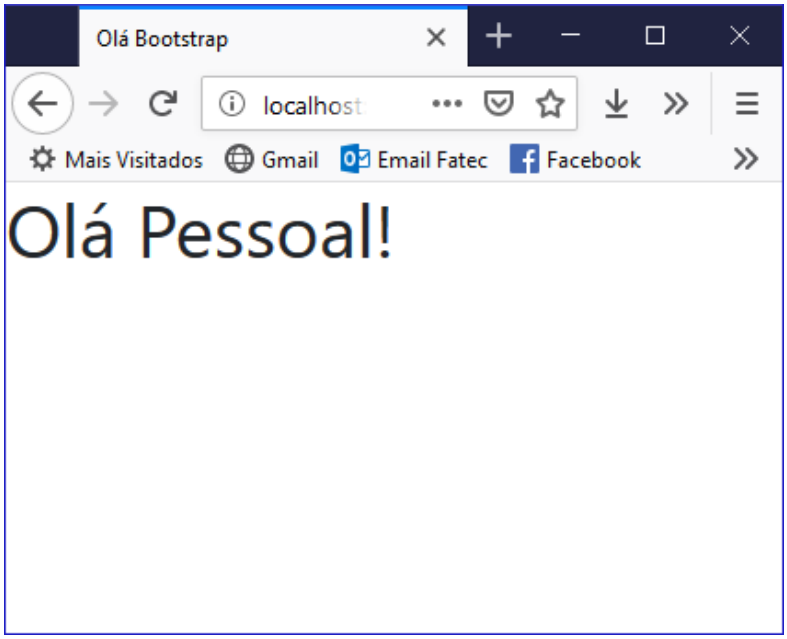


Figura 57: "Olá Bootstrap"

Design responsivo

O design responsivo é uma técnica de design para a Internet que procura fazer com que as páginas renderizarem bem em uma variedade de dispositivos com tamanhas de janela ou tela bastante distintos.

```
<!DOCTYPE html>
<html lang="pt-br">
  <head>
    <title>Página Responsiva</title>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1">
    <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.mi
n.css">
    <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js"
></script>
    <script
src="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.
js"></script>
  </head>
  <body>
    <div class="jumbotron text-center">
      <h1>Página responsiva usando Bootstrap</h1>
      <p>Redimensione a janela para observar o ajuste da página em
áreas com diferentes dimensões</p>
    </div>
```

```
    <div class="container">
      <div class="row">
        <div class="col-sm-4">
          <h3>Coluna 1</h3>
          <p>A ligeira raposa marrom salta sobre o cão preguiçoso. A
ligeira raposa marrom salta sobre o cão preguiçoso. A ligeira raposa
marrom salta sobre o cão preguiçoso.</p>
          <p>A ligeira raposa marrom salta sobre o cão preguiçoso. A
ligeira raposa marrom salta sobre o cão preguiçoso. A ligeira raposa
marrom salta sobre o cão preguiçoso.</p>
        </div>
        <div class="col-sm-4">
          <h3>Coluna 2</h3>
          <p>A ligeira raposa marrom salta sobre o cão preguiçoso. A
ligeira raposa marrom salta sobre o cão preguiçoso. A ligeira raposa
marrom salta sobre o cão preguiçoso.</p>
          <p>A ligeira raposa marrom salta sobre o cão preguiçoso. A
ligeira raposa marrom salta sobre o cão preguiçoso. A ligeira raposa
marrom salta sobre o cão preguiçoso.</p>
        </div>
        <div class="col-sm-4">
          <h3>Coluna 3</h3>
          <p>A ligeira raposa marrom salta sobre o cão preguiçoso. A
ligeira raposa marrom salta sobre o cão preguiçoso. A ligeira raposa
marrom salta sobre o cão preguiçoso.</p>
          <p>A ligeira raposa marrom salta sobre o cão preguiçoso. A
ligeira raposa marrom salta sobre o cão preguiçoso. A ligeira raposa
marrom salta sobre o cão preguiçoso.</p>
        </div>
      </div>
    </div>
  </body>
</html>
```

bs-responsivo.html

Na Figura 58 podemos observar a mesma página sendo exibida em áreas com diferentes dimensões.

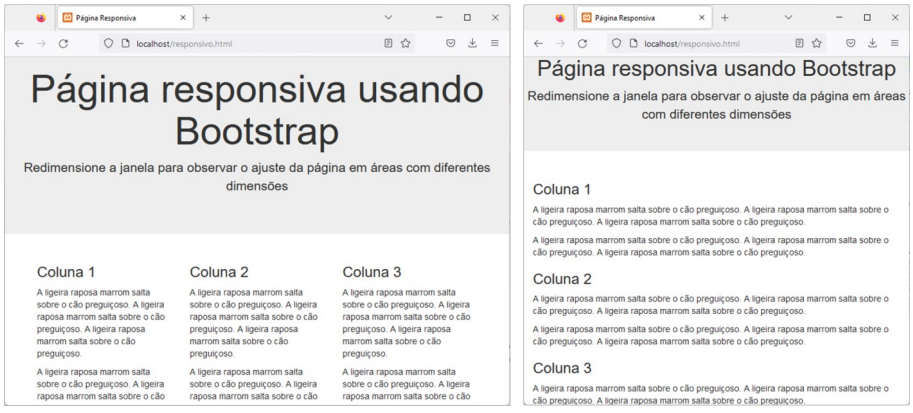


Figura 58: Página responsiva

Formulários com Bootstrap

Vamos modificar a página, criada anteriormente, que realizava a validação dos campos do formulário e cálculo do preço total de produto, só que agora usando o framework Bootstrap. Também vamos retirar o código JavaScript da página HTML e colocá-lo em um arquivo a parte.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-
scale=1">
    <title>Produto</title>

    <!-- Bootstrap -->
    <!-- Versão minimizada do CSS -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.
min.css" rel="stylesheet">

    <!-- Versão minimizada do JQuery -->
    <script
src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

    <!-- Versão minimizada do Boostrap -->
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bu
ndle.min.js"></script>
  </head>
  <body>
    <script src="bs-produto.js" type="text/javascript"></script>
    <div class="container">
      <h1>Produto</h1>
      <div class="form-group">
        Descrição:
        <input type="text" id="desc" class="form-control"
maxlength="20" />
      </div>
      <div class="form-group">
        Quantidade:
        <input type="text" id="qtd" class="form-control" maxlength="3"
/>
      </div>
      <div class="form-group">
        Preço Unitário:
        <input type="text" id="unit" class="form-control"
maxlength="10" />
      </div>
```

```
      <div class="form-group">
        <input type="submit" value="Calcular" class="btn btn-primary"
onclick="calcular()"/>
      </div>
      <div class="form-group">
        Preço Total:
        <input type="text" id="total" class="form-control" readonly />
      </div>

      <!-- Modal -->
      <div class="modal" id="alerta">
        <div class="modal-dialog modal-sm">
          <div class="modal-content">
            <div class="modal-header">
              <h4 class="modal-title">Atenção</h4>
              <button type="button" class="btn-close" data-bs-
dismiss="modal"></button>
            </div>
            <div id="mensagem" class="modal-body">
              Preencha o campo.
            </div>
            <div class="modal-footer">
              <button type="button" class="btn btn-danger" data-bs-
dismiss="modal">Ok</button>
            </div>
          </div>
        </div>
      </div>
    </div>
  </body>
</html>
```

bs-produto.html

Note que vamos explorar os recursos do Bootstrap utilizando a classe form-group para deixar o formulário responsivo e a classe modal para criar uma caixa de mensagem que substituirá a função alert do JavaScript.

Na sequência temos o arquivo que implementa as funções JavaScript.

{JS}

```
function validarDescricao() {
  if ($('#desc').val().trim() === '') {
    $('#desc').attr('background','yellow');
    $('#mensagem').html('Preencha a descrição do produto!');
    $('#alerta').modal('show');
    return false;
  }
  else {
    $('#desc').attr('background','white');
    return true;
  }
}
```

```
function validarQuantidade() {
  var erro = false;
  if ($('#qtd').val().trim() === '') {
    erro = true;
  }
  else {
    if (isNaN($('#qtd').val()) === true) {
      erro = true;
    }
    else {
      var nQtd = parseInt($('#qtd').val());
      if (nQtd < 1 || nQtd > 999) {
        erro = true;
      }
    }
  }

  if (erro === true) {
    $('#qtd').attr('background','yellow');
    $('#mensagem').html("A quantidade deve ser um número entre 1 e
999!");
    $('#alerta').modal('show');
  }
  else {
    $('#qtd').attr('background','white');
  }
  return (!erro);
}

function validarPreco() {
  var erro = false;
  if ($('#unit').val().trim() === "") {
    erro = true;
  }
  else {
    if (isNaN($('#unit').val()) === true) {
      erro = true;
    }
    else {
      var nUnit = parseFloat($('#unit').val());
      if (nUnit <= 0.0) {
        erro = true;
      }
      else {
        $('#unit').val(nUnit.toFixed(2));
      }
    }
  }

  if (erro === true) {
```

```
    $('#unit').attr('background','yellow');
    $('#mensagem').html("O preço unitário deve ser um número maior que
zero!");
    $('#alerta').modal('show');
  }
  else {
    $('#unit').attr('background','white');
  }
  return (!erro);
}

function calcular() {
  if (validarDescricao() && validarQuantidade() && validarPreco()) {
    var nTotal = parseFloat($('#unit').val()) * parseInt($
('#qtd').val());
    $('#total').val(nTotal.toFixed(2));
  }
}
```

bs-produto.js

Na Figura 59 temos a página remodelada com o uso do Bootstrap. Observe a caixa de mensagem sendo exibida, indicando que o campo quantidade deve ser corretamente preenchido. Além do design responsivo é importante notar que a visualização da página fica bem mais fluida e profissional.

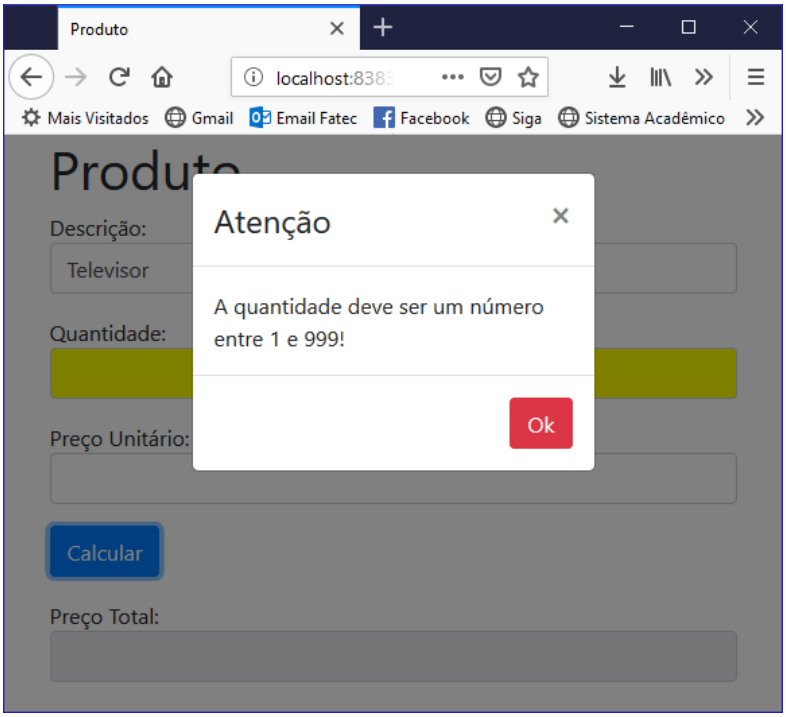


Figura 59: Caixa de diálogo

Exercícios

1) Desenvolver uma página Web, com JavaScript, jQuery e Bootstrap, que receba quatro números reais digitados pelo usuário e, em seguida, verifique se o usuário apenas digitou número, calcule e exiba a valor da média dos números digitados.
2) Criar uma página Web, com JavaScript, jQuery e Bootstrap, para uma determinada loja que precisa calcular o preço de venda de um produto. O cálculo deverá ser efetuado através da multiplicação do preço unitário pela quantidade vendida e, posteriormente, subtrair o valor do desconto. Verificar se todos os campos do formulário foram preenchidos e considerar todas as variáveis do tipo de dado real.
3) Considerar o desenvolvimento de uma Página HTML com BootStrap e jQuery que irá receber as informações através de um formulário, realizar a validação dos dados digitados, considerando que uma seguradora de veículos precisa calcular o valor da apólice com base nas seguintes informações: nome, sexo e ano de nascimento do segurado, marca, modelo, ano de fabricação, valor do veículo e porcentagem do bônus. As seguintes validações deverão ser realizadas no formulário utilizando, para isso, um arquivo JavaScript externo:
    a) O campo sexo deverá aceitar apenas F (Feminino) ou M (Masculino).
    b) O campo ano de nascimento deve aceitar um valor entre 2001 e 1901.
    c) O campo ano de fabricação deverá ser um valor inteiro positivo.
    d) O campo valor do veículo deve ser um número real positivo.
    e) O campo porcentagem do bônus deverá ser um número real entre 0 e 25.
4) Adotando como referência o formulário desenvolvido no exercício anterior, crie uma função que use jQuery para determinar o valor da apólice, a partir dos seguintes critérios para cálculo:
    a) Para veículos 2010 ou mais recentes o valor da apólice é de 1,25% do valor do veículo, veículos entre 2009 e 2000 o valor da apólice é de 1,75% do valor do veículo, veículos entre 1999 e 1980 o valor da apólice é de 2,00% e para os demais anos de fabricação devemos utilizar 2,50% como base de cálculo.
    b) Caso o segurado seja do sexo feminino aplicar um desconto 10% sobre o valor calculado no item a, caso contrário, acrescer 5% ao valor calculado no item a.
    c) Se o segurado possuir menos de 30 anos ou mais de 60 anos, acrescentar 20% ao valor da apólice após os cálculos realizados no item a e no item b.
    d) A partir do valor apurado nos itens a, b e c aplicar o desconto com base na porcentagem de bônus informada pelo usuário.
5) Colocar em uma página HTML contendo jQuery e Bootstrap um botão com o texto “Obter Data e Hora Atuais”. Este botão, quando pressionado, deverá realizar uma requisição AJAX e exibir na página a data e hora atuais obtidas de um servidor da Internet, por exemplo, http://worldclockapi.com/.